JVH Systems

Generated by Doxygen 1.9.5

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

Chapter 4

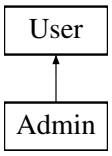
Class Documentation

4.1 Admin Class Reference

This class is the responsible for instantiating an Admin object to distinguish between Users and Admins.

`#include <Admin.h>`

Inheritance diagram for Admin:



Public Member Functions

- Admin (int id)
  Builds a new Admin object.

Static Public Member Functions

- static bool adminStatus (const int id)
  Gets current object status.

Additional Inherited Members

4.1.1 Detailed Description

This class is the responsible for instantiating an Admin object to distinguish between Users and Admins.

Definition at line 17 of file Admin.h.

4.1.2 Constructor & Destructor Documentation

4.1.2.1 Admin()

```
Admin::Admin (
            int id )
```

Builds a new Admin object.

Parameters

id	Admin identificator

Definition at line 8 of file Admin.cpp.

```
00008                   :  User(id){
00009
00010     this->isAdmin = true;
00011
00012 }
```

References User::isAdmin.

4.1.3 Member Function Documentation

4.1.3.1 adminStatus()

```
bool Admin::adminStatus (
            const int id )  [static]
```

Gets current object status.

Parameters

id	User Identifier

Returns

(True) is Admin. (False) is Guest

Definition at line 15 of file Admin.cpp.

```
00015                                   {
00016
00017     try{
00018         int* userData = find(id);
00019         return (userData[ADMIN_COLUMN] == ADMIN_IDENTIFIER);
00020
00021     // User not found
00022     }catch(UserNotFoundException &e){
00023         return false;
```

```
00024     }
00025 }
```

References User::ADMIN_COLUMN, User::ADMIN_IDENTIFIER, and User::find().

Referenced by Login::startLogin().

The documentation for this class was generated from the following files:

- Admin.h
- Admin.cpp

4.2 AI Class Reference

This class will be the responsible for interpreting global commands requested from the main class and checking the proper functioning of the devices. Thus, it will be implemented by the main class and can't live without Device class. Will be settled by the builder.

```
#include <AI.h>
```

Public Member Functions

- AI ()

  Creates a new Automatic Interface ready for controlling device and main program status.
- void startCollectingData ()

  Search for every DataDevice and fills all its data storage with measurements.
- void refreshData ()

  Refresh last data to every device built.
- void throwDemon ()

  Throws a process-like demon, which will be updating the DataDevice measurements while waiting for key-interruption.

Static Public Member Functions

- static void requestOption (const std::string &opt, bool isAdmin)

  Requests an option from main settings interface.

Private Types

- enum {
  TURNDEVICES = 1 , TURNSECURITY = 2 , CHECKSECURITY = 3 , MICROPHONE = 4 ,
  FORCEERROR = 5 , BUILDDEVICE = 6 , ADDUSER = 7 , REMOVEUSER = 8 ,
  RETURNSECURITY = 9 , FORCEBADALLOC = 10 , SHOWUSERS = 11 , MORE = 12 ,
  EXIT = -1 }

Static Private Member Functions

- static void turnDevices ()

  Turns all devices on/off.
- static void turnSecurity ()

  Turns all security on/off.
- static void checkSecurity ()

  Checks and print current security status.
- static void microphone ()

  Sends a message using the microphone.
- static void forceError ()

  Forces an error in all Security devices.
- static void buildDevice ()

  Calls the builder for starting a interface to build a new device in execution time.
- static void recoverSecurity ()

  Recover all security breaches to safe status.
- static void addUser ()

  Calls User class to add a new user.
- static void removeUser ()

  Calls User class to remove a existing user.
- static void forceBadAlloc ()

  Generates a bad_alloc exception in a safe environment.
- static void showAllUsers ()

  Calls to print all users registered.
- static void ensureAdminAccess (bool isAdmin)
- static void signalHandler (int signum)

Private Attributes

- int time_counter

  Used for counting number of time-stamps.

Static Private Attributes

- static std::string TAG = "AI"

  Name of class (for exception throwing)
- static bool abort = false

  Alerts if exit request has been detected.
- static int MAX_DATA = 30

  Sets maximum of data/measurements collected from Data Devices.
- static int SLEEPTIME = 1

  Sets time to sleep between interactions.

4.2.1 Detailed Description

This class will be the responsible for interpreting global commands requested from the main class and checking the proper functioning of the devices. Thus, it will be implemented by the main class and can't live without Device class. Will be settled by the builder.

Definition at line 24 of file AI.h.

4.2.2 Member Enumeration Documentation

4.2.2.1 anonymous enum

```
anonymous enum  [private]
```

Enumerator

TURNDEVICES	
TURNSECURITY	
CHECKSECURITY	
MICROPHONE	
FORCEERROR	
BUILDDEVICE	
ADDUSER	
REMOVEUSER	
RETURNSECURITY	
FORCEBADALLOC	
SHOWUSERS	
MORE	
EXIT	

Definition at line 66 of file AI.h.

```
00066          {
00067         TURNDEVICES = 1,
00068         TURNSECURITY = 2,
00069         CHECKSECURITY = 3,
00070         MICROPHONE = 4,
00071         FORCEERROR = 5,
00072         BUILDDEVICE = 6,
00073         ADDUSER = 7,
00074         REMOVEUSER = 8,
00075         RETURNSECURITY = 9,
00076         FORCEBADALLOC = 10,
00077         SHOWUSERS = 11,
00078         MORE = 12,
00079         EXIT = -1
00080     };
```

4.2.3 Constructor & Destructor Documentation

4.2.3.1 AI()

```
AI::AI ( )
```

Creates a new Automatic Interface ready for controlling device and main program status.

Definition at line 25 of file AI.cpp.

```
00025        {
00026     this->time_counter = 0;
00027     signal(SIGINT, signalHandler);
00028
00029 }
```

References signalHandler(), and time_counter.

4.2.4 Member Function Documentation

4.2.4.1 addUser()

```
void AI::addUser ( )  [static], [private]
```

Calls User class to add a new user.

Definition at line 459 of file AI.cpp.

```
00459                 {
00460
00461     int user, password;
00462     bool isAdmin;
00463     std::string entry;
00464
00465     SimpleDisplay::cout("[AI] Insert a NIF id:  ");
00466     std::getline(std::cin, entry);
00467
00468     try {
00469         user = std::stoi(entry);
00470     }catch (std::exception e){
00471         SimpleDisplay::cout("[AI] Input not valid");
00472         sleep(SLEEPTIME);
00473         return;
00474     }
00475
00476     SimpleDisplay::cout("[AI] Insert a password:  ");
00477
00478     try {
00479         password = std::stoi(entry);
00480     }catch (std::exception e) {
00481         SimpleDisplay::cout("[AI] Input not valid");
00482         sleep(SLEEPTIME);
00483         return;
00484     }
00485
00486     std::getline(std::cin, entry);
00487     do {
00488         SimpleDisplay::cout("[AI] Give the user Admin Status?  [y/n]:  ");
00489         std::getline(std::cin, entry);
00490     }while(entry!= "y" && entry!= "n");
00491
00492     isAdmin = (entry == "y" ?  1 :  0);
00493     User::addUser(user,password,isAdmin);
00494
00495 }
```

References User::addUser(), SimpleDisplay::cout(), and SLEEPTIME.

Referenced by requestOption().

4.2.4.2 buildDevice()

```
void AI::buildDevice ( )  [static], [private]
```

Calls the builder for starting a interface to build a new device in execution time.

Definition at line 386 of file AI.cpp.

```
00386                   {
00387
00388     SimpleDisplay::execBuildName();
00389     std::string name;
00390     std::getline(std::cin,name);
00391     if (name == "EXIT") {
00392
00393         SimpleDisplay::cout("\n[AI] Exit Request");
```

```
00394         sleep(SLEEPTIME);
00395         return;
00396     }
00397 
00398     SimpleDisplay::execBuildType();
00399     std::string type;
00400     std::getline(std::cin,type);
00401 
00402     if (type != "data" && type != "kernel"){
00403         SimpleDisplay::cout("\n[AI] Exit Request");
00404         sleep(SLEEPTIME);
00405         return;
00406     }
00407 
00408     std::string generator;
00409     if (type == "data"){
00410         SimpleDisplay::execBuildGenerator(true);
00411         std::getline(std::cin,generator);
00412     }else{
00413         SimpleDisplay::execBuildGenerator(false);
00414         std::getline(std::cin,generator);
00415     }
00416 
00417     SimpleDisplay::execBuildFinish();
00418     sleep(SLEEPTIME*2);
00419     SimpleDisplay::cout("[AI] Testing Building...\n");
00420     sleep(SLEEPTIME*2);
00421     SimpleDisplay::cout("[AI] Testing Device...\n");
00422     sleep(SLEEPTIME*2);
00423 
00424     DeviceBuilder::clearAll();
00425     DeviceBuilder* newDevice = new DeviceBuilder(name);
00426     DataDevice::numSensors = 0;
00427     KernelDevice::numKerns = 0;
00428     Device::numDevices = 0;
00429     if (type == "data"){
00430         (*newDevice).setAsDataDevice();
00431         (*newDevice).setDataGenerator(generateRandomData);
00432         DeviceBuilder::buildAllDevices();
00433     }else{
00434         (*newDevice).setAsKernelDevice();
00435         (*newDevice).setSecurityGenerator(exampleTest);
00436         DeviceBuilder::buildAllDevices();
00437 
00438     }
00439     SimpleDisplay::cout("[AI] Building SUCCESS\n");
00440     std::cout « "\nPress ENTER to start";
00441     std::cin.get();
00442     std::cin.clear();
00443 
00444 }
```

References DeviceBuilder::buildAllDevices(), DeviceBuilder::clearAll(), SimpleDisplay::cout(), exampleTest(), SimpleDisplay::execBuildFinish(), SimpleDisplay::execBuildGenerator(), SimpleDisplay::execBuildName(), SimpleDisplay::execBuildType(), generateRandomData(), Device::numDevices, KernelDevice::numKerns, DataDevice::numSensors, and SLEEPTIME.

Referenced by requestOption().

4.2.4.3 checkSecurity()

```
void AI::checkSecurity ( )  [static], [private]
```

Checks and print current security status.

Definition at line 323 of file AI.cpp.

```
00323                       {
00324 
00325     for (int i = 0; i < KernelDevice::numKerns; i++) {
00326 
00327         // Acessing Device
00328         KernelDevice* k = &KernelDevice::allKernelDevices[i];
00329 
00330         // Displaying chart
```

```
00331         SimpleDisplay::cout("[AI] [" + k->getName() + "] ");
00332
00333         if (~*(k))
00334             SimpleDisplay::cout("[RUNNING]: ");
00335         else
00336             SimpleDisplay::cout("[DISCONNECTED]: ");
00337
00338         if (!k->securityHasBeenBroken)
00339             SimpleDisplay::cout("Security status correct");
00340         else
00341             SimpleDisplay::cout("[WARNING] Security FAILURE! Please check the logs.");
00342
00343         SimpleDisplay::cout("\n");
00344
00345     }
00346     // ----- Exiting...  -----
00347     std::string none;
00348     SimpleDisplay::cout("Press ENTER to continue");
00349     getline(std::cin,none);
00350
00351 }
```

References KernelDevice::allKernelDevices, SimpleDisplay::cout(), Device::getName(), KernelDevice::numKerns, and KernelDevice::securityHasBeenBroken.

Referenced by requestOption().

4.2.4.4 ensureAdminAccess()

```
void AI::ensureAdminAccess (
            bool isAdmin )  [static], [private]
```

Ensures Administrator Access to a function

Parameters

isAdmin	previous status of user. (True) is Admin. False (is Guest).

Exceptions

InsufficientPermissionsException	Administrator Authentication Failed

Definition at line 244 of file AI.cpp.

```
00244                                       {
00245
00246     // ----- Checking Admin Status -----
00247     if (!isAdmin) {
00248         SimpleDisplay::cout("THIS ACTION REQUIRES ADMINISTRATOR ACCESS");
00249         sleep(SLEEPTIME*2);
00250         throw InsufficientPermissionsException(TAG);
00251     }
00252
00253     // ----- Entering a console and requesting password -----
00254     try {
00255         std::string message;
00256         SimpleDisplay::printRequestPassword();
00257         SimpleDisplay::cout("Keyboard:  ");
00258         std::getline(std::cin,message);
00259
00260         // Checking input
00261         if (message != "0")
00262             throw InsufficientPermissionsException(TAG);
00263
00264         // Invalid Input
00265         }catch(std::exception e){
00266
```

```
00267             SimpleDisplay::cout("INVALID PASSWORD");
00268             sleep(SLEEPTIME);
00269             throw InsufficientPermissionsException(TAG);
00270         }
00271
00272 }
```

References SimpleDisplay::cout(), SimpleDisplay::printRequestPassword(), SLEEPTIME, and TAG.

Referenced by requestOption().

4.2.4.5 forceBadAlloc()

```
void AI::forceBadAlloc ( )  [static], [private]
```

Generates a bad_alloc exception in a safe environment.

Definition at line 520 of file AI.cpp.

```
00520                       {
00521
00522     SimpleDisplay::cout("\nBuilding Secure Environment...");
00523     sleep(SLEEPTIME);
00524     SimpleDisplay::cout("\nGenerating Error...");
00525     try {
00526         std::vector<int> vector;
00527         for (int i = 0; i > -1; i++)
00528             vector.push_back(i);
00529     }catch(std::exception &e) {
00530         SimpleDisplay::cout("\n[WARNING] Environment Failure:  \n-> what():  ");
00531         SimpleDisplay::cout(e.what());
00532         SimpleDisplay::cout("\nReturning...");
00533         sleep(SLEEPTIME*4);
00534     }
00535 }
```

References SimpleDisplay::cout(), and SLEEPTIME.

Referenced by requestOption().

4.2.4.6 forceError()

```
void AI::forceError ( )  [static], [private]
```

Forces an error in all Security devices.

Note

Error can be restored with recoverSecurity()

Definition at line 373 of file AI.cpp.

```
00373                     {
00374
00375     SimpleDisplay::printAIInterface();
00376     SimpleDisplay::cout("Forcing FAILURE in all security devices");
00377
00378     for (int i = 0; i < KernelDevice::numKerns; i++) {
00379         KernelDevice::allKernelDevices[i].securityHasBeenBroken = true;
00380     }
00381     SimpleDisplay::currentDisplay->setSecurityStatus(false);
00382
00383     sleep(SLEEPTIME*2);
00384 }
```

References KernelDevice::allKernelDevices, SimpleDisplay::cout(), SimpleDisplay::currentDisplay, KernelDevice::numKerns, SimpleDisplay::printAIInterface(), KernelDevice::securityHasBeenBroken, SimpleDisplay::setSecurityStatus(), and SLEEPTIME.

Referenced by requestOption().

4.2.4.7 microphone()

```
void AI::microphone ( )  [static], [private]
```

Sends a message using the microphone.

Definition at line 353 of file AI.cpp.

```
00353                   {
00354
00355
00356     try {
00357         std::string message;
00358
00359         SimpleDisplay::cout("\nEnter a message:  ");
00360         getline(std::cin, message);
00361
00362         SimpleDisplay::cout("[AI] SENDING MESSAGE:\n");
00363         SimpleDisplay::cout("|| -> " + message);
00364
00365         sleep(SLEEPTIME);
00366
00367     }catch (std::exception &e){
00368         SimpleDisplay::cout("Input not valid");
00369     }
00370
00371 }
```

References SimpleDisplay::cout(), and SLEEPTIME.

Referenced by requestOption().

4.2.4.8 recoverSecurity()

```
void AI::recoverSecurity ( )  [static], [private]
```

Recover all security breaches to safe status.

Definition at line 446 of file AI.cpp.

```
00446                           {
00447
00448     SimpleDisplay::printAIInterface();
00449     SimpleDisplay::cout("Recovering all Devices to Secure Status...");
00450
00451     for (int i = 0; i < KernelDevice::numKerns; i++) {
00452         KernelDevice::allKernelDevices[i].securityHasBeenBroken = false;
00453     }
00454     SimpleDisplay::currentDisplay->setSecurityStatus(true);
00455
00456     sleep(SLEEPTIME*2);
00457 }
```

References KernelDevice::allKernelDevices, SimpleDisplay::cout(), SimpleDisplay::currentDisplay, KernelDevice::numKerns, SimpleDisplay::printAIInterface(), KernelDevice::securityHasBeenBroken, SimpleDisplay::setSecurityStatus(), and SLEEPTIME.

Referenced by requestOption().

4.2.4.9 refreshData()

```
void AI::refreshData ( )
```

Refresh last data to every device built.

Definition at line 52 of file AI.cpp.

```
00052                       {
00053
00054     // ----- Declarations ------
00055     int numDevices = DataDevice::getNumSensors();
00056
00057     // ----- Refreshing every data device one time ------
00058     for (int i = 0; i < DataDevice::numSensors; i++) {
00059
00060         DataDevice* target = &DataDevice::allDataDevices[i];
00061
00062         // Check if it's active
00063         if (!~(*target)) continue;
00064
00065         // Check if it is full
00066         if (target->collectedData.size() >= MAX_DATA) {
00067
00068             // Remove oldest measurement
00069             target->collectedData.erase(target->collectedData.begin());
00070
00071             // Remove oldest time
00072             target->collectedTimes.erase(target->collectedTimes.begin());
00073
00074         }
00075         // Append new measurement and time
00076         target->appendNewData();
00077     }
00078
00079     // ----- Refreshing every kernel device one time -----
00080     for (int i = 0; i < KernelDevice::numKerns; i++) {
00081
00082         KernelDevice* k = &KernelDevice::allKernelDevices[i];
00083         // Check if it's active
00084         if (!~(*k)) continue;
00085
00086         if (!(*k).securityGenerator() || k->securityHasBeenBroken) {
00087             k->securityHasBeenBroken = true;
00088             SimpleDisplay::currentDisplay->setSecurityStatus(false);
00089         }
00090     }
00091
00092 }
```

References DataDevice::allDataDevices, KernelDevice::allKernelDevices, DataDevice::appendNewData(), DataDevice::collectedData, DataDevice::collectedTimes, SimpleDisplay::currentDisplay, DataDevice::getNumSensors(), MAX_DATA, KernelDevice::numKerns, DataDevice::numSensors, KernelDevice::securityHasBeenBroken, and SimpleDisplay::setSecurityStatus().

Referenced by main(), and throwDemon().

4.2.4.10 removeUser()

```
void AI::removeUser ( )  [static], [private]
```

Calls User class to remove a existing user.

Definition at line 497 of file AI.cpp.

```
00497                       {
00498
00499     std::string id, confirmation;
00500
00501     SimpleDisplay::cout("[" + TAG + "] Insert the id of the user to be removed:  ");
00502     std::getline(std::cin,id);
00503     do {
00504         SimpleDisplay::cout("[" + TAG + "] Are you sure to remove " + id
```

```
00505         + " from the User list?\nThis can't be undone [y/n]:");
00506         std::getline(std::cin,confirmation);
00507     }while(confirmation != "y" && confirmation != "n");
00508 
00509     if (confirmation == "y") {
00510         try {
00511             User::rmUser(std::stoi(id));
00512         } catch (InvalidUserException &e) {
00513             SimpleDisplay::cout("[" + TAG + "] User requested not found");
00514         }
00515     }
00516     sleep(SLEEPTIME);
00517 
00518 }
```

References SimpleDisplay::cout(), User::rmUser(), SLEEPTIME, and TAG.

Referenced by requestOption().

4.2.4.11 requestOption()

```
void AI::requestOption (
            const std::string & opt,
            bool isAdmin )  [static]
```

Requests an option from main settings interface.

Parameters

opt	Option to select
isAdmin	Admin status of current user

Definition at line 134 of file AI.cpp.

```
00134                                                       {
00135     try {
00136         int num = atoi(opt.c_str());
00137 
00138         switch (num) {
00139 
00140             case TURNDEVICES:
00141                 turnDevices();
00142                 break;
00143 
00144             case TURNSECURITY:
00145                 turnSecurity();
00146                 break;
00147 
00148             case CHECKSECURITY:
00149                 checkSecurity();
00150                 break;
00151 
00152             case MICROPHONE:
00153                 microphone();
00154                 break;
00155 
00156             case FORCEERROR:  {
00157                 try {
00158                     AI::ensureAdminAccess(isAdmin);
00159                 }catch (InsufficientPermissionsException &e){
00160                     break;
00161                 }
00162 
00163                 forceError();
00164                 break;
00165             }
00166 
00167             case BUILDDEVICE:{
00168                 try {
00169                     AI::ensureAdminAccess(isAdmin);
```

```
00170                 }catch (InsufficientPermissionsException &e){
00171                     break;
00172                 }
00173 
00174                 buildDevice();
00175                 break;
00176             }
00177 
00178             case ADDUSER:{
00179                 try {
00180                     AI::ensureAdminAccess(isAdmin);
00181                 }catch(InsufficientPermissionsException &e){
00182                     break;
00183                 }
00184                 addUser();
00185                 break;
00186             }
00187 
00188             case REMOVEUSER:{
00189                 try{
00190                     AI::ensureAdminAccess(isAdmin);
00191                 }catch(InsufficientPermissionsException &e){
00192                     break;
00193                 }
00194                 removeUser();
00195                 break;
00196             }
00197 
00198             case RETURNSECURITY:  {
00199                 try {
00200                     AI::ensureAdminAccess(isAdmin);
00201                 }catch (InsufficientPermissionsException &e){
00202                     break;
00203                 }
00204                 recoverSecurity();
00205                 break;
00206             }
00207 
00208             case FORCEBADALLOC:{
00209                 try {
00210                     AI::ensureAdminAccess(isAdmin);
00211                 }catch (InsufficientPermissionsException &e){
00212                     break;
00213                 }
00214                 forceBadAlloc();
00215                 break;
00216             }
00217 
00218             case SHOWUSERS:  {
00219                 try {
00220                     AI::ensureAdminAccess(isAdmin);
00221                 } catch (InsufficientPermissionsException &e){
00222                     break;
00223                 }
00224                 showAllUsers();
00225                 break;
00226             }
00227             case MORE:{
00228                 SimpleDisplay::printAboutUs();
00229                 break;
00230             }
00231 
00232             case EXIT:
00233                 break;
00234         }
00235 
00236 
00237     }catch (std::exception &e){
00238         SimpleDisplay::cout("Option NOT valid");
00239         sleep(1);
00240     }
00241 
00242 }
```

References ADDUSER, addUser(), BUILDDEVICE, buildDevice(), CHECKSECURITY, checkSecurity(), SimpleDisplay::cout(), ensureAdminAccess(), EXIT, FORCEBADALLOC, forceBadAlloc(), FORCEERROR, forceError(), MICROPHONE, microphone(), MORE, SimpleDisplay::printAboutUs(), recoverSecurity(), REMOVEUSER, removeUser(), RETURNSECURITY, showAllUsers(), SHOWUSERS, TURNDEVICES, turnDevices(), TURNSECURITY, and turnSecurity().

Referenced by main().

4.2.4.12 showAllUsers()

```
void AI::showAllUsers ( )  [static], [private]
```

Calls to print all users registered.

Definition at line 554 of file AI.cpp.

```
00554                          {
00555     User::printUsers();
00556
00557     std::cout « "\nPress ENTER to return";
00558     std::cin.get();
00559
00560     //  Clear bad inputs
00561     std::cin.clear();
00562
00563 }
```

References User::printUsers().

Referenced by requestOption().

4.2.4.13 signalHandler()

```
void AI::signalHandler (
            int signum )  [static], [private]
```

Function to end the program if the exit call is raised

Parameters

signum	Exit call

Warning

Function should not be used directly. Will be controlled by AI

Definition at line 537 of file AI.cpp.

```
00537                                  {
00538
00539     // ----- Save all data -----
00540     User::saveUserData();
00541
00542     // ----- Send abortation signal -----
00543     abort = true;
00544     SimpleDisplay::security_abort = true;
00545
00546     // ----- Display EXIT CODE -----
00547     std::cout « "[" + TAG + "] Exit call detected.  Exiting JVH Systems...";
00548     sleep(SLEEPTIME);
00549     std::cout « "\n[" + TAG + "] Thank you for trusting us!\n\n";
00550     exit(0);
00551
00552 }
```

References abort, User::saveUserData(), SimpleDisplay::security_abort, SLEEPTIME, and TAG.

Referenced by AI().

4.2.4.14 startCollectingData()

```
void AI::startCollectingData ( )
```

Search for every DataDevice and fills all its data storage with measurements.

Warning

All devices must be built before using this function.

Definition at line 32 of file AI.cpp.

```
00032                                {
00033
00034     // ----- Intializing ------
00035     std::cout « "[AI] Collecting Data" « std::endl;
00036     int dataCounter = 0;
00037
00038     // ----- Filling every device ------
00039     int numDevices = DataDevice::getNumSensors();
00040     for (int device = 0; device < numDevices; device++)
00041         for (int data = 0; data < MAX_DATA; data++) {
00042             DataDevice::allDataDevices[device].appendNewData();
00043             dataCounter++;
00044         }
00045
00046     // ----- Sending Output ------
00047     std::cout « "[AI] Collected " « dataCounter « " measures" « std::endl;
00048 }
```

References DataDevice::allDataDevices, DataDevice::appendNewData(), DataDevice::getNumSensors(), and MAX_DATA.

Referenced by main().

4.2.4.15 throwDemon()

```
void AI::throwDemon ( )
```

Throws a process-like demon, which will be updating the DataDevice measurements while waiting for key-interruption.

Definition at line 95 of file AI.cpp.

```
00095                        {
00096
00097 #ifdef _WIN32
00098
00099     while (true) {
00100         try {
00101
00102             if (kbhit()) break;
00103             if (abort){
00104                 SimpleDisplay::cout("[WARNING] You should not exit directly from main menu.  Use -1
      instead.\n");
00105                 exit(0);
00106             }
00107             refreshData();
00108             sleep(SLEEPTIME);
00109
00110         } catch (std::exception &e) {
00111             sleep(SLEEPTIME);
00112             continue;
00113         }
00114     }
00115 #else
00116     while (true){
00117     try{
00118
00119         if (kb::kbhit()) break;
```

```
00120         if (abort){
00121             SimpleDisplay::cout("[WARNING] You should not exit directly from main menu.  Use -1
    instead.\n");
00122             exit(0);
00123         }
00124         refreshData();
00125         sleep(SLEEPTIME);
00126
00127     }catch (std::exception e){
00128         sleep(SLEEPTIME);
00129     }
00130 }
00131 #endif
00132 }
```

References abort, SimpleDisplay::cout(), kb::kbhit(), refreshData(), and SLEEPTIME.

Referenced by main().

4.2.4.16 turnDevices()

```
void AI::turnDevices ( )  [static], [private]
```

Turns all devices on/off.

Definition at line 274 of file AI.cpp.

```
00274                     {
00275
00276     // ----- Instantiating IODi for easy device control -----
00277     IODi iodi;
00278
00279     // ----- Turning all Devices ON/OFF -----
00280     for (int i = 0; i < DataDevice::numSensors; i++) {
00281         iodi.setCurrentDevice(i);
00282
00283         if (~DataDevice::allDataDevices[i])
00284             iodi.requestSkill("turnoff");
00285         else
00286             iodi.requestSkill("turnon");
00287
00288     }
00289
00290     // ----- Exiting...  -----
00291     std::string none;
00292     SimpleDisplay::cout("Press ENTER to continue");
00293     getline(std::cin,none);
00294 }
```

References DataDevice::allDataDevices, SimpleDisplay::cout(), DataDevice::numSensors, IODi::requestSkill(), and IODi::setCurrentDevice().

Referenced by requestOption().

4.2.4.17 turnSecurity()

```
void AI::turnSecurity ( )  [static], [private]
```

Turns all security on/off.

Definition at line 296 of file AI.cpp.

```
00296                     {
00297
00298     for (int i = 0; i < KernelDevice::numKerns; i++) {
00299
```

```
00300         // Accessing Device
00301         KernelDevice *k = &KernelDevice::allKernelDevices[i];
00302
00303         // Checking Status
00304         k->isActive = !~(*k);
00305
00306         // Turning ON/OFF
00307         if (~(*k)) {
00308             SimpleDisplay::cout(k->getName() + " turned ON\n");
00309             SimpleDisplay::currentDisplay->setSecurityConnected(true);
00310         }
00311         else {
00312             SimpleDisplay::cout(k->getName() + " turned OFF\n");
00313             SimpleDisplay::currentDisplay->setSecurityConnected(false);
00314         }
00315     }
00316
00317     // ----- Exiting -----
00318     std::string none;
00319     SimpleDisplay::cout("Press ENTER to continue");
00320     getline(std::cin,none);
00321 }
```

References KernelDevice::allKernelDevices, SimpleDisplay::cout(), SimpleDisplay::currentDisplay, Device::getName(), Device::isActive, KernelDevice::numKerns, and SimpleDisplay::setSecurityConnected().

Referenced by requestOption().

4.2.5 Member Data Documentation

4.2.5.1 abort

```
bool AI::abort = false  [static], [private]
```

Alerts if exit request has been detected.

Definition at line 137 of file AI.h.

Referenced by signalHandler(), and throwDemon().

4.2.5.2 MAX_DATA

```
int AI::MAX_DATA = 30  [static], [private]
```

Sets maximum of data/measurements collected from Data Devices.

Definition at line 139 of file AI.h.

Referenced by refreshData(), and startCollectingData().

4.2.5.3 SLEEPTIME

```
int AI::SLEEPTIME = 1  [static], [private]
```

Sets time to sleep between interactions.

Definition at line 141 of file AI.h.

Referenced by addUser(), buildDevice(), ensureAdminAccess(), forceBadAlloc(), forceError(), microphone(), recoverSecurity(), removeUser(), signalHandler(), and throwDemon().

4.2.5.4 TAG

```
std::string AI::TAG = "AI"  [static], [private]
```

Name of class (for exception throwing)

Definition at line 135 of file AI.h.

Referenced by ensureAdminAccess(), removeUser(), and signalHandler().

4.2.5.5 time_counter

```
int AI::time_counter  [private]
```

Used for counting number of time-stamps.

Definition at line 143 of file AI.h.

Referenced by AI().

The documentation for this class was generated from the following files:

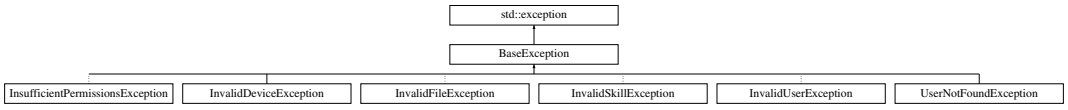
- AI.h
- AI.cpp

4.3 BaseException Class Reference

This class establishes a basic format for JVH exceptions.

```
#include <BaseException.h>
```

Inheritance diagram for BaseException:



Public Member Functions

- BaseException ()

  Builds a new BaseException.
- const char * what ()

  Prints the cause of the exception with the tag of the owner.

Protected Attributes

- std::string tag

  Owner/Causant of the exception.
- std::string exception

  Description of the exception.

4.3.1 Detailed Description

This class establishes a basic format for JVH exceptions.

Definition at line 16 of file BaseException.h.

4.3.2 Constructor & Destructor Documentation

4.3.2.1 BaseException()

```
BaseException::BaseException ( )
```

Builds a new BaseException.

Definition at line 2 of file BaseException.cpp.

```
00002 {}
```

4.3.3 Member Function Documentation

4.3.3.1 what()

```
const char * BaseException::what ( )
```

Prints the cause of the exception with the tag of the owner.

Returns

Exception info

Definition at line 4 of file BaseException.cpp.

```
00004                                {
00005
00006     return ("[" + tag + "] " + exception).c_str();
00007
00008 }
```

References exception, and tag.

4.3.4 Member Data Documentation

4.3.4.1 exception

```
std::string BaseException::exception  [protected]
```

Description of the exception.

Definition at line 38 of file BaseException.h.

Referenced by InsufficientPermissionsException::InsufficientPermissionsException(), InvalidDeviceException::InvalidDeviceException
InvalidFileException::InvalidFileException(), InvalidSkillException::InvalidSkillException(), InvalidUserException::InvalidUserException
UserNotFoundException::UserNotFoundException(), and what().

4.3.4.2 tag

```
std::string BaseException::tag  [protected]
```

Owner/Causant of the exception.

Definition at line 36 of file BaseException.h.

Referenced by InsufficientPermissionsException::InsufficientPermissionsException(), InvalidDeviceException::InvalidDeviceException
InvalidFileException::InvalidFileException(), InvalidSkillException::InvalidSkillException(), InvalidUserException::InvalidUserException
UserNotFoundException::UserNotFoundException(), and what().

The documentation for this class was generated from the following files:

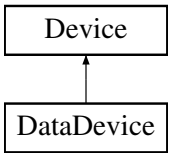
- BaseException.h
- BaseException.cpp

4.4 DataDevice Class Reference

This class will contain specific properties of output devices and a set of functionalities for managing and checking their data. It inherits properties from Device class.

```
#include <DataDevice.h>
```

Inheritance diagram for DataDevice:



Public Member Functions

- DataDevice ()

  Creates a New DataDevice. It uses invalid properties by default. Not recommended.
- DataDevice (const std::string &name, int id, std::vector< void(*)() > *skill, std::string *skillNames)

  Creates a new DataDevice and assign its basic properties.
- void appendNewData ()

  Collect new measurement from main data generator and saves it in data vectors.

Static Public Member Functions

- static int getNumSensors ()

  Returns the current number of DataDevices detected.

Static Public Attributes

- static DataDevice * allDataDevices

  Pointers to all Data Devices built.
- static std::string * allDataDeviceNames

  Pointers to all Data Device names.

Private Member Functions

- void setLimit (int *limitArray)

  Sets a Warning limit for the device. This limit will be supervised by AI.

Static Private Member Functions

- static void setAllDataDevices (DataDevice *allDevices)

  Sets all pointers to DataDevices. Used for general Data Device control.

Private Attributes

- std::vector< int > collectedData

  vector with all measurements collected from the sensor
- std::vector< std::string > collectedTimes

  Vector with all time-stamps of the measurements.
- int(* dataGenerator)()

  Main generator of measurements/data of the sensor.
- bool needsLimit = false

  Distinguish between Data Devices which uses or not Limits.
- int * limit

  Limit to control by AI.

Static Private Attributes

- static int numSensors = 0

  Number of DataDevices stablished.
- static int numSensorsActive = 0

  Number of DataDevices turned on.

Friends

- class DeviceBuilder
- class AI
- class IODi

Additional Inherited Members

4.4.1 Detailed Description

This class will contain specific properties of output devices and a set of functionalities for managing and checking their data. It inherits properties from Device class.

Definition at line 16 of file DataDevice.h.

4.4.2 Constructor & Destructor Documentation

4.4.2.1 DataDevice() [1/2]

```
DataDevice::DataDevice ( )
```

Creates a New DataDevice. It uses invalid properties by default. Not recommended.

Definition at line 24 of file DataDevice.cpp.

```
00024                       {
00025     this->name = "";
00026     this->id = INVALID_DEVICE;
00027     this->isDataDevice = true;
00028
00029 };
```

References Device::INVALID_DEVICE, Device::isDataDevice, and Device::name.

4.4.2.2 DataDevice() [2/2]

```
DataDevice::DataDevice (
            const std::string & name,
            int id,
            std::vector< void(*)()> * skill,
            std::string * skillNames )
```

Creates a new DataDevice and assign its basic properties.

Parameters

name	public name for user
id	private identifier for AI
skillVector	Vector with all skills implemented in the devices as pointers to functions
skillNames	Names for the skills implemented.

Note

Skills and skill-names must be in the same order so the kernel can show them to the user.

Definition at line 32 of file DataDevice.cpp.

```
00032
    {
00033
00034     // ----- Counting Devices ------
00035     std::cout « "[DataDevice] New Device Detected:  0x" « id « std::endl;
00036     numSensors++;
00037
00038     // ----- Property assignation ------
00039     this->name = name;
00040     this->id = id;
00041     this->skills = skillVector;
00042     this->skillNames = skillNames;
00043     this->isDataDevice = true;
00044     this->numSkills = skillVector->size();
00045     this->isActive = true;
00046 }
```

References Device::id, Device::isActive, Device::isDataDevice, Device::name, numSensors, Device::numSkills, Device::skillNames, and Device::skills.

4.4.3 Member Function Documentation

4.4.3.1 appendNewData()

```
void DataDevice::appendNewData ( )
```

Collect new measurement from main data generator and saves it in data vectors.

Definition at line 55 of file DataDevice.cpp.

```
00055                                {
00056     auto clock = std::chrono::system_clock::now();
00057     time_t time = std::chrono::system_clock::to_time_t(clock);
00058     std::string time_stamp = std::ctime(&time);
00059     this->collectedData.push_back(this->dataGenerator());
00060     this->collectedTimes.push_back(time_stamp);
00061 }
```

References collectedData, collectedTimes, and dataGenerator.

Referenced by AI::refreshData(), and AI::startCollectingData().

4.4.3.2 getNumSensors()

```
int DataDevice::getNumSensors ( )  [static]
```

Returns the current number of DataDevices detected.

Returns

Number of detected Sensors

Definition at line 49 of file DataDevice.cpp.

```
00049                                {
00050     return numSensors;
00051 };
```

References numSensors.

Referenced by AI::refreshData(), IODi::requestNumberOfDataDevices(), IODi::setCurrentDevice(), and AI::startCollectingData().

4.4.3.3 setAllDataDevices()

```
void DataDevice::setAllDataDevices (
            DataDevice * allDevices )  [static], [private]
```

Sets all pointers to DataDevices. Used for general Data Device control.

Warning

It can cause conflict with other processes, recommended only with builder implementation.

Definition at line 76 of file DataDevice.cpp.

```
00076                                                    {
00077     allDataDevices = allDevices;
00078     allDataDeviceNames = new std::string[numSensors];
00079
00080     for (int i = 0; i < numSensors; i++){
00081         allDataDeviceNames[i] = allDevices[i].getName();
00082     }
00083 }
```

References allDataDeviceNames, allDataDevices, Device::allDevices, Device::getName(), and numSensors.

Referenced by DeviceBuilder::buildAllDevices().

4.4.3.4 setLimit()

```
void DataDevice::setLimit (
            int * limitArray )  [private]
```

Sets a Warning limit for the device. This limit will be supervised by AI.

Warning

It can cause conflict with other processes, recommended only with builder implementation

Parameters

limitArray	array of values to supervise.

Definition at line 69 of file DataDevice.cpp.

```
00069                                           {
00070     this->limit = limitArray;
00071     this->needsLimit = true;
00072 }
```

References limit, and needsLimit.

4.4.4 Friends And Related Function Documentation

4.4.4.1 AI

```
friend class AI  [friend]
```

Definition at line 20 of file DataDevice.h.

4.4.4.2 DeviceBuilder

```
friend class DeviceBuilder  [friend]
```

Definition at line 19 of file DataDevice.h.

4.4.4.3 IODi

```
friend class IODi  [friend]
```

Definition at line 21 of file DataDevice.h.

4.4.5 Member Data Documentation

4.4.5.1 allDataDeviceNames

```
std::string * DataDevice::allDataDeviceNames  [static]
```

Pointers to all Data Device names.

Definition at line 59 of file DataDevice.h.

Referenced by IODi::requestDeviceNames(), setAllDataDevices(), and IODi::setCurrentDevice().

4.4.5.2 allDataDevices

```
DataDevice * DataDevice::allDataDevices  [static]
```

Pointers to all Data Devices built.

Definition at line 57 of file DataDevice.h.

Referenced by AI::refreshData(), IODi::requestAllStatus(), IODi::requestDeviceSkills(), IODi::requestNumberOfSkills(), IODi::requestSkill(), setAllDataDevices(), AI::startCollectingData(), and AI::turnDevices().

4.4.5.3 collectedData

```
std::vector<int> DataDevice::collectedData  [private]
```

vector with all measurements collected from the sensor

Definition at line 87 of file DataDevice.h.

Referenced by appendNewData(), AI::refreshData(), and IODi::requestSkill().

4.4.5.4 collectedTimes

```
std::vector<std::string> DataDevice::collectedTimes  [private]
```

Vector with all time-stamps of the measurements.

Definition at line 89 of file DataDevice.h.

Referenced by appendNewData(), AI::refreshData(), and IODi::requestSkill().

4.4.5.5 dataGenerator

```
int(* DataDevice::dataGenerator) ()  [private]
```

Main generator of measurements/data of the sensor.

Definition at line 91 of file DataDevice.h.

Referenced by appendNewData().

4.4.5.6 limit

```
int* DataDevice::limit  [private]
```

Limit to control by AI.

Definition at line 95 of file DataDevice.h.

Referenced by setLimit().

4.4.5.7 needsLimit

```
bool DataDevice::needsLimit = false  [private]
```

Distinguish between Data Devices which uses or not Limits.

Definition at line 93 of file DataDevice.h.

Referenced by setLimit().

4.4.5.8 numSensors

```
int DataDevice::numSensors = 0  [static], [private]
```

Number of DataDevices stablished.

Definition at line 110 of file DataDevice.h.

Referenced by AI::buildDevice(), DataDevice(), getNumSensors(), AI::refreshData(), IODi::requestAllStatus(), setAllDataDevices(), and AI::turnDevices().

4.4.5.9 numSensorsActive

```
int DataDevice::numSensorsActive = 0  [static], [private]
```

Number of DataDevices turned on.

Definition at line 112 of file DataDevice.h.

The documentation for this class was generated from the following files:

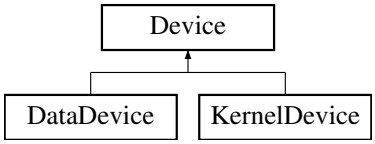
- DataDevice.h
- DataDevice.cpp

4.5 Device Class Reference

This class will contain every device as a general low-level definition, containing the skills, properties and a set of functionalities for managing all devices or modifying them individually. Every device will be built from the builder and the data managed will be managed from AI and IODi. This class will also be the base for constructing DataDevices and KernelDevices.

`#include <Device.h>`

Inheritance diagram for Device:



Public Member Functions

- Device ()

  Builds a New Device. Sets Invalid properties by default. Not recommended.
- Device (const std::string &deviceName, const int deviceId)

  Creates a New Device and assign a identifier.
- void operator[ ] (const int index)

  Uses a built-in device-skill calling it by its name.
- void operator[ ] (const std::string &skillName)

  Uses a built-in device-skill calling it by its position.
- bool operator∼ ()

  Checks if device is active.
- std::string getName ()

  Returns the name of the device.
- int getNumSkills ()

  Returns number of skills of the device.

Static Public Attributes

- static Device ∗ allDevices

  Pointer to array with all Devices built.

Protected Types

- enum { INVALID_DEVICE = -1 }

Protected Attributes

- bool isDataDevice

  Distinguish between Data Devices and other Devices.
- bool isActive

  (True). Is turned ON. (False). Is turned OFF.
- int id

  Kernel number of identification.
- int numSkills

  Number of skills detected for the device.
- std::string * skillNames

  Pointers to names of each skill.
- std::string name

  Name of the device.
- std::vector< void(*)()> * skills

  Pointer to all functions (skills) build.

Static Private Member Functions

- static void setAllDevices (Device *allNewDevices, std::string *allNewDeviceNames, int numDevices)

  Sets all pointers to Devices. Used to general device control, recommended only for DeviceBuilder.
- static void setDeviceCounter (int num)

  Sets number of Detected Devices. Used to general device control, recommended only for DeviceBuilder.

Static Private Attributes

- static const std::string TAG = "Device"

  Name of class (for exception throwing)
- static std::string * allDeviceNames

  Pointers to every Device built.
- static int numDevices

  Number of Devices built.

Friends

- class DeviceBuilder
- class IODi
- class AI

4.5.1 Detailed Description

This class will contain every device as a general low-level definition, containing the skills, properties and a set of functionalities for managing all devices or modifying them individually. Every device will be built from the builder and the data managed will be managed from AI and IODi. This class will also be the base for constructing DataDevices and KernelDevices.

Definition at line 20 of file Device.h.

4.5.2 Member Enumeration Documentation

4.5.2.1 anonymous enum

```
anonymous enum  [protected]
```

Enumerator

INVALID_DEVICE	

Definition at line 112 of file Device.h.

```
00112         {
00113         INVALID_DEVICE = -1  // Change to set invalid ID of devices
00114     };
```

4.5.3 Constructor & Destructor Documentation

4.5.3.1 Device() [1/2]

```
Device::Device ( )
```

Builds a New Device. Sets Invalid properties by default. Not recommended.

Definition at line 22 of file Device.cpp.

```
00022 :  name(""),id(INVALID_DEVICE){};
```

4.5.3.2 Device() [2/2]

```
Device::Device (
            const std::string & deviceName,
            const int deviceId )
```

Creates a New Device and assign a identifier.

Parameters

name	Public name for device
id	Private identifier for device

Definition at line 25 of file Device.cpp.

```
00025 :  name(deviceName),id(deviceId){}
```

4.5.4 Member Function Documentation

4.5.4.1 getName()

```
std::string Device::getName ( )
```

Returns the name of the device.

Returns

Name of device

Definition at line 29 of file Device.cpp.

```
00029                          {
00030     return this->name;
00031 }
```

References name.

Referenced by AI::checkSecurity(), IODi::requestSkill(), DataDevice::setAllDataDevices(), and AI::turnSecurity().

4.5.4.2 getNumSkills()

```
int Device::getNumSkills ( )
```

Returns number of skills of the device.

Returns

Number of skills

Definition at line 33 of file Device.cpp.

```
00033                            {
00034     return this->numSkills;
00035 }
```

References numSkills.

Referenced by IODi::requestDeviceSkills(), and IODi::requestNumberOfSkills().

4.5.4.3 operator[]() [1/2]

```
void Device::operator[] (
            const int index )
```

Uses a built-in device-skill calling it by its name.

Parameters

skillName	Name of the skill

Exceptions

InvalidSkillException	Skill-index not valid/found

Definition at line 38 of file Device.cpp.

```
00038                                    {
```

```
00039     try{
00040         (*this->skills)[index]();
00041
00042     }catch (std::exception){
00043         throw InvalidSkillException(TAG);
00044     }
00045 }
```

References skills, and TAG.

4.5.4.4 operator[]() [2/2]

```
void Device::operator[] (
            const std::string & skillName )
```

Uses a built-in device-skill calling it by its position.

Parameters

index	Position of the skill (order of implementation)

Exceptions

InvalidSkillException	Skill-name not valid

Definition at line 48 of file Device.cpp.

```
00048                                                {
00049     // ----- Set number of skills ------
00050     int skillSize = this->numSkills;
00051
00052     // ----- Find and use Skill ------
00053     for (int i = 0; i < skillSize; i++){
00054         if (skillName == skillNames[i])
00055             try{
00056                 (*this->skills)[i]();
00057                 break;
00058             }catch (std::exception){
00059                 throw InvalidSkillException(TAG);
00060             }
00061
00062     }
00063 };
```

References numSkills, skillNames, skills, and TAG.

4.5.4.5 operator~()

```
bool Device::operator~ ( )
```

Checks if device is active.

Returns

Status of the device. (True) Active. (False) Inactive.

Definition at line 66 of file Device.cpp.

```
00066                         {
00067     return this->isActive;
00068 }
```

References isActive.

4.5.4.6 setAllDevices()

```
void Device::setAllDevices (
            Device * allNewDevices,
            std::string * allNewDeviceNames,
            int numDevices )  [static], [private]
```

Sets all pointers to Devices. Used to general device control, recommended only for DeviceBuilder.

Parameters

allNewDevices	Array with pointers to all Devices
Array	with names for each Device
numDevices	Number of devices to be setted

Definition at line 76 of file Device.cpp.

```
00076                                                                                                {
00077
00078     // ----- Setting Device pointers and names ------
00079     std::cout « "[DEVICES] Collecting all devices:  " « std::endl;
00080     Device::allDevices = allNewDevices;
00081     Device::allDeviceNames = allNewDeviceNames;
00082     Device::numDevices = numDevices;
00083
00084     // ----- Log Success Device ------
00085     for (int i = 0; i < numDevices; i++){
00086         std::cout « "-> (SUCCESS) " « allNewDeviceNames[i] « std::endl;
00087     }
00088
00089 }
```

References allDeviceNames, allDevices, and numDevices.

Referenced by DeviceBuilder::buildAllDevices().

4.5.4.7 setDeviceCounter()

```
void Device::setDeviceCounter (
            int num )  [static], [private]
```

Sets number of Detected Devices. Used to general device control, recommended only for DeviceBuilder.

Parameters

num	Number of devices (total)

Definition at line 92 of file Device.cpp.

```
00092                                    {
00093     Device::numDevices = num;
00094 }
```

References numDevices.

4.5.5 Friends And Related Function Documentation

4.5.5.1 AI

```
friend class AI  [friend]
```

Definition at line 24 of file Device.h.

4.5.5.2 DeviceBuilder

```
friend class DeviceBuilder  [friend]
```

Definition at line 22 of file Device.h.

4.5.5.3 IODi

```
friend class IODi  [friend]
```

Definition at line 23 of file Device.h.

4.5.6 Member Data Documentation

4.5.6.1 allDeviceNames

```
std::string * Device::allDeviceNames  [static], [private]
```

Pointers to every Device built.

Definition at line 87 of file Device.h.

Referenced by setAllDevices().

4.5.6.2 allDevices

```
Device * Device::allDevices  [static]
```

Pointer to array with all Devices built.

Definition at line 73 of file Device.h.

Referenced by DataDevice::setAllDataDevices(), setAllDevices(), and KernelDevice::setAllKernelDevices().

4.5.6.3 id

```
int Device::id  [protected]
```

Kernel number of identification.

Definition at line 123 of file Device.h.

Referenced by DataDevice::DataDevice(), and KernelDevice::KernelDevice().

4.5.6.4 isActive

```
bool Device::isActive  [protected]
```

(True). Is turned ON. (False). Is turned OFF.

Definition at line 121 of file Device.h.

Referenced by DataDevice::DataDevice(), KernelDevice::KernelDevice(), operator~(), IODi::requestSkill(), and AI::turnSecurity().

4.5.6.5 isDataDevice

```
bool Device::isDataDevice  [protected]
```

Distinguish between Data Devices and other Devices.

Definition at line 119 of file Device.h.

Referenced by DataDevice::DataDevice(), and KernelDevice::KernelDevice().

4.5.6.6 name

```
std::string Device::name  [protected]
```

Name of the device.

Definition at line 130 of file Device.h.

Referenced by DataDevice::DataDevice(), getName(), and KernelDevice::KernelDevice().

4.5.6.7 numDevices

```
int Device::numDevices  [static], [private]
```

Number of Devices built.

Definition at line 89 of file Device.h.

Referenced by AI::buildDevice(), setAllDevices(), and setDeviceCounter().

4.5.6.8 numSkills

```
int Device::numSkills  [protected]
```

Number of skills detected for the device.

Definition at line 125 of file Device.h.

Referenced by DataDevice::DataDevice(), getNumSkills(), and operator[ ]().

4.5.6.9 skillNames

```
std::string* Device::skillNames  [protected]
```

Pointers to names of each skill.

Warning

Must have the same order that numSkills

Definition at line 128 of file Device.h.

Referenced by DataDevice::DataDevice(), operator[ ](), IODi::requestDeviceSkills(), and IODi::requestSkill().

4.5.6.10 skills

```
std::vector<void(*)()>* Device::skills  [protected]
```

Pointer to all functions (skills) build.

Definition at line 132 of file Device.h.

Referenced by DataDevice::DataDevice(), and operator[ ]().

4.5.6.11 TAG

```
const std::string Device::TAG = "Device"  [static], [private]
```

Name of class (for exception throwing)

Definition at line 85 of file Device.h.

Referenced by operator[ ]().

The documentation for this class was generated from the following files:

- Device.h
- Device.cpp

4.6 DeviceBuilder Class Reference

This class lets a high-level programmer add easily more devices to the main program. You can use the implemented public functions wherever you need. For adding a new device you can follow this steps:

```
#include <DeviceBuilder.h>
```

Public Member Functions

- DeviceBuilder (const std::string &name="STD_DEVICE")

  Creates a temporal builder for specific device.
- void setSkill (void(∗skill)(), const std::string &skillName)

  Build one skill into the device Builder. All names and skills of each device will be ordered and used in kernel.
- void setLimit (int ∗newLimit)

  Builds limits into the device Builder. All limits will be controlled and implemented in execution by the kernel.
- void setAsDataDevice ()

  Sets type of Device as DataDevice.
- void setAsKernelDevice ()

  Sets type of Device as KernelDevice.
- void setDataGenerator (int(∗dataGenerator)())

  Sets the main data generator for DataDevices.
- void setSecurityGenerator (bool(∗securityGenerator)())

  Sets the security generator for KernelDevices.

Static Public Member Functions

- static void buildAllDevices ()

  Builds every builder instances.
- static SimpleDisplay ∗ buildDisplay ()

  Builds and returns a new SimpleDisplay object with properties according to builder devices.
- static void clearAll ()

  Kills every builder instances.

Private Member Functions

- void buildDKDevice ()

  Builds selected device as data or kernel device into the kernel.

Private Attributes

- std::string name = "STD Device"

  Name of Device.
- std::vector< void(*)()> skillVector

  Skills of the Device.
- std::vector< std::string > skillNamesVector

  Skill-names of each skill.
- int(* dataGenerator)()

  Device's function to generate data.
- bool(* securityGenerator)()

  Kernel's Devices function to check security.
- int id

  Identifier of Device.
- int * limit = nullptr

  Limits of the Device.
- bool isDataDevice

  Type of device. (True) Data Device. (False) Kernel Device.

Static Private Attributes

- static std::vector< DeviceBuilder * > allBuilders

  Vector with all builders instantiated.
- static std::vector< Device > allDevices

  Vector with all Devices instantiated.
- static std::vector< DataDevice > allDataDevices

  Vector with all Data Devices instantiated.
- static std::vector< KernelDevice > allKernelDevices

  Vector with all Kernel Devices instantiated.
- static int deviceCounter

  Number of devices detected.

4.6.1 Detailed Description

This class lets a high-level programmer add easily more devices to the main program. You can use the implemented public functions wherever you need. For adding a new device you can follow this steps:

1. Instantiate a deviceBuilder with DeviceBuilder();
2. Set the type of device with setAsDataDevice() or setAsKernelDevice();
3. Set the main generator (function where the device extracts data) with setDataGenerator() or setSecurityGenerator()
4. (Optional) Add some skills (more functionalities) to your device with setSkill().
5. (Optional) Set a warning if measurement surpasses a limit with setLimit()
6. Repeat this process with as many devices as you need.
7. Use buildAllDevices() to build every instance.

Note

This class must be inited by the main class and will configure 3 classes. First, the builder will set every input device into the main class and set their corresponding global commands. Then it will build every output device into the Device class assigning the high-level functionalities and properties. Finally, the builder will assign AI properties of devices into the AI class if requested. When the relationships between high and low layer are established, the builder can be destroyed.

Definition at line 34 of file DeviceBuilder.h.

4.6.2 Constructor & Destructor Documentation

4.6.2.1 DeviceBuilder()

```
DeviceBuilder::DeviceBuilder (
            const std::string & name = "STD_DEVICE" )
```

Creates a temporal builder for specific device.

Parameters

name	Name for the device.

Note

ID is assigned by kernel.

Warning

Device must be set as kernel or data device for being able to be built.

Definition at line 96 of file DeviceBuilder.cpp.

```
00096                                                {
00097
00098     // ----- Name Assign ------
00099     std::cout « "[BUILDER] Building " « name « std::endl;
00100     this->name = name;
00101
00102     // ----- Storing Builder Pointer------
00103     DeviceBuilder* devicePointer = this;
00104     allBuilders.push_back(devicePointer);
00105
00106     // ----- Settling Device ID ------
00107     enum {FANCY_ID = 55324,FANCY_SUM = 37};
00108     //Shhh..  It looks cooler with strange nums
00109     this->id = deviceCounter + FANCY_ID;
00110     deviceCounter += FANCY_SUM;
00111
00112 };
```

References allBuilders, deviceCounter, and name.

4.6.3 Member Function Documentation

4.6.3.1 buildAllDevices()

```
void DeviceBuilder::buildAllDevices ( )  [static]
```

Builds every builder instances.

Warning

This function should only be called before starting or resetting the main program.

Definition at line 37 of file DeviceBuilder.cpp.

```
00037                                   {
00038
00039     // ----- Initializing ------
00040     std::cout « "[BUILDER] Sending " « allBuilders.size() « " Devices to Kernel" « std::endl;
00041
00042     // ----- Declarations ------
00043     std::string *deviceNames = new std::string[allBuilders.size()]; // Array to Store Device names
00044     int numBuilders = allBuilders.size();
00045
00046     // ----- Building All Devices Saved as Kernel or Data Device ------
00047     for (int i = 0; i < numBuilders; i++){
00048
00049         // Saving device name in array
00050         deviceNames[i] = (*allBuilders[i]).name;
00051
00052         // Building as Data or Kernel Device
00053         (*allBuilders[i]).buildDKDevice();
00054     }
00055
00056     std::cout « "[BUILDER] Optimizing property access" « std::endl;
00057     // ----- All Devices Vector to Array ------
00058     Device *deviceArray = new Device[allDevices.size()];
00059     for (int i = 0; i < allDevices.size(); i++){
00060         deviceArray[i] = allDevices[i];
00061     }
00062
00063     // ----- All DataDevices Vector to Array ------
00064     DataDevice *dataDeviceArray = new DataDevice[allDataDevices.size()];
00065     for (int i = 0; i < allDataDevices.size(); i++){
00066         dataDeviceArray[i] = allDataDevices[i];
00067     }
00068
00069
00070     // ----- All KernelDevices Vector to Array ------
00071     KernelDevice *kernelDeviceArray = new KernelDevice[allKernelDevices.size()];
00072     for (int i = 0; i < allKernelDevices.size(); i++){
00073         kernelDeviceArray[i] = allKernelDevices[i];
00074     }
00075
00076     // ----- Storing Arrays ------
00077     std::cout « "[BUILDER] Storing General properties to Kernel..." « std::endl;
00078     Device::setAllDevices(deviceArray, deviceNames,numBuilders);
00079     DataDevice::setAllDataDevices(dataDeviceArray);
00080     KernelDevice::setAllKernelDevices(kernelDeviceArray);
00081
00082 }
```

References allBuilders, allDataDevices, allDevices, allKernelDevices, buildDKDevice(), name, DataDevice::setAllDataDevices(), Device::setAllDevices(), and KernelDevice::setAllKernelDevices().

Referenced by AI::buildDevice(), and main().

4.6.3.2 buildDisplay()

```
SimpleDisplay * DeviceBuilder::buildDisplay ( )  [static]
```

Builds and returns a new SimpleDisplay object with properties according to builder devices.

Returns

Display with properties

Note

Must be used after buildAllDevices()

Definition at line 24 of file DeviceBuilder.cpp.

```
00024                                       {
00025
00026     SimpleDisplay* display = new SimpleDisplay();
00027     display->setDataDevices(allDataDevices.size());
00028     display->setKernelDevices(allKernelDevices.size());
00029     display->setSecurityConnected(true);
00030     display->setSecurityStatus(true);
00031     return display;
00032
00033 }
```

References allDataDevices, allKernelDevices, SimpleDisplay::setDataDevices(), SimpleDisplay::setKernelDevices(), SimpleDisplay::setSecurityConnected(), and SimpleDisplay::setSecurityStatus().

Referenced by main().

4.6.3.3 buildDKDevice()

```
void DeviceBuilder::buildDKDevice ( )  [private]
```

Builds selected device as data or kernel device into the kernel.

Warning

This will NOT kill the builder.

All properties must be settled before using.

Definition at line 153 of file DeviceBuilder.cpp.

```
00153                                   {
00154
00155     // ----- Skills to Dynamic Vector (so it doesn't get deleted when builder dies) ------
00156     std::vector<void(*)()>* dynamicSkillVector = new std::vector<void(*)()>;
00157     dynamicSkillVector->assign(skillVector.begin(), skillVector.end());
00158
00159     // ----- SkillNames to Array (Optimizing data) ------
00160     std::string *skillNamesArray = new std::string[skillNamesVector.size()];
00161     for (int i = 0; i < this->skillNamesVector.size(); i++){
00162         skillNamesArray[i] = skillNamesVector[i];
00163     }
00164
00165     // ----- Building DataDevice ------
00166     if (this->isDataDevice) {
00167         DataDevice *dataDev = new DataDevice(this->name, this->id, dynamicSkillVector,
      skillNamesArray);
00168
00169         // Setting DataDevice limit (optional)
00170         if (this->limit != nullptr)
00171             (*dataDev).setLimit(this->limit);
00172
00173         (*dataDev).dataGenerator = this->dataGenerator;
00174
00175         // Saving Device Pointers
00176         allDevices.push_back(*dataDev);
00177         allDataDevices.push_back(*dataDev);
00178
00179     // ----- Building KernelDevice ------
00180     }else {
```

```
00181
00182         KernelDevice* kernDev = new KernelDevice(this->name, this->id);
00183
00184         (*kernDev).securityGenerator = this->securityGenerator;
00185
00186         // Saving Device Pointers
00187         allDevices.push_back(*kernDev);
00188         allKernelDevices.push_back(*kernDev);
00189     }
00190
00191 }
```

References allDataDevices, allDevices, allKernelDevices, dataGenerator, isDataDevice, limit, name, securityGenerator, skillNamesVector, and skillVector.

Referenced by buildAllDevices().

4.6.3.4 clearAll()

```
void DeviceBuilder::clearAll ( )  [static]
```

Kills every builder instances.

Warning

Builder's will not work anymore after cleaning until built again.

Definition at line 85 of file DeviceBuilder.cpp.

```
00085                               {
00086
00087     // ----- Cleaning Vectors ------
00088     std::cout « "[BUILDER] Cleaning Builder..." « std::endl;
00089
00090     allDataDevices.clear();
00091     allDevices.clear();
00092     allKernelDevices.clear();
00093 }
```

References allDataDevices, allDevices, and allKernelDevices.

Referenced by AI::buildDevice().

4.6.3.5 setAsDataDevice()

```
void DeviceBuilder::setAsDataDevice ( )
```

Sets type of Device as DataDevice.

Definition at line 130 of file DeviceBuilder.cpp.

```
00130                                   {
00131     this->isDataDevice = true;
00132 }
```

References isDataDevice.

4.6.3.6 setAsKernelDevice()

```
void DeviceBuilder::setAsKernelDevice ( )
```

Sets type of Device as KernelDevice.

Definition at line 136 of file DeviceBuilder.cpp.

```
00136                                       {
00137     this->isDataDevice = false;
00138 }
```

References isDataDevice.

4.6.3.7 setDataGenerator()

```
void DeviceBuilder::setDataGenerator (
             int(*)() dataGenerator )
```

Sets the main data generator for DataDevices.

Warning

Data Devices could not work properly without generator.

Note

Generator must return an integer.

Parameters

dataGenerator	Pointer to function (to generate data)

Definition at line 140 of file DeviceBuilder.cpp.

```
00140                                                    {
00141     this->dataGenerator = dataGenerator;
00142 }
```

References dataGenerator.

4.6.3.8 setLimit()

```
void DeviceBuilder::setLimit (
             int * newLimit )
```

Builds limits into the device Builder. All limits will be controlled and implemented in execution by the kernel.

Warning

KernelDevices can't use limits.

Parameters

newLimit	Array of ints to be checked

Definition at line 123 of file DeviceBuilder.cpp.

```
00123                                          {
00124
00125     // ----- Settling Device Limit ------
00126     this->limit = newLimit;
00127 }
```

References limit.

4.6.3.9 setSecurityGenerator()

```
void DeviceBuilder::setSecurityGenerator (
            bool(*)() securityGenerator )
```

Sets the security generator for KernelDevices.

Warning

Kernel Devices could not work properly without generator.

Note

Generator must return a boolean

Parameters

securityGenerator	Pointer to function (to generate data)

Definition at line 144 of file DeviceBuilder.cpp.

```
00144                                                                    {
00145     this->securityGenerator = securityGenerator;
00146 }
```

References securityGenerator.

4.6.3.10 setSkill()

```
void DeviceBuilder::setSkill (
            void(*)() skill,
            const std::string & skillName )
```

Build one skill into the device Builder. All names and skills of each device will be ordered and used in kernel.

Parameters

Pointer	to function (definition of skill)
skillName	Name of skill (only for user)

Definition at line 115 of file DeviceBuilder.cpp.

```
00115                                                                    {
00116
00117     // ----- Storing Device Skills ------
00118     this->skillVector.push_back(skill);
00119     this->skillNamesVector.push_back(skillName);
00120 }
```

References skillNamesVector, and skillVector.

4.6.4 Member Data Documentation

4.6.4.1 allBuilders

```
std::vector< DeviceBuilder * > DeviceBuilder::allBuilders  [static], [private]
```

Vector with all builders instantiated.

Definition at line 134 of file DeviceBuilder.h.

Referenced by buildAllDevices(), and DeviceBuilder().

4.6.4.2 allDataDevices

```
std::vector< DataDevice > DeviceBuilder::allDataDevices  [static], [private]
```

Vector with all Data Devices instantiated.

Definition at line 138 of file DeviceBuilder.h.

Referenced by buildAllDevices(), buildDisplay(), buildDKDevice(), and clearAll().

4.6.4.3 allDevices

```
std::vector< Device > DeviceBuilder::allDevices  [static], [private]
```

Vector with all Devices instantiated.

Definition at line 136 of file DeviceBuilder.h.

Referenced by buildAllDevices(), buildDKDevice(), and clearAll().

4.6.4.4 allKernelDevices

```
std::vector< KernelDevice > DeviceBuilder::allKernelDevices  [static], [private]
```

Vector with all Kernel Devices instantiated.

Definition at line 140 of file DeviceBuilder.h.

Referenced by buildAllDevices(), buildDisplay(), buildDKDevice(), and clearAll().

4.6.4.5 dataGenerator

```
int(* DeviceBuilder::dataGenerator) ()  [private]
```

Device's function to generate data.

Definition at line 119 of file DeviceBuilder.h.

Referenced by buildDKDevice(), and setDataGenerator().

4.6.4.6 deviceCounter

```
int DeviceBuilder::deviceCounter  [static], [private]
```

Number of devices detected.

Definition at line 142 of file DeviceBuilder.h.

Referenced by DeviceBuilder().

4.6.4.7 id

```
int DeviceBuilder::id  [private]
```

Identifier of Device.

Definition at line 123 of file DeviceBuilder.h.

4.6.4.8 isDataDevice

```
bool DeviceBuilder::isDataDevice  [private]
```

Type of device. (True) Data Device. (False) Kernel Device.

Definition at line 127 of file DeviceBuilder.h.

Referenced by buildDKDevice(), setAsDataDevice(), and setAsKernelDevice().

4.6.4.9 limit

```
int* DeviceBuilder::limit = nullptr  [private]
```

Limits of the Device.

Definition at line 125 of file DeviceBuilder.h.

Referenced by buildDKDevice(), and setLimit().

4.6.4.10 name

```
std::string DeviceBuilder::name = "STD Device"  [private]
```

Name of Device.

Definition at line 113 of file DeviceBuilder.h.

Referenced by buildAllDevices(), buildDKDevice(), and DeviceBuilder().

4.6.4.11 securityGenerator

```
bool(* DeviceBuilder::securityGenerator) ()  [private]
```

Kernel's Devices function to check security.

Definition at line 121 of file DeviceBuilder.h.

Referenced by buildDKDevice(), and setSecurityGenerator().

4.6.4.12 skillNamesVector

```
std::vector<std::string> DeviceBuilder::skillNamesVector  [private]
```

Skill-names of each skill.

Definition at line 117 of file DeviceBuilder.h.

Referenced by buildDKDevice(), and setSkill().

4.6.4.13 skillVector

```
std::vector<void(*)()> DeviceBuilder::skillVector  [private]
```

Skills of the Device.

Definition at line 115 of file DeviceBuilder.h.

Referenced by buildDKDevice(), and setSkill().

The documentation for this class was generated from the following files:

- DeviceBuilder.h
- DeviceBuilder.cpp

4.7 DeviceComposite Class Reference

This class is the responsible of managing multiple orders to devices simultaneously. It will offer a easy interface to compose devices and send orders to them.

```
#include <DeviceComposite.h>
```

Public Member Functions

- DeviceComposite ()
  Simple Composite construction. Empty as default.
- void add (const std::string &name)
  Adds a new device to composite.
- void pop ()
  Pops last device stacked.
- void requestSkill (const std::string &skill)
  Request a skill to all devices appended to the composite.
- void clear ()
  clears all devices inside the composite
- std::string toString ()
  Search and find all device names inside the composite.

Private Attributes

- std::vector< IODi > allIODis

  Vector with all Device Managers (IODi's)

Static Private Attributes

- static const std::string CALL_KEYWORD

  Keyword for adding more devices.

4.7.1 Detailed Description

This class is the responsible of managing multiple orders to devices simultaneously. It will offer a easy interface to compose devices and send orders to them.

Definition at line 15 of file DeviceComposite.h.

4.7.2 Constructor & Destructor Documentation

4.7.2.1 DeviceComposite()

```
DeviceComposite::DeviceComposite ( )
```

Simple Composite construction. Empty as default.

Definition at line 5 of file DeviceComposite.cpp.

```
00005 {}
```

4.7.3 Member Function Documentation

4.7.3.1 add()

```
void DeviceComposite::add (
            const std::string & name )
```

Adds a new device to composite.

Parameters

name	Name of device to search

Exceptions

InvalidDeviceException	If device not found

Definition at line 8 of file DeviceComposite.cpp.

```
00008                                                   {
00009
00010     // -----  Intantiating new IODi -----
00011     IODi iodi;
00012
00013     // -----  Adding device to IODi -----
00014     try{
00015         iodi.setCurrentDevice(name);
00016     }catch(InvalidDeviceException &e){
00017         throw;
00018         return;
00019     }
00020
00021     // -----  Saving IODi -----
00022     this->allIODis.push_back(iodi);
00023 }
```

References allIODis, and IODi::setCurrentDevice().

Referenced by main(), and requestSkill().

4.7.3.2 clear()

```
void DeviceComposite::clear ( )
```

clears all devices inside the composite

Definition at line 55 of file DeviceComposite.cpp.

```
00055                                {
00056
00057     this->allIODis.clear();
00058
00059 }
```

References allIODis.

Referenced by main().

4.7.3.3 pop()

```
void DeviceComposite::pop ( )
```

Pops last device stacked.

Definition at line 24 of file DeviceComposite.cpp.

```
00024                               {
00025     this->allIODis.pop_back();
00026 }
```

References allIODis.

Referenced by main().

4.7.3.4 requestSkill()

```
void DeviceComposite::requestSkill (
            const std::string & skill )
```

Request a skill to all devices appended to the composite.

Parameters

skill	Name of skill to use;

Definition at line 29 of file DeviceComposite.cpp.

```
00029                                                       {
00030
00031     // -----  Check if skill is a keyword to add to composite -----
00032     if (skill == ::CALL_KEYWORD){
00033
00034         std::string answer;
00035         SimpleDisplay::deviCout("Write a Device name for adding it to the composite.");
00036         SimpleDisplay::deviCout("Ex.  <Thermometer>");
00037         SimpleDisplay::cout("~ Device Name:  ");
00038         std::getline(std::cin,answer);
00039         try{
00040             this->add(answer);
00041             SimpleDisplay::deviCout("Device Succesfully added");
00042             return;
00043         }catch(InvalidDeviceException &e){
00044             SimpleDisplay::deviCout("Invalid Device");
00045             return;
00046         }
00047     }
00048
00049     // -----  Else the keyword is for IODIs -----
00050     for (int i = 0; i < this->allIODis.size(); i++)
00051         this->allIODis[i].requestSkill(skill);
00052 }
```

References add(), allIODis, CALL_KEYWORD, SimpleDisplay::cout(), SimpleDisplay::deviCout(), and requestSkill().

Referenced by main(), and requestSkill().

4.7.3.5 toString()

```
std::string DeviceComposite::toString ( )
```

Search and find all device names inside the composite.

Returns

String with the composite as a string

Definition at line 61 of file DeviceComposite.cpp.

```
00061                                       {
00062
00063     // -----  Declarations -----
00064     std::string stringBuilder = "[ ";
00065
00066     for (int i = 0; i < this->allIODis.size(); i++){
00067
00068         // -----  Replacing name spaces with underscores.  -----
00069         std::string name = allIODis[i].requestCurrentDeviceName();
00070         std::replace(name.begin(), name.end(), ' ', '_');
00071
00072         // -----  Appending name -----
00073         stringBuilder += name + " ";
00074     }
00075
00076     // -----  Finishing string -----
00077     stringBuilder += "]";
00078     return stringBuilder;
00079 }
```

References allIODis.

Referenced by main().

4.7.4 Member Data Documentation

4.7.4.1 allIODis

```
std::vector<IODi> DeviceComposite::allIODis  [private]
```

Vector with all Device Managers (IODi's)

Definition at line 56 of file DeviceComposite.h.

Referenced by add(), clear(), pop(), requestSkill(), and toString().

4.7.4.2 CALL_KEYWORD

```
const std::string DeviceComposite::CALL_KEYWORD  [static], [private]
```

Keyword for adding more devices.

Definition at line 62 of file DeviceComposite.h.

Referenced by requestSkill().

The documentation for this class was generated from the following files:

- DeviceComposite.h
- DeviceComposite.cpp

4.8 deviceDataset Class Reference

Static Public Member Functions

- static void initDataset ()

4.8.1 Detailed Description

Definition at line 7 of file deviceDataset.cpp.

4.8.2 Member Function Documentation

4.8.2.1 initDataset()

```
static void deviceDataset::initDataset ( )  [inline], [static]
```

Definition at line 10 of file deviceDataset.cpp.

```
00010                             {
00011
00012         // ----- Building All Devices ------
00013         std::cout « "[BUILDER] Calling Default builders..." « std::endl;
00014         DeviceBuilder* thermometer = new DeviceBuilder("Thermometer");
00015         DeviceBuilder* humiditySensor = new DeviceBuilder("Humidity Sensor");
00016         DeviceBuilder* airSensor = new DeviceBuilder("Air Sensor");
00017         DeviceBuilder* lightSensor = new DeviceBuilder("Light Sensor");
00018         DeviceBuilder* rgbCamera = new DeviceBuilder("RGB Camera");
00019         DeviceBuilder* thermalCamera = new DeviceBuilder("Thermal Camera");
00020
00021         DeviceBuilder* securityCamera1 = new DeviceBuilder("Security Camera 1");
00022         DeviceBuilder* securityCamera2 = new DeviceBuilder("Security Camera 2");
00023         DeviceBuilder* laserDetector = new DeviceBuilder("Laser Detector");
00024
00025         // ----- Building Device Skills ------
00026         std::cout « "[BUILDER] Building skills" « std::endl;
00027
00028         // ----- Building Main Data Generator ------
00029         (*thermometer).setDataGenerator(generateRandomData);
00030         (*humiditySensor).setDataGenerator(generateRandomData);
00031         (*airSensor).setDataGenerator(generateRandomData);
00032         (*lightSensor).setDataGenerator(generateRandomData);
00033         (*rgbCamera).setDataGenerator(generateRandomData);
00034         (*thermalCamera).setDataGenerator(generateRandomData);
00035
00036         (*securityCamera1).setSecurityGenerator(exampleTest);
00037         (*securityCamera2).setSecurityGenerator(exampleTest);
00038         (*laserDetector).setSecurityGenerator(exampleTest);
00039
00040         // ----- Building Skills (Only for DataDevices) -----
00041         (*thermometer).setSkill(exampleSkill1, "calculate");
00042         (*thermometer).setSkill(exampleSkill2, "set celsius");
00043
00044         (*humiditySensor).setSkill(exampleSkill3,"hum decrease");
00045         (*humiditySensor).setSkill(exampleSkill4,"hum increase");
00046
00047         (*airSensor).setSkill(exampleSkill3,"air type --british");
00048
00049         (*lightSensor).setSkill(exampleSkill4,"setmode blind");
00050         (*lightSensor).setSkill(exampleSkill4,"setmode colorful");
00051         (*lightSensor).setSkill(exampleSkill4,"setmode hacker");
00052
00053         (*rgbCamera).setSkill(exampleSkill3,"print camera");
00054         (*rgbCamera).setSkill(exampleSkill4,"lookfor --myself");
00055
00056         (*thermalCamera).setSkill(exampleSkill3,"tcconfig up");
00057         (*thermalCamera).setSkill(exampleSkill4,"tcconfig down");
00058
00059         // ----- Building Device Limits (optional) ------
00060         std::cout « "[BUILDER] Building limits" « std::endl;
00061
00062     //    int lim[3] = {1, 2, 3};
00063     //    (*test).setLimit(lim);
00064
00065         // ----- Setting Device Types [Data/Kernel]
00066         std::cout « "[BUILDER] Settling Types" « std::endl;
00067         (*thermometer).setAsDataDevice();
00068         (*humiditySensor).setAsDataDevice();
00069         (*airSensor).setAsDataDevice();
00070         (*lightSensor).setAsDataDevice();
00071         (*rgbCamera).setAsDataDevice();
00072         (*thermalCamera).setAsDataDevice();
00073
00074         (*securityCamera1).setAsKernelDevice();
00075         (*securityCamera2).setAsKernelDevice();
00076         (*laserDetector).setAsKernelDevice();
00077     }
```

References exampleSkill1(), exampleSkill2(), exampleSkill3(), exampleSkill4(), exampleTest(), and generateRandomData().

Referenced by main().

The documentation for this class was generated from the following file:

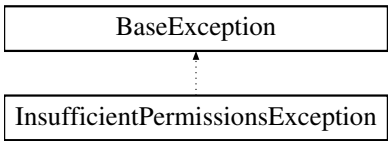
- deviceDataset.cpp

4.9 InsufficientPermissionsException Class Reference

This class lets the program throw an exception when the permissions of the user are not valid.

`#include <InsufficientPermissionsException.h>`

Inheritance diagram for InsufficientPermissionsException:



Public Member Functions

- InsufficientPermissionsException (const std::string &tag)

  Throws main exception.

Static Private Attributes

- static std::string ERROR_MESSAGE = "Invalid User"

  Message to display in what()

Additional Inherited Members

4.9.1 Detailed Description

This class lets the program throw an exception when the permissions of the user are not valid.

Definition at line 16 of file InsufficientPermissionsException.h.

4.9.2 Constructor & Destructor Documentation

4.9.2.1 InsufficientPermissionsException()

```
InsufficientPermissionsException::InsufficientPermissionsException (
            const std::string & tag )  [explicit]
```

Throws main exception.

Parameters

tag	Founder of the exception

Definition at line 4 of file InsufficientPermissionsException.cpp.

```
00004                                                                                {
00005
00006     this->exception = ERROR_MESSAGE;
00007     this->tag = tag;
00008
00009
00010 }
```

References ERROR_MESSAGE, BaseException::exception, and BaseException::tag.

4.9.3 Member Data Documentation

4.9.3.1 ERROR_MESSAGE

```
std::string InsufficientPermissionsException::ERROR_MESSAGE = "Invalid User"  [static], [private]
```

Message to display in what()

Definition at line 33 of file InsufficientPermissionsException.h.

Referenced by InsufficientPermissionsException().

The documentation for this class was generated from the following files:

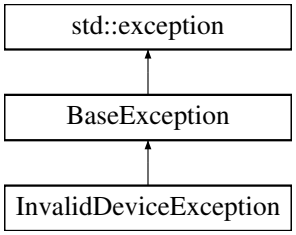
- InsufficientPermissionsException.h
- InsufficientPermissionsException.cpp

4.10 InvalidDeviceException Class Reference

This class lets the program throw an exception when the selected device is not valid.

```
#include <InvalidDeviceException.h>
```

Inheritance diagram for InvalidDeviceException:



Public Member Functions

- InvalidDeviceException (const std::string &tag)

  Throws main exception.

Static Private Attributes

- static std::string ERROR_MESSAGE = "Invalid User"

  Message to display in what()

Additional Inherited Members

4.10.1 Detailed Description

This class lets the program throw an exception when the selected device is not valid.

Definition at line 16 of file InvalidDeviceException.h.

4.10.2 Constructor & Destructor Documentation

4.10.2.1 InvalidDeviceException()

```
InvalidDeviceException::InvalidDeviceException (
            const std::string & tag )  [explicit]
```

Throws main exception.

Parameters

tag	Founder of the exception

Definition at line 4 of file InvalidDeviceException.cpp.

```
00004                                                                   {
00005
00006     this->exception = ERROR_MESSAGE;
00007     this->tag = tag;
00008
00009 }
```

References ERROR_MESSAGE, BaseException::exception, and BaseException::tag.

4.10.3 Member Data Documentation

4.10.3.1 ERROR_MESSAGE

```
std::string InvalidDeviceException::ERROR_MESSAGE = "Invalid User"  [static], [private]
```

Message to display in what()

Definition at line 33 of file InvalidDeviceException.h.

Referenced by InvalidDeviceException().

The documentation for this class was generated from the following files:

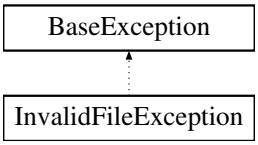
- InvalidDeviceException.h
- InvalidDeviceException.cpp

4.11 InvalidFileException Class Reference

This class lets the program throw an exception when the File to be read is not valid.

`#include <InvalidFileException.h>`

Inheritance diagram for InvalidFileException:



Public Member Functions

- InvalidFileException (const std::string &tag)
  Throws main exception.

Static Private Attributes

- static std::string ERROR_MESSAGE = "Invalid File"
  Message to display in what()

Additional Inherited Members

4.11.1 Detailed Description

This class lets the program throw an exception when the File to be read is not valid.

Definition at line 16 of file InvalidFileException.h.

4.11.2 Constructor & Destructor Documentation

4.11.2.1 InvalidFileException()

```
InvalidFileException::InvalidFileException (
            const std::string & tag )  [explicit]
```

Throws main exception.

Parameters

tag	Founder of the exception

Definition at line 4 of file InvalidFileException.cpp.

```
00004                                                                    {
00005
00006     this->exception = ERROR_MESSAGE;
00007     this->tag = tag;
00008 }
```

References ERROR_MESSAGE, BaseException::exception, and BaseException::tag.

4.11.3 Member Data Documentation

4.11.3.1 ERROR_MESSAGE

`std::string InvalidFileException::ERROR_MESSAGE = "Invalid File"  [static], [private]`

Message to display in what()

Definition at line 33 of file InvalidFileException.h.

Referenced by InvalidFileException().

The documentation for this class was generated from the following files:

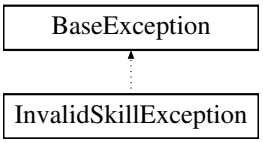
- InvalidFileException.h
- InvalidFileException.cpp

4.12 InvalidSkillException Class Reference

This class lets the program throw an exception when the selected skill is not valid.

`#include <InvalidSkillException.h>`

Inheritance diagram for InvalidSkillException:



Public Member Functions

- InvalidSkillException (const std::string &tag)

  Throws main exception.

Static Private Attributes

- static std::string ERROR_MESSAGE = "Invalid File"

  Message to display in what()

Additional Inherited Members

4.12.1 Detailed Description

This class lets the program throw an exception when the selected skill is not valid.

Definition at line 16 of file InvalidSkillException.h.

4.12.2 Constructor & Destructor Documentation

4.12.2.1 InvalidSkillException()

```
InvalidSkillException::InvalidSkillException (
            const std::string & tag )  [explicit]
```

Throws main exception.

Parameters

tag	Founder of the exception

Definition at line 4 of file InvalidSkillException.cpp.

```
00004                                                                   {
00005
00006     this->exception = ERROR_MESSAGE;
00007     this->tag = tag;
00008
00009 }
```

References ERROR_MESSAGE, BaseException::exception, and BaseException::tag.

4.12.3 Member Data Documentation

4.12.3.1 ERROR_MESSAGE

```
std::string InvalidSkillException::ERROR_MESSAGE = "Invalid File"  [static], [private]
```

Message to display in what()

Definition at line 33 of file InvalidSkillException.h.

Referenced by InvalidSkillException().

The documentation for this class was generated from the following files:

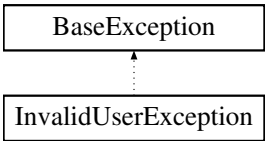
- InvalidSkillException.h
- InvalidSkillException.cpp

4.13 InvalidUserException Class Reference

This class lets the program throw an exception when the selected user is not valid.

`#include <InvalidUserException.h>`

Inheritance diagram for InvalidUserException:



Public Member Functions

- InvalidUserException (const std::string &tag)
  Throws main exception.

Static Private Attributes

- static std::string ERROR_MESSAGE = "Invalid User"
  Message to display in what()

Additional Inherited Members

4.13.1 Detailed Description

This class lets the program throw an exception when the selected user is not valid.

Definition at line 16 of file InvalidUserException.h.

4.13.2 Constructor & Destructor Documentation

4.13.2.1 InvalidUserException()

```
InvalidUserException::InvalidUserException (
            const std::string & tag )  [explicit]
```

Throws main exception.

Parameters

tag	Founder of the exception

Definition at line 5 of file InvalidUserException.cpp.

```
00005                                                                    {
00006
00007     this->exception = ERROR_MESSAGE;
00008     this->tag = tag;
00009
00010 }
```

References ERROR_MESSAGE, BaseException::exception, and BaseException::tag.

4.13.3 Member Data Documentation

4.13.3.1 ERROR_MESSAGE

```
std::string InvalidUserException::ERROR_MESSAGE = "Invalid User"  [static], [private]
```

Message to display in what()

Definition at line 33 of file InvalidUserException.h.

Referenced by InvalidUserException().

The documentation for this class was generated from the following files:

- InvalidUserException.h
- InvalidUserException.cpp

4.14 IODi Class Reference

This class will be the responsible for interpreting the status of the devices and sending proper information, and executing the requested skills for the main class when it needs to interact with any device. Thus, it will be implemented by the main class and can't exist without Device class.

```
#include <IODi.h>
```

Public Types

- enum { DEFAULT_ID = 33 }

Public Member Functions

- IODi ()

  Creates a new IOD interface ready for accessing to Device properties.
- std::string * requestDeviceNames ()

  Requests all names of detected devices.
- std::string * requestDeviceSkills ()

  Requests all skill-names of current device.
- std::string requestCurrentDeviceName ()

  Requests name of current device.
- int requestNumberOfDataDevices ()

  Requests number of Data Devices Detected.
- int requestNumberOfSkills ()

  Requests number of skill-names in current Device.
- bool * requestAllStatus ()

  Request status of all devices.
- void requestSkill (const std::string &skill)

  Uses a skill from selected device.
- void requestSkill (const int skill)

  Uses a skill from selected device.
- void setCurrentDevice (const std::string &name)

  Sets the id and position of current device inside IODi.
- void setCurrentDevice (const int pos)

  Sets the id and position of current device inside IODi.

Private Attributes

- int id

  id of current IODi
- std::string currentDeviceName

  Name of current device.
- int currentDevicePos

  Position of current device in allDevices.

Static Private Attributes

- static std::string TAG = "IODi"

  Name of class (for exception throwing)
- static int maxID = 0

  maximum IODi ID detected

4.14.1 Detailed Description

This class will be the responsible for interpreting the status of the devices and sending proper information, and executing the requested skills for the main class when it needs to interact with any device. Thus, it will be implemented by the main class and can't exist without Device class.

Definition at line 19 of file IODi.h.

4.14.2 Member Enumeration Documentation

4.14.2.1 anonymous enum

```
anonymous enum
```

Enumerator

DEFAULT_ID	

Definition at line 26 of file IODi.h.

```
00026         {
00027         DEFAULT_ID = 33
00028     };
```

4.14.3 Constructor & Destructor Documentation

4.14.3.1 IODi()

```
IODi::IODi ( )
```

Creates a new IOD interface ready for accessing to Device properties.

Definition at line 22 of file IODi.cpp.

```
00022           {
00023     this->id = DEFAULT_ID;
00024     maxID++;
00025 };
```

References DEFAULT_ID, and maxID.

4.14.4 Member Function Documentation

4.14.4.1 requestAllStatus()

```
bool * IODi::requestAllStatus ( )
```

Request status of all devices.

Returns

Pointer to boolean with all status. True if it is turned on. Else False.

Definition at line 106 of file IODi.cpp.

```
00106                                {
00107
00108     bool* status = new bool[DataDevice::numSensors];
00109
00110     for (int i = 0; i < DataDevice::numSensors; i++)
00111         status[i] = ~DataDevice::allDataDevices[i];
00112
00113     return status;
00114 };
```

References DataDevice::allDataDevices, and DataDevice::numSensors.

Referenced by main().

4.14.4.2 requestCurrentDeviceName()

```
std::string IODi::requestCurrentDeviceName ( )
```

Requests name of current device.

Returns

Name of current device

Definition at line 124 of file IODi.cpp.

```
00124                                          {
00125     return this->currentDeviceName;
00126 }
```

References currentDeviceName.

Referenced by main().

4.14.4.3 requestDeviceNames()

```
std::string * IODi::requestDeviceNames ( )
```

Requests all names of detected devices.

Returns

Pointer to array with names of Devices

Definition at line 79 of file IODi.cpp.

```
00079                                    {
00080     return DataDevice::allDataDeviceNames;
00081 };
```

References DataDevice::allDataDeviceNames.

Referenced by main().

4.14.4.4 requestDeviceSkills()

```
std::string * IODi::requestDeviceSkills ( )
```

Requests all skill-names of current device.

Returns

Pointer to array with name of every skill of current device

Definition at line 94 of file IODi.cpp.

```
00094                                       {
00095     int numSkills = DataDevice::allDataDevices[this->currentDevicePos].getNumSkills();
00096
00097     std::string* deviceSkills = new std::string[numSkills];
00098
00099     for (int i = 0; i < numSkills; i++){
00100         deviceSkills[i] = DataDevice::allDataDevices[this->currentDevicePos].skillNames[i];
00101     }
00102
00103     return deviceSkills;
00104 };
```

References DataDevice::allDataDevices, currentDevicePos, Device::getNumSkills(), and Device::skillNames.

4.14.4.5 requestNumberOfDataDevices()

```
int IODi::requestNumberOfDataDevices ( )
```

Requests number of Data Devices Detected.

Returns

Number of DataDevices built

Definition at line 84 of file IODi.cpp.

```
00084                                       {
00085     return DataDevice::getNumSensors();
00086 };
```

References DataDevice::getNumSensors().

Referenced by setCurrentDevice().

4.14.4.6 requestNumberOfSkills()

```
int IODi::requestNumberOfSkills ( )
```

Requests number of skill-names in current Device.

Returns

Number of skills of current device

Definition at line 89 of file IODi.cpp.

```
00089                                   {
00090     return DataDevice::allDataDevices[this->currentDevicePos].getNumSkills();
00091 }
```

References DataDevice::allDataDevices, currentDevicePos, and Device::getNumSkills().

4.14.4.7 requestSkill() [1/2]

```
void IODi::requestSkill (
            const int skill )
```

Uses a skill from selected device.

Parameters

skill	Position in SkillList of skill

Definition at line 117 of file IODi.cpp.

```
00117                                       {
00118     try {
00119         DataDevice::allDataDevices[this->currentDevicePos][skill];
00120     }catch(InvalidSkillException &e){};
00121 };
```

References DataDevice::allDataDevices, and currentDevicePos.

4.14.4.8 requestSkill() [2/2]

```
void IODi::requestSkill (
            const std::string & skill )
```

Uses a skill from selected device.

Parameters

skill	Name of skill

Definition at line 129 of file IODi.cpp.

```
00129                                           {
00130
00131     // ----- Obtaining current device -----
00132     DataDevice* target = &DataDevice::allDataDevices[this->currentDevicePos];
00133
00134     // ----- Switch between possible arguments ------
00135     if (skill == ":help")
00136         SimpleDisplay::printDeviHelp();
00137
00138     else if (skill == ":dev")
00139         SimpleDisplay::printSkillManual(target->skillNames,\
00140                                         this->requestNumberOfSkills());
00141                                         // -> Vector with the name of skills
00142                                         // -> Int with the number of skills
00143
00144     else if (skill == ":clean")
00145         SimpleDisplay::cleanDevi(this->currentDeviceName);
00146
00147     else if (skill == "data")
00148         SimpleDisplay::deviPrintData(target->collectedData,\
00149                                      target->collectedTimes);
00150                                      // -> Vector with measurements
00151                                      // -> Vector with times of measurements
00152
00153     else if(skill == "turnoff"){
00154         target->isActive = false;
00155         target->collectedData.resize(1);
00156         target->collectedData[0] = 0;
00157         target->collectedTimes.resize(1);
00158         target->collectedTimes[0] = "REBOOT";
00159         SimpleDisplay::deviCout("[IODi] " + target->getName() + " turned OFF");
00160     }
00161
00162     else if(skill == "turnon"){
00163         target->isActive = true;
00164         SimpleDisplay::deviCout("[IODi] " + target->getName() + " turned ON");
00165     }
00166
00167     else if (skill == "reboot"){
00168         this->requestSkill("turnoff");
00169         this->requestSkill("turnon");
00170     }
00171
```

```
00172     else if (skill == "pwd")
00173         SimpleDisplay::deviCout("/home/devices/" + std::to_string(this->currentDevicePos));
00174
00175     // ----- Frequent Invalid Commands ------
00176
00177     else if (skill == "help")
00178         SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :help");
00179
00180     else if (skill == "exit")
00181         SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :wq");
00182
00183     else if (skill == "clear")
00184         SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :clean");
00185
00186     else if (skill == "clean")
00187         SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :clean");
00188
00189     else if (skill == "dev")
00190         SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :dev");
00191
00192     else if (skill[0] == '\"' || skill[0] == '\") {
00193         SimpleDisplay::deviCout("Are you trying to Inject SQL? Lol ok");
00194         SimpleDisplay::deviCout("[YOU HAVE BEEN BANNED]");
00195         exit(1);
00196     }
00197
00198     // ----- Not Global command, request to current device ------
00199     else {
00200         try{
00201             (*target)[skill];
00202         }catch(InvalidSkillException &e){};
00203     }
00204 }
```

References DataDevice::allDataDevices, SimpleDisplay::cleanDevi(), DataDevice::collectedData, DataDevice::collectedTimes, currentDeviceName, currentDevicePos, SimpleDisplay::deviCout(), SimpleDisplay::deviPrintData(), Device::getName(), Device::isActive, SimpleDisplay::printDeviHelp(), SimpleDisplay::printSkillManual(), requestSkill(), and Device::skillNames.

Referenced by requestSkill(), and AI::turnDevices().

4.14.4.9 setCurrentDevice() [1/2]

```
void IODi::setCurrentDevice (
            const int pos )
```

Sets the id and position of current device inside IODi.

Parameters

Position	of device in allDevices

Exceptions

InvalidDeviceException	Device settled not valid/built

Definition at line 28 of file IODi.cpp.

```
00028                                       {
00029
00030     // ----- If position is valid ------
00031     if (pos < IODi::requestNumberOfDataDevices()) {
00032
00033         // Set position in current device
00034         this->currentDevicePos = pos;
00035
00036         // Search and set name in current device
00037         this->currentDeviceName = DataDevice::allDataDeviceNames[pos];
```

```
00038
00039     }
00040     else
00041         // ----- Argument not valid ------
00042         throw InvalidDeviceException(TAG);
00043
00044 }
```

References DataDevice::allDataDeviceNames, currentDeviceName, currentDevicePos, requestNumberOfDataDevices(), and TAG.

4.14.4.10 setCurrentDevice() [2/2]

```
void IODi::setCurrentDevice (
             const std::string & name )
```

Sets the id and position of current device inside IODi.

Parameters

Name	of device

Exceptions

InvalidDeviceException	Device settled not valid/built

Definition at line 47 of file IODi.cpp.

```
00047                                                 {
00048
00049     // ----- Check type argument is not a number ------
00050     if (std::isdigit(name[0])){
00051
00052         this->setCurrentDevice(std::stoi(name));
00053         return;
00054     }
00055
00056     // ----- Declarations ------
00057     int maxDevices = DataDevice::getNumSensors();
00058
00059     // ----- Search Device ------
00060     for (int i = 0; i < maxDevices; i++){
00061
00062         if (name == DataDevice::allDataDeviceNames[i]){
00063
00064             // Save position and name in current device
00065             this->currentDevicePos = i;
00066             this->currentDeviceName = name;
00067             return;
00068
00069         }
00070
00071     }
00072     // ----- Device not valid ------
00073     throw InvalidDeviceException(TAG);
00074
00075 }
```

References DataDevice::allDataDeviceNames, currentDeviceName, currentDevicePos, DataDevice::getNumSensors(), setCurrentDevice(), and TAG.

Referenced by DeviceComposite::add(), main(), setCurrentDevice(), and AI::turnDevices().

4.14.5 Member Data Documentation

4.14.5.1 currentDeviceName

```
std::string IODi::currentDeviceName  [private]
```

Name of current device.

Definition at line 91 of file IODi.h.

Referenced by requestCurrentDeviceName(), requestSkill(), and setCurrentDevice().

4.14.5.2 currentDevicePos

```
int IODi::currentDevicePos  [private]
```

Position of current device in allDevices.

Definition at line 93 of file IODi.h.

Referenced by requestDeviceSkills(), requestNumberOfSkills(), requestSkill(), and setCurrentDevice().

4.14.5.3 id

```
int IODi::id  [private]
```

id of current IODi

Definition at line 89 of file IODi.h.

4.14.5.4 maxID

```
int IODi::maxID = 0  [static], [private]
```

maximum IODi ID detected

Definition at line 102 of file IODi.h.

Referenced by IODi().

4.14.5.5 TAG

```
std::string IODi::TAG = "IODi"  [static], [private]
```

Name of class (for exception throwing)

Definition at line 100 of file IODi.h.

Referenced by setCurrentDevice().

The documentation for this class was generated from the following files:

- IODi.h
- IODi.cpp

4.15 kb Class Reference

```
#include <kb.h>
```

Static Public Member Functions

- static int kbhit ()

4.15.1 Detailed Description

Definition at line 10 of file kb.h.

4.15.2 Member Function Documentation

4.15.2.1 kbhit()

```
int kb::kbhit ( )  [static]
```

Definition at line 3 of file kb.cpp.

```
00003             {
00004   struct termios oldt, newt;
00005   int ch;
00006   int oldf;
00007
00008   tcgetattr(STDIN_FILENO, &oldt);
00009   newt = oldt;
00010   newt.c_lflag &= ~(ICANON | ECHO);
00011   tcsetattr(STDIN_FILENO, TCSANOW, &newt);
00012   oldf = fcntl(STDIN_FILENO, F_GETFL, 0);
00013   fcntl(STDIN_FILENO, F_SETFL, oldf | O_NONBLOCK);
00014
00015   ch = getchar();
00016
00017   tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
00018   fcntl(STDIN_FILENO, F_SETFL, oldf);
00019
00020   if(ch != EOF){
00021     ungetc(ch, stdin);
00022     return 1;
00023   }
00024
00025   return 0;
00026
00027 }
```

Referenced by AI::throwDemon().

The documentation for this class was generated from the following files:

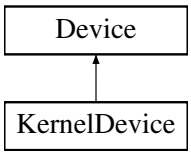
- kb.h
- kb.cpp

4.16 KernelDevice Class Reference

This class will contain specific properties of only kernel interacting devices and a set of functionalities for managing and checking their data. It also inherits properties from Device class.

`#include <KernelDevice.h>`

Inheritance diagram for KernelDevice:



Public Member Functions

- KernelDevice ()

  Creates a New KernelDevice. It uses invalid properties by default. Not recommended.
- KernelDevice (const std::string &name, const int id)

  Builds a new KernelDevice and assign its basic properties.

Static Private Member Functions

- static void setAllKernelDevices (KernelDevice *allDevices)

  Sets all pointers to KernelDevices. Used for general Kernel Device control, It can cause conflict with other processes, recommended only with builder implementation.

Private Attributes

- bool(* securityGenerator)()

  Pointer to function to check security status.
- bool securityHasBeenBroken

  Pointer to function to check security status.

Static Private Attributes

- static KernelDevice * allKernelDevices

  Pointer to array with all Kernel Devices.
- static int numKerns = 0

  number of detected Kernel Devices

Friends

- class DeviceBuilder
- class AI

Additional Inherited Members

4.16.1 Detailed Description

This class will contain specific properties of only kernel interacting devices and a set of functionalities for managing and checking their data. It also inherits properties from Device class.

Definition at line 17 of file KernelDevice.h.

4.16.2 Constructor & Destructor Documentation

4.16.2.1 KernelDevice() [1/2]

```
KernelDevice::KernelDevice ( )
```

Creates a New KernelDevice. It uses invalid properties by default. Not recommended.

Definition at line 21 of file KernelDevice.cpp.

```
00021                                {
00022     this->name = "";
00023     this->isActive = false;
00024     this->id = INVALID_DEVICE;
00025     this->isDataDevice = false;
00026     this->securityHasBeenBroken = false;
00027 };
```

References Device::INVALID_DEVICE, Device::isActive, Device::isDataDevice, Device::name, and securityHasBeenBroken.

4.16.2.2 KernelDevice() [2/2]

```
KernelDevice::KernelDevice (
            const std::string & name,
            const int id )
```

Builds a new KernelDevice and assign its basic properties.

Parameters

name	public name for user
id	private identifier for AI

Definition at line 30 of file KernelDevice.cpp.

```
00030                                                                {
00031
00032     std::cout « "[KernelDevice] New Device Detected:  0x" « id « std::endl;
00033
00034     numKerns++;
00035
00036     this->name = name;
00037     this->id = id;
```

```
00038     this->isDataDevice = false;
00039     this->isActive = true;
00040     this->securityHasBeenBroken = false;
00041 }
```

References Device::id, Device::isActive, Device::isDataDevice, Device::name, numKerns, and securityHasBeenBroken.

4.16.3 Member Function Documentation

4.16.3.1 setAllKernelDevices()

```
void KernelDevice::setAllKernelDevices (
            KernelDevice * allDevices )  [static], [private]
```

Sets all pointers to KernelDevices. Used for general Kernel Device control, It can cause conflict with other processes, recommended only with builder implementation.

Definition at line 48 of file KernelDevice.cpp.

```
00048                                                    {
00049     allKernelDevices = allDevices;
00050 }
```

References Device::allDevices, and allKernelDevices.

Referenced by DeviceBuilder::buildAllDevices().

4.16.4 Friends And Related Function Documentation

4.16.4.1 AI

```
friend class AI  [friend]
```

Definition at line 21 of file KernelDevice.h.

4.16.4.2 DeviceBuilder

```
friend class DeviceBuilder  [friend]
```

Definition at line 20 of file KernelDevice.h.

4.16.5 Member Data Documentation

4.16.5.1 allKernelDevices

```
KernelDevice * KernelDevice::allKernelDevices  [static], [private]
```

Pointer to array with all Kernel Devices.

Definition at line 58 of file KernelDevice.h.

Referenced by AI::checkSecurity(), AI::forceError(), AI::recoverSecurity(), AI::refreshData(), setAllKernelDevices(), and AI::turnSecurity().

4.16.5.2 numKerns

```
int KernelDevice::numKerns = 0  [static], [private]
```

number of detected Kernel Devices

Definition at line 60 of file KernelDevice.h.

Referenced by AI::buildDevice(), AI::checkSecurity(), AI::forceError(), KernelDevice(), AI::recoverSecurity(), AI::refreshData(), and AI::turnSecurity().

4.16.5.3 securityGenerator

```
bool(* KernelDevice::securityGenerator) ()  [private]
```

Pointer to function to check security status.

Definition at line 49 of file KernelDevice.h.

4.16.5.4 securityHasBeenBroken

```
bool KernelDevice::securityHasBeenBroken  [private]
```

Pointer to function to check security status.

Definition at line 51 of file KernelDevice.h.

Referenced by AI::checkSecurity(), AI::forceError(), KernelDevice(), AI::recoverSecurity(), and AI::refreshData().

The documentation for this class was generated from the following files:

- KernelDevice.h
- KernelDevice.cpp

4.17 Login Class Reference

This class is the responsible for superimposing a login interface to make the user log in as user. This will be a graphical interface belonging to the User class.

`#include <Login.h>`

Public Member Functions

- Login ()

  Builds a logger.
- User startLogin ()

  Creates a loop where the user must log in. It will never exit until the user input a valid password and user (data obtained from User.h class)

Private Attributes

- int user

  Current user entry.
- int pass

  Current password entry.

Static Private Attributes

- static int SLEEP_TIME = 1

  Time to sleep between bad inputs.
- static int EXIT_REQUEST = -1

  Value of exit request. -1 by default.

4.17.1 Detailed Description

This class is the responsible for superimposing a login interface to make the user log in as user. This will be a graphical interface belonging to the User class.

Programmer Info: Designed as a logger obj. for multiple and simultaneous logger interfaces. Isn't any problem with changing to static functions.

Definition at line 20 of file Login.h.

4.17.2 Constructor & Destructor Documentation

4.17.2.1 Login()

```
Login::Login ( )
```

Builds a logger.

Note

needed for use startLogin()

Definition at line 19 of file Login.cpp.

```
00019 {};
```

4.17.3 Member Function Documentation

4.17.3.1 startLogin()

```
User Login::startLogin ( )
```

Creates a loop where the user must log in. It will never exit until the user input a valid password and user (data obtained from User.h class)

Returns

Valid User

Definition at line 24 of file Login.cpp.

```
00024                       {
00025
00026
00027     // ----- Declarations ------
00028     bool badAccess = false;          // Bool to detect if input is not correct.
00029
00030     // ----- Logger Bucle ------
00031     do {
00032
00033         //  Checking Bad Inputs
00034         if (badAccess) {
00035             SimpleDisplay::cout("Bad Access");
00036             sleep(SLEEP_TIME);
00037         }
00038
00039         //  Reset Variables
00040         user = User::INVALID_USER;
00041         pass = User::INVALID_PASS;
00042
00043         //  Prompt user
00044         SimpleDisplay::printLoginInterface(user, pass, User::INVALID_USER, User::INVALID_PASS);
00045         SimpleDisplay::cout("\nKeyboard:  ");
00046         std::cin » user;
00047
00048         if (user == EXIT_REQUEST) raise(SIGINT);
00049
00050         //  Prompt password
00051         SimpleDisplay::printLoginInterface(user, pass, User::INVALID_USER, User::INVALID_PASS);
00052         SimpleDisplay::cout("\nKeyboard:  ");
00053         std::cin » pass;
00054
00055         if (pass == EXIT_REQUEST) raise(SIGINT);
00056
00057         //  Print checking interface
00058         SimpleDisplay::printLoginInterface(user, pass, User::INVALID_USER, User::INVALID_PASS);
00059         SimpleDisplay::printChecking();
```

```
00060
00061         //  Prepare next logger (if needed)
00062         badAccess = true;
00063         sleep(SLEEP_TIME);
00064
00065         //  Clear bad inputs
00066         std::cin.clear();
00067         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00068
00069         //  Loop until the user and password are valid
00070     } while (!User::check(user, pass));
00071
00072     // Return Admin (If needed)
00073     if (Admin::adminStatus(user))
00074         return Admin(user);
00075
00076     // ----- Returning User ------
00077     return User(user);
00078 };
```

References Admin::adminStatus(), User::check(), SimpleDisplay::cout(), EXIT_REQUEST, User::INVALID_PASS, User::INVALID_USER, pass, SimpleDisplay::printChecking(), SimpleDisplay::printLoginInterface(), SLEEP_TIME, and user.

Referenced by main().

4.17.4 Member Data Documentation

4.17.4.1 EXIT_REQUEST

```
int Login::EXIT_REQUEST = -1  [static], [private]
```

Value of exit request. -1 by default.

Definition at line 48 of file Login.h.

Referenced by startLogin().

4.17.4.2 pass

```
int Login::pass  [private]
```

Current password entry.

Definition at line 55 of file Login.h.

Referenced by startLogin().

4.17.4.3 SLEEP_TIME

```
int Login::SLEEP_TIME = 1  [static], [private]
```

Time to sleep between bad inputs.

Definition at line 46 of file Login.h.

Referenced by startLogin().

4.17.4.4 user

```
int Login::user  [private]
```

Current user entry.

Definition at line 53 of file Login.h.

Referenced by startLogin().

The documentation for this class was generated from the following files:

- Login.h
- Login.cpp

4.18 SimpleDisplay Class Reference

This library will contain a set of methods implemented by the main for graphicating in screen the main interface. This class can be changed by any other library whose functionalities are named in the same way.

```
#include <SimpleDisplay.h>
```

Public Member Functions

- SimpleDisplay ()
  Builds a SimpleDisplay object. Used for storing display properties and fastering accessing.
- void setSecurityConnected (const bool securityConnected)
  Sets the connection status of the alarms.
- void setSecurityStatus (const bool securityStatus)
  Sets the security status of the alarms.
- void setKernelDevices (const int kernelDevices)
  Sets number of Kernel Devices.
- void setDataDevices (const int dataDevices)
  Sets number of Data Devices.
- void setUser (const int user)
  Sets current user identificator.
- void setGroup (const std::string &group)
  Sets current user group.
- void operator<< (const SimpleDisplay &display)
  Inherit configuration from other display.
- void printMainInterface (std::string *deviceNames, bool *deviceStatus)
  Prints main interface to screen using SimpleDisplay properties.

Static Public Member Functions

- static void printLoginInterface (const int user, const int pass, const int INVALID_USER=-1, const int INVALID_PASS=-1)

  Prints the login interface (encrypted)
- static void printChecking ()

  Prints text "Checking...".
- static void printPresentation ()

  Prints presentation text before main program.
- static void deviCout (const std::string &text)

  Prints text to Screen using DEVi++ prefix.
- static void printDeviHeader (const std::string &deviceName)

  Prints Devi++ initial interface.
- static void cleanDevi (const std::string &deviceName)

  Prints Devi++ header without help information.
- static void printAboutUs ()

  Prints information about the version and original designer.
- static void printDeviHelp ()

  Prints Devi++ help manual.
- static void deviPrintData (const std::vector< int > &data, const std::vector< std::string > &time)

  Prints a dataframe in screen using DEVi++ model.
- static void printSkillManual (const std::string *skillnames, const int count)

  Prints a self-generated manual based in current SimpleDisplay.
- static void jump ()

  Cleans the display using '\n'.
- static void cout (const std::string &text)

  Prints text to Screen.
- static void printAIInterface ()

  Prints AI interface.
- static void printRequestPassword ()

  Prints Authentication Screen from AI.
- static void execBuildName ()

  Prints first section of ExecBuilding.
- static void execBuildType ()

  Prints second section of ExecBuilding.
- static void execBuildGenerator (const bool isDataDevice)

  Prints third section of ExecBuilding.
- static void execBuildFinish ()

  Prints final section of ExecBuilding.
- static void checkAbortStatus ()

  Checks if abort status has jumped out. If so, calls a Rescue Thread.

Static Public Attributes

- static std::string COLOR = "\033[36m"

  String to call bold font.
- static std::string GREY_COLOR = "\033[32m"

  String to call secondary font.
- static std::string RESET_COLOR = "\033[0m"

  String to call default font.

- static std::string WARNING_COLOR = "\033[31m"

  String to call warning color.
- static SimpleDisplay * currentDisplay

  Pointer to current Display.
- static bool security_abort = false

  Variable to jump if program fail. Default in False.

Private Attributes

- bool securityConnected

  Conection status of alarms.
- bool securityStatus

  Security status of alarms.
- int kernelDevices

  Number of Kernel Devices detected.
- int dataDevices

  Number of Data Devices detected.
- int user

  Identifier of current user.
- std::string group

  Group of current user.

Static Private Attributes

- static const int MAXIMUM_OPTIONS = 12

  Maximum options to display in main interface.

4.18.1 Detailed Description

This library will contain a set of methods implemented by the main for graphicating in screen the main interface. This class can be changed by any other library whose functionalities are named in the same way.

Definition at line 21 of file SimpleDisplay.h.

4.18.2 Constructor & Destructor Documentation

4.18.2.1 SimpleDisplay()

```
SimpleDisplay::SimpleDisplay ( )
```

Builds a SimpleDisplay object. Used for storing display properties and fastering accessing.

Definition at line 19 of file SimpleDisplay.cpp.

```
00019                                {
00020
00021     SimpleDisplay::currentDisplay = this;
00022
00023 };
```

References currentDisplay.

4.18.3 Member Function Documentation

4.18.3.1 checkAbortStatus()

```
void SimpleDisplay::checkAbortStatus ( )  [static]
```

Checks if abort status has jumped out. If so, calls a Rescue Thread.

Definition at line 530 of file SimpleDisplay.cpp.

```
00530                                       {
00531
00532     if (security_abort) {
00533         std::cout « "\n----------------- [RESCUE PROTOCOL THREAD] -------------------------";
00534         std::cout « "\n[WARNING] Abort signal called from insecure process, if you are trying to\n";
00535         std::cout « "exit the program, please use -1 to return to login interface.\n";
00536         std::cout « "[AI] Executing Secure Exit...";
00537         exit(1);
00538     }
00539 }
```

References security_abort.

Referenced by cout(), deviCout(), execBuildFinish(), execBuildGenerator(), execBuildName(), execBuildType(), and printAIInterface().

4.18.3.2 cleanDevi()

```
void SimpleDisplay::cleanDevi (
             const std::string & deviceName )  [static]
```

Prints Devi++ header without help information.

Definition at line 453 of file SimpleDisplay.cpp.

```
00453                                                   {
00454
00455     jump();
00456
      printf("#=============================================================================================================#\n");
00457     printf("|                                                 %-20s
      |\n",deviceName.c_str());
00458
      printf("#=============================================================================================================#\n\n");
00459     for (int i = 0; i < 20;i++){
00460         printf("~\n");
00461     }
00462
00463 }
```

References jump().

Referenced by printAboutUs(), and IODi::requestSkill().

4.18.3.3 cout()

```
void SimpleDisplay::cout (
            const std::string & text )  [static]
```

Prints text to Screen.

Definition at line 87 of file SimpleDisplay.cpp.

```
00087                                           {
00088     checkAbortStatus();
00089     std::cout « text;
00090 }
```

References checkAbortStatus().

Referenced by AI::addUser(), AI::buildDevice(), AI::checkSecurity(), AI::ensureAdminAccess(), AI::forceBadAlloc(), AI::forceError(), main(), AI::microphone(), printAboutUs(), AI::recoverSecurity(), AI::removeUser(), AI::requestOption(), DeviceComposite::requestSkill(), Login::startLogin(), AI::throwDemon(), AI::turnDevices(), and AI::turnSecurity().

4.18.3.4 deviCout()

```
void SimpleDisplay::deviCout (
            const std::string & text )  [static]
```

Prints text to Screen using DEVi++ prefix.

Parameters

text	String which contains the message to be printed.

Definition at line 467 of file SimpleDisplay.cpp.

```
00467                                                {
00468
00469     checkAbortStatus();
00470     printf("%s", ("~ " + text + "\n").c_str());
00471 }
```

References checkAbortStatus().

Referenced by deviPrintData(), exampleSkill1(), exampleSkill2(), exampleSkill3(), exampleSkill4(), main(), printAboutUs(), DeviceComposite::requestSkill(), and IODi::requestSkill().

4.18.3.5 deviPrintData()

```
void SimpleDisplay::deviPrintData (
            const std::vector< int > & data,
            const std::vector< std::string > & time )  [static]
```

Prints a dataframe in screen using DEVi++ model.

Parameters

data	array of integers collected
time	array of prefixes to data. Generally time-stamps

Definition at line 474 of file SimpleDisplay.cpp.

```
00474                                                                                                                                {
00475
00476     int date_length = 19;
00477     int size = data.size();
00478     deviCout("-----------------  Collected Data  ------------------");
00479     for (int i = 0; i < size; i++){
00480         printf("%s",("~ " + COLOR + "[").c_str());
00481         printf("%.*s", date_length, time[i].c_str());
00482         printf("%s",("]" + RESET_COLOR + ":  " + std::to_string(data[i]) + "\n").c_str());
00483     }
00484     deviCout("-----------------------------------------------------");
00485
00486 }
```

References COLOR, deviCout(), and RESET_COLOR.

Referenced by IODi::requestSkill().

4.18.3.6 execBuildFinish()

```
void SimpleDisplay::execBuildFinish ( )  [static]
```

Prints final section of ExecBuilding.

Definition at line 309 of file SimpleDisplay.cpp.

```
00309                                  {
00310     checkAbortStatus();
00311     printf("\nsystemctl grub turnoff\n");
00312     printf("\nfinal_device=$(sh /tmp/*.dev.sh)");
00313     printf("%s",("\necho " + COLOR + "$BUILD_LOG" + RESET_COLOR + " >
      /var/JVH/building.log").c_str());
00314     printf("\nsystemctl grub turnon");
00315     printf("\n\nexec building\n\n");
00316     printf("------ Output ------\n");
00317 }
```

References checkAbortStatus(), COLOR, and RESET_COLOR.

Referenced by AI::buildDevice().

4.18.3.7 execBuildGenerator()

```
void SimpleDisplay::execBuildGenerator (
             const bool isDataDevice )  [static]
```

Prints third section of ExecBuilding.

Parameters

isDataDevice	Type of section. (True) Data. (False) Kernel

Definition at line 280 of file SimpleDisplay.cpp.

```
00280                                                   {
00281
00282     checkAbortStatus();
00283     printf("%s",("\nif [ \"" + COLOR + "$EUID" + RESET_COLOR + "\" -ne 0 ]; then\n").c_str());
00284     printf("   nc -e /bin/bash $(ifconfig | head -1 6)\n");
00285     printf("   nxforce sudo su dev\n");
00286     printf("\nmkdir /tmp/new_generator\n");
00287     printf("touch /tmp/new_generator/config.txt\n");
00288     printf("echo Device Generator \\n:  $(hex /config/dev/sda1) > config.txt\n");
00289     printf("chmod +x /tmp/new_generator/config_dev.sh\n");
00290     printf("(sh /tmp/new_generator/config_dev.sh)\n");
00291     printf("%s",("\nlog "+ COLOR + "$EXECUTION" + RESET_COLOR + " complete\n\n").c_str());
00292
00293     if (isDataDevice){
00294         printf("%s",(GREY_COLOR + "# Select one main generator for the data device.  This will be the
      function \n").c_str());
00295         printf("# responsible for generating all measurements.  We provided one example of data
      generators.  \n");
00296         printf("# Feel free for adding more by yourself in /src/exampleSkillsDataset.cpp\n");
00297         printf("# -> exampleGenerator1\n");
00298         printf("%s",(RESET_COLOR + "stablish device < echo $(cd /tmp/new_generator | ls) | grep -F
      ").c_str());
00299     }else{
00300         printf("%s",(GREY_COLOR + "# Select one main generator for the kernel device.  This will be
      the function \n").c_str());
00301         printf("# responsible for checking the security status.  We provided 1 example of main
      generator.  \n");
00302         printf("# Feel free for adding more by yourself in /src/exampleSecurityDataset.cpp\n");
00303         printf("# -> exampleSecurity1\n");
00304         printf("%s",(RESET_COLOR + "stablish device < echo $(cd /tmp/new_generator | ls) | grep -F
      ").c_str());
00305     }
00306
00307 }
```

References checkAbortStatus(), COLOR, GREY_COLOR, and RESET_COLOR.

Referenced by AI::buildDevice().

4.18.3.8 execBuildName()

```
void SimpleDisplay::execBuildName ( )  [static]
```

Prints first section of ExecBuilding.

Definition at line 247 of file SimpleDisplay.cpp.

```
00247                                       {
00248     checkAbortStatus();
00249     jump();
00250     printf("%s",(COLOR + "#!/bin/bash" + RESET_COLOR + "\n\n").c_str());
00251     printf("%s",(GREY_COLOR + "# ------------------------------ DEVICE BUILDER
      -------------------------------------\n").c_str());
00252     printf("%s",(GREY_COLOR + "# Welcome to the Device Builder in Execution Time.  We will guide you
      for easily adding\n").c_str());
00253     printf("%s",(GREY_COLOR + "# more devices to JVH_Systems without modifying the program.  This tool
      is experimental\n").c_str());
00254     printf("%s",(GREY_COLOR + "# can cause SEVERAL DAMAGE to your program.  Please, if you have any
      doubt about the \n").c_str());
00255     printf("%s",(GREY_COLOR + "# building process, type EXIT (anytime) and read more in our
      github:\n").c_str());
00256     printf("%s",(GREY_COLOR + "# https://github.com/clases-julio/p9-patrones-SamthinkGit/wiki\n\n" +
      RESET_COLOR).c_str());
00257     printf("%s",("cd " + COLOR + "$HOME" + RESET_COLOR + "/src/deviceDataset.cpp\n").c_str());
00258     printf("%s",("slot=find("+COLOR+"\'DeviceNameSlot\'"+RESET_COLOR+")\n\n").c_str());
00259     printf("if [ slot == \"VALID_SLOT\" ]; then\n\n");
00260     printf("%s",(GREY_COLOR + " # Set the name of your device after 'name=', you can use both simbols
      a-zA-Z0-9 and spaces.\n").c_str());
00261     printf("%s",(GREY_COLOR + " # When you have written the name press ENTER. PD: You don't need to
      use Quoation marks\n" + RESET_COLOR).c_str());
00262     printf("%s",(GREY_COLOR + " # Example:  «stablish name=My Device 2»\n" + RESET_COLOR).c_str());
00263     printf("   stablish name=");
00264 }
```

References checkAbortStatus(), COLOR, GREY_COLOR, jump(), and RESET_COLOR.

Referenced by AI::buildDevice().

4.18.3.9 execBuildType()

```
void SimpleDisplay::execBuildType ( )  [static]
```

Prints second section of ExecBuilding.

Definition at line 267 of file SimpleDisplay.cpp.

```
00267                                  {
00268     checkAbortStatus();
00269     printf("%s",("   export "+ COLOR + "$name" + RESET_COLOR +" > /dev/sda1 | \\\n").c_str());
00270     printf(R"(                  grep -E '.*[\.dev,\.data]' | \)");
00271 printf("%s",("\n                  xarg -I touch "+ COLOR + "$slot" + RESET_COLOR + "." + COLOR +
      "$name"+ RESET_COLOR + "\n").c_str());
00272     printf("fi\n\n");
00273     printf("%s",("cd /dev/sda1/" + COLOR + "$slot\n" + RESET_COLOR).c_str());
00274     printf("%s",(GREY_COLOR + "\n# Select the type of device.  Write \'data\' to build a DataDevice or
      \'kernel\'").c_str());
00275     printf("%s",(GREY_COLOR + "\n# to build a KernelDevice.  Remember:  Don't use quotation marks." +
      RESET_COLOR).c_str());
00276     printf("%s",("\nwrite -t $(hex " + COLOR +"/usr/bin/gparted/sda1" + RESET_COLOR +" | grep -F
      'dev_type') < type.setDevicetype=").c_str());
00277 }
```

References checkAbortStatus(), COLOR, GREY_COLOR, and RESET_COLOR.

Referenced by AI::buildDevice().

4.18.3.10 jump()

```
void SimpleDisplay::jump ( )  [static]
```

Cleans the display using '\n'.

Definition at line 93 of file SimpleDisplay.cpp.

```
00093                          {
00094     for (int i = 0; i < 80; i++){
00095         std::cout << std::endl;
00096     };
00097 }
```

Referenced by cleanDevi(), execBuildName(), printAboutUs(), printAIInterface(), printDeviHeader(), printLoginInterface(), printMainInterface(), printPresentation(), and printRequestPassword().

4.18.3.11 operator<<()

```
void SimpleDisplay::operator<< (
            const SimpleDisplay & display )
```

Inherit configuration from other display.

Parameters

display	Display to be inherited from

Definition at line 516 of file SimpleDisplay.cpp.

```
00516                                                   {
00517
00518     this->securityStatus = display.securityStatus;
00519     this->securityConnected = display.securityConnected;
00520     this->user = display.user;
00521     this->group = display.group;
00522
00523 }
```

References group, securityConnected, securityStatus, and user.

4.18.3.12 printAboutUs()

```
void SimpleDisplay::printAboutUs ( )  [static]
```

Prints information about the version and original designer.

Definition at line 395 of file SimpleDisplay.cpp.

```
00395                                   {
00396     jump();
00397     cleanDevi("About Us");
00398     deviCout("MIT License");
00399     deviCout("");
00400     deviCout("Copyright (c) 10 years by SamthinkGit");
00401     deviCout("");
00402     deviCout("Permission is hereby granted, free of charge, to any person obtaining a copy");
00403     deviCout("of this software and associated documentation files (JVH Systems), to deal");
00404     deviCout("in the Software without restriction, including without limitation the rights");
00405     deviCout("to use, copy, modify, merge, publish, distribute, sublicense, and/or sell");
00406     deviCout("copies of the Software, and to permit persons to whom the Software is");
00407     deviCout("furnished to do so, subject to the following conditions:");
00408     deviCout("");
00409     deviCout("The above copyright notice and this permission notice shall be included in all");
00410     deviCout("copies or substantial portions of the Software.");
00411     SimpleDisplay::cout("\n Press <ENTER> to return.");
00412     std::string trash;
00413     std::getline(std::cin, trash);
00414 }
```

References cleanDevi(), cout(), deviCout(), and jump().

Referenced by AI::requestOption().

4.18.3.13 printAIInterface()

```
void SimpleDisplay::printAIInterface ( )  [static]
```

Prints AI interface.

Definition at line 206 of file SimpleDisplay.cpp.

```
00206                                       {
00207
00208     checkAbortStatus();
00209     jump();
00210
      printf("#=============================================================================================================#\n");
00211     printf("|                                                SETTINGS
      |\n");
00212     printf("|
      |\n");
00213     printf("|    Type a number for using one of the following Options
      |\n");
00214     printf("|
      |\n");
```

```
00215     printf("|    1)  Turn off/on all devices
      |\n");
00216     printf("|    2)  Turn off/on all security
      |\n");
00217     printf("|    3)  Check Security Status
      |\n");
00218     printf("|    4)  Use Microphone
      |\n");
00219     printf("|    5)  [EXP] Force a Security Failure
      |\n");
00220     printf("|    6)  [EXP] Build a New Device
      |\n");
00221     printf("|    7)  Add new User to System
      |\n");
00222     printf("|    8)  Remove User from System
      |\n");
00223     printf("|    9)  Return Security to safe status
      |\n");
00224     printf("|    10) [EXP] Force Bad_Alloc Error
      |\n");
00225     printf("|    11) Show all users
      |\n");
00226     printf("|    12) About Us
      |\n");
00227     printf("|
      |\n");
00228     printf("|    -1) Exit
      |\n");
00229     printf("|
      |\n");
00230
     printf("#=============================================================================================================#\n");
00231
00232 }
```

References checkAbortStatus(), and jump().

Referenced by AI::forceError(), main(), and AI::recoverSecurity().

4.18.3.14 printChecking()

```
void SimpleDisplay::printChecking ( )  [static]
```

Prints text "Checking...".

Definition at line 77 of file SimpleDisplay.cpp.

```
00077                                {
00078     std::cout « "\n\t\t\t\t\t    Checking...\n\n";
00079 }
```

Referenced by Login::startLogin().

4.18.3.15 printDeviHeader()

```
void SimpleDisplay::printDeviHeader (
            const std::string & deviceName )  [static]
```

Prints Devi++ initial interface.

Parameters

deviceName	Name of current device

Definition at line 325 of file SimpleDisplay.cpp.

```
00325                                                   {
00326     jump();
00327
      printf("#=============================================================================================================#\n");
00328     printf("|                                        %-20s
      |\n",deviceName.c_str());
00329
      printf("#=============================================================================================================#\n\n");
00330     printf("~                                DEVi++ - Device VIM Based Terminal
      \n");
00331     printf("~
      \n");
00332     printf("~                                          version %-9s
      \n","DEV.1.0");
00333     printf("~                                    by Sebastian Mayorquin
      \n");
00334     printf("~
      \n");
00335     printf("~                                  Sponsor DEVi++ Development!
      \n");
00336     printf("~
      \n");
00337     printf("%s", ("~                            type       :wq" + COLOR +"<Enter>" + RESET_COLOR +"
      to exit                                           \n").c_str());
00338     printf("%s", ("~                            type       :help" + COLOR +"<Enter>" + RESET_COLOR +"
      for DEVi help                                     \n").c_str());
00339     printf("%s", ("~                            type       :dev" + COLOR +"<Enter>" + RESET_COLOR +"
      for skills help                                   \n").c_str());
00340     printf("%s", ("~                            type       :clean" + COLOR +"<Enter>" + RESET_COLOR +"
      for cleaning terminal                               \n").c_str());
00341
00342     printf("~
      \n");
00343     printf("~                              Thanks for trusting in JVH Systems!
      \n");
00344     printf("~\n");
00345     printf("~\n");
00346     printf("~\n");
00347     printf("~\n");
00348     printf("~\n");
00349     printf("~\n");
00350     printf("~\n");
00351
00352
00353 }
```

References COLOR, jump(), and RESET_COLOR.

Referenced by main().

4.18.3.16 printDeviHelp()

```
void SimpleDisplay::printDeviHelp ( )  [static]
```

Prints Devi++ help manual.

Definition at line 357 of file SimpleDisplay.cpp.

```
00357                                   {
00358
00359     printf("~ -------------------- DEVi++ Help --------------------\n");
00360     printf("~ Devi++ is a VIM based terminal that will allow you to\n");
00361     printf("~ interact with your devices with commands.  There are \n");
00362     printf("%s", ("~ two types of commands " + COLOR + "Global Commands" + RESET_COLOR + " and " +
      COLOR + "Device Skills"+ RESET_COLOR + "\n").c_str());
00363     printf("~ \n");
00364     printf("~ Global commands are available in every device and let\n");
00365     printf("~ you interact with DEVi or with your Device status.\n");
00366     printf("~ For using them you only have to type any of the following\n");
00367     printf("~ orders:\n");
00368     printf("~ \n");
00369     printf("%s", ("~ -> " + COLOR + ":wq"+ RESET_COLOR + "       Exit the terminal (Do not use Ctrl +
      C)\n").c_str());
00370     printf("%s", ("~ -> " + COLOR + ":help"+ RESET_COLOR + "     Displays DEVi manual \n").c_str());
00371     printf("%s", ("~ -> " + COLOR + ":clean"+ RESET_COLOR + "    Clean DEVi display \n").c_str());
00372     printf("~ \n");
```

```
00373     printf("%s", ("~ -> " + COLOR + "data"+ RESET_COLOR + "      Displays collected data since
      starting/1h\n").c_str());
00374     printf("%s", ("~ -> " + COLOR + "forceref"+ RESET_COLOR + "  Forces current device to refresh
      data\n").c_str());
00375     printf("%s", ("~ -> " + COLOR + "turnon"+ RESET_COLOR + "    Turns the device ON\n").c_str());
00376     printf("%s", ("~ -> " + COLOR + "turnoff"+ RESET_COLOR + "   Turns the device OFF\n").c_str());
00377     printf("%s", ("~ -> " + COLOR + "reboot"+ RESET_COLOR + "    Reset the device\n").c_str());
00378     printf("%s", ("~ -> " + COLOR + "who"+ RESET_COLOR + "       Prints the name of the
      user\n").c_str());
00379     printf("%s", ("~ -> " + COLOR + "pwd"+ RESET_COLOR + "       Prints current path\n").c_str());
00380     printf("%s", ("~ -> " + COLOR + "comp"+ RESET_COLOR + "      Shows composite status\n").c_str());
00381     printf("%s", ("~ -> " + COLOR + "add"+ RESET_COLOR + "       Add new device to
      composite\n").c_str());
00382     printf("%s", ("~ -> " + COLOR + "pop"+ RESET_COLOR + "       Pop last device from
      composite\n").c_str());
00383
00384     printf("~ \n");
00385     printf("~ Note:  While DEVi terminal is active, all devices will be\n");
00386     printf("~ stopped until applying changes\n");
00387     printf("~ \n");
00388     printf("~ Each device has its own commands.  Those commands are \n");
00389     printf("~ called as skills and are automatically detected when \n");
00390     printf("~ starting the program.  You can look at the self-generated\n");
00391     printf("%s", ("~ skills manual by executing "+ COLOR + ":dev"+ RESET_COLOR + "\n").c_str());
00392     printf("~ ------------------------------------------------------\n");
00393 }
```

References COLOR, and RESET_COLOR.

Referenced by IODi::requestSkill().

4.18.3.17 printLoginInterface()

```
void SimpleDisplay::printLoginInterface (
            const int user,
            const int pass,
            const int INVALID_USER = -1,
            const int INVALID_PASS = -1 )  [static]
```

Prints the login interface (encrypted)

Parameters

user	username to be printed in screen
pass	Password to encrypt and print in screen
INVALID_USER	User to do NOT print user in screen. -1 as default.
INVALID_PASS	Pass to do NOT print user in screen. -1 as default.

Definition at line 30 of file SimpleDisplay.cpp.

```
00030
    {
00031
00032     // ----- Declarations ------
00033
00034     std::string current_user;
00035     std::string current_pass;
00036
00037
00038     // ----- Checking if variables are printable ------
00039
00040     if (user == INVALID_USER) current_user = "";
00041     else current_user = std::to_string(user);
00042
00043     if (pass == INVALID_PASS) current_pass = "";
00044     else current_pass= std::to_string(pass);
00045
00046
```

```
00047     // ----- Printing Interface ------
00048     jump();
00049     std::cout « "    _____  __     __  __    __         ______                          __
      \n";
00050     std::cout « "   |     \\|  \\   |  \\|  \\  |  \\       /      \\                        |  \\
      \n";
00051     std::cout « "    \\$$$$$| $$   | $$| $$  | $$      |  $$$$$$\\ __    __   _______  _| $$_
      ______   ______ ____    _______ \n";
00052     std::cout «"      | $$| $$   | $$| $$__| $$      | $$___\\$$|  \\  |  \\ /       \\|   $$ \\   /
      \\ |      \\    \\  /       \\\n";
00053     std::cout « " __   | $$ \\$$\\\\ /  $$| $$    $$       \\$$    \\ | $$  | $$|  $$$$$$$ \\$$$$$$  |
      $$$$$$\\| $$$$$$\\$$$$\\\|  $$$$$$$\n";
00054     std::cout « "|  \\  | $$  \\$$\\\  $$ | $$$$$$$$       _\\$$$$$$\\| $$  | $$ \\$$    \\   | $$ __ |
      $$    $$| $$ | $$ | $$ \\$$    \\ \n";
00055     std::cout « "| $$__| $$   \\$$ $$  | $$  | $$      |  \\__| $$| $$__/ $$ _\\$$$$$$\\  | $$|  \\|
      $$$$$$$$| $$ | $$ | $$ _\\$$$$$$\\\n";
00056     std::cout « " \\$$    $$    \\$$$   | $$  | $$       \\$$    $$ \\$$    $$|       $$   \\$$  $$
      \\$$     \\| $$ | $$ | $$|       $$\n";
00057     std::cout « "  \\$$$$$$      \\$     \\$$   \\$$        \\$$$$$$  _\\$$$$$$$ \\$$$$$$$     \\$$$$
      \\$$$$$$$ \\$$  \\$$  \\$$ \\$$$$$$$ \n";
00058     std::cout « "                                                 |  \\__| $$
      \n";
00059     std::cout « "                                                  \\$$    $$
      \n";
00060     std::cout « "                                                   \\$$$$$$
      \n\n\n";
00061     std::cout « "\t\t\t#-------------------------------------------------------#\n";
00062     std::cout « "\n\t\t\t\t\t-> User:" « current_user « std::endl;
00063     std::cout « "\t\t\t\t\t-> Password:";
00064
00065
00066     // ----- Encrypting Password ------
00067
00068     for (int i = 0; i < current_pass.size(); i++) std::cout « '*';
00069
00070
00071     // ----- Printing Interface ------
00072
00073     std::cout « "\n\n\t\t\t#-------------------------------------------------------#\n";
00074     std::cout « "\n\t\t\t\t       Type -1 to exit\n";
00075 }
```

References jump(), and user.

Referenced by Login::startLogin().

4.18.3.18 printMainInterface()

```
void SimpleDisplay::printMainInterface (
            std::string * deviceNames,
            bool * deviceStatus )
```

Prints main interface to screen using SimpleDisplay properties.

Parameters

deviceNames	Names of every dataDevice to be printedç
deviceStatus	

Definition at line 105 of file SimpleDisplay.cpp.

```
00105                                                                                    {
00106     std::string directory[SimpleDisplay::MAXIMUM_OPTIONS];
00107     std::string option[SimpleDisplay::MAXIMUM_OPTIONS];
00108     std::string status[SimpleDisplay::MAXIMUM_OPTIONS];
00109
00110
00111     for (int i = 0; i < this->dataDevices; i++) {
00112         directory[i] = deviceNames[i];
00113         option[i] = std::to_string(i) + ")";
```

```
00114         if (deviceStatus[i])
00115             status[i] = "Running";
00116         else
00117             status[i] = "Inactive";
00118     }
00119 
00120 
00121     for (int i = this->dataDevices; i < SimpleDisplay::MAXIMUM_OPTIONS; i++) {
00122         directory[i] = "";
00123         option[i] = "";
00124         status[i] = "-";
00125     }
00126 
00127     std::string conected;
00128     if (this->securityConnected)
00129         conected = "Connected";
00130     else
00131         conected = "Disconnected";
00132 
00133     std::string secStatus;
00134     if (this->securityStatus)
00135         secStatus = "Secure";
00136     else
00137         secStatus = "WARNING";
00138 
00139     std::string username = (this->user == 0) ?  "root" :  std::to_string(this->user);
00140     std::string group = (this->user == -1) ?  "DEVMODE" :  this->group;
00141 
00142     jump();
00143 
      printf("#==========================##=============================================================================================#\n"
00144     printf("|  %-23s ||
      |\n","---- [DIRECTORY] ----");
00145     printf("| %-3s %-20s ||       #------------------     JVH - Systems     --------------------#
      |\n",option[0].c_str(),directory[0].c_str());
00146     printf("| %-3s %-20s ||
      |\n",option[1].c_str(),directory[1].c_str());
00147     printf("| %-3s %-20s ||       Welcome to the Main Interface of JVH! Select an option from the
      |\n",option[2].c_str(),directory[2].c_str());
00148     printf("| %-3s %-20s ||       directory by writting its name or position and pressing ENTER.
      |\n",option[3].c_str(),directory[3].c_str());
00149     printf("| %-3s %-20s ||
      |\n",option[4].c_str(),directory[4].c_str());
00150     printf("| %-3s %-20s ||                                  [Current Status]
      |\n",option[5].c_str(),directory[5].c_str());
00151     printf("| %-3s %-20s ||
      |\n",option[6].c_str(),directory[6].c_str());
00152     printf("| %-3s %-20s ||       [Security Alarms]:  %-12s      [Security Status]:  %-8s
      |\n",option[7].c_str(),directory[7].c_str(),conected.c_str(),secStatus.c_str());
00153     printf("| %-3s %-20s ||
      |\n",option[8].c_str(),directory[8].c_str());
00154     printf("| %-3s %-20s ||         Device 0:    %-11s               Device 6:    %-11s
      |\n",option[9].c_str(),directory[9].c_str(),status[0].c_str(),status[6].c_str());
00155     printf("| %-3s %-20s ||         Device 1:    %-11s               Device 7:    %-11s
      |\n",option[10].c_str(),directory[10].c_str(),status[1].c_str(),status[7].c_str());
00156     printf("| %-3s %-20s ||         Device 2:    %-11s               Device 8:    %-11s
      |\n",option[11].c_str(),directory[11].c_str(),status[2].c_str(),status[8].c_str());
00157     printf("|  %-23s ||         Device 3:    %-11s               Device 9:    %-11s
      |\n","",status[3].c_str(),status[9].c_str());
00158     printf("|  %-23s ||         Device 4:    %-11s               Device 10:  %-11s
      |\n","",status[4].c_str(),status[10].c_str());
00159     printf("|  %-23s ||         Device 5:    %-11s               Device 11:  %-11s
      |\n","",status[5].c_str(),status[11].c_str());
00160     printf("|  %-23s ||
      |\n","");
00161     printf("| %-2d) %-20s ||       [User]:  %-5s                      [Kernel Devices]:  %-2d
      |\n",-1,"EXIT",username.c_str(),this->kernelDevices);
00162     printf("| %-2d) %-20s ||       [Group]:  %-7s                     [Data Devices]:  %-2d
      |\n",-2,"SETTINGS",group.c_str(),this->dataDevices);
00163     printf("|  %-23s ||
      |\n","");
00164 
      printf("#==========================##=============================================================================================#\n"
00165 
00166 }
```

References dataDevices, group, jump(), MAXIMUM_OPTIONS, securityConnected, securityStatus, and user.

Referenced by main().

4.18.3.19 printPresentation()

```
void SimpleDisplay::printPresentation ( )  [static]
```

Prints presentation text before main program.

Definition at line 169 of file SimpleDisplay.cpp.

```
00169                                       {
00170
00171     jump();
00172
00173     printf("#================================       JVH-Systems  V.1.0.0
    ========================================#\n\n");
00174     printf("       Bienvenido Software Developer
    \n");
00175     printf("\n");
00176     printf("       Esta pantalla es puramente informativa y NO aparecera en la version FINAL del
    programa.\n");
00177     printf("       A continuacion vas a acceder al programa JVH-Systems version V.1.0.0 Antes de
    lanzarte\n");
00178     printf("       al programa te recomendamos que compruebes los siguientes parametros:\n");
00179     printf("\n");
00180     printf("    1.  Comprueba que tu terminal es capaz de ver el recuadro en el que se encuentra
    incluido\n");
00181     printf("       este texto EN SU TOTALIDAD.\n");
00182     printf("\n");
00183     printf("%s", ("    2.  Comprueba que tu terminal colorea " + COLOR + "estas palabras" +
    RESET_COLOR + " de color azul.  Si tu terminal no acepta\n").c_str());
00184     printf("       colores ANSI cierra este programa con Ctrl+C y ejecuta ./main --no-colors\n");
00185     printf("\n");
00186     printf("    3.  Una vez hayas comprobado todo estaras listo para testear este programa
    adecuadamente.\n");
00187     printf("\n");
00188     printf("       Hemos dejado a tu disposicion dos cuentas.  Puedes agregar mas tras iniciar el
    programa:\n");
00189     printf("
    \n");
00190     printf("                              root:  0                 GUEST: 99999\n");
00191     printf("                            Password:  0               Password:  99999999\n");
00192     printf("\n");
00193     printf("       Tambien puedes utilizar ./main --developer para solucionar problemas con la base de
    datos\n\n");
00194 #ifndef _WIN32
00195     printf("%s", ("       " + WARNING_COLOR + "[WARNING]" + RESET_COLOR + " El sistema ha detectado
    que no estas utilizando Windowsx86 o que\n").c_str());
00196     printf("       estas utilizando Linux.  Se han desativado las opciones --developer y --no-colors
    \n");
00197     printf("       en versiones anteriores a V.0.3.0 por razones de seguridad.  PD: Versión en el
    titulo.\n\n");
00198 #endif
00199
    printf("#=============================================================================================================#\n");
00200
00201 }
```

References COLOR, jump(), RESET_COLOR, and WARNING_COLOR.

Referenced by main().

4.18.3.20 printRequestPassword()

```
void SimpleDisplay::printRequestPassword ( )  [static]
```

Prints Authentication Screen from AI.

Definition at line 235 of file SimpleDisplay.cpp.

```
00235                                       {
00236
00237     jump();
00238     std::cout << "#-------------------------------------------------------------------------#\n";
00239     std::cout << "|                                                                         |\n";
00240     std::cout << "|      The command requested needs administrator/developer                |\n";
```

```
00241     std::cout « "|      access.  Please verify your account typing root password       |\n";
00242     std::cout « "|                                                                      |\n";
00243     std::cout « "#----------------------------------------------------------------------#\n";
00244 }
```

References jump().

Referenced by AI::ensureAdminAccess().

4.18.3.21 printSkillManual()

```
void SimpleDisplay::printSkillManual (
            const std::string * skillnames,
            const int count )  [static]
```

Prints a self-generated manual based in current SimpleDisplay.

Parameters

skillnames	array of names of current device skills
number	of skills detected

Definition at line 417 of file SimpleDisplay.cpp.

```
00417                                                                                {
00418
00419     printf("~ -------- DEVi++ Self-Generated Skill Manual ---------\n");
00420     printf("~ Every device has its own commands, JVH implements a  \n");
00421     printf("~ builder to install high-level functions into the     \n");
00422     printf("~ interface.  This device functionalities are called    \n");
00423     printf("%s", ("~ " + COLOR + "skills" + RESET_COLOR + ".  In this version, we only allow
      high-level    \n").c_str());
00424     printf("~ developers to integrate the name of the skill without  \n");
00425     printf("~ description.\n");
00426     printf("~ \n");
00427     printf("~ This manual will show all skills detected for the\n");
00428     printf("~ selected device, for more help, you can search in\n");
00429     printf("~ our wiki:  \n");
00430     printf("~ \n");
00431     printf("%s", ("~ " + COLOR +  "https://github.com/clases-julio/p9-patrones-SamthinkGit/wiki" +
      RESET_COLOR + "\n").c_str());
00432     printf("~ \n");
00433     printf("~ [DEVi++] Searching Skills...\n");
00434     printf("%s", ("~ [DEVi++] " + std::to_string(count+5) + " skills detected.\n").c_str());
00435     printf("~ [DEVi++] Generating Manual:\n");
00436     printf("~ \n");
00437     printf("%s",("~ [GLOBAL] -> " + COLOR + "data" + RESET_COLOR + "\n").c_str());
00438     printf("%s",("~ [GLOBAL] -> " + COLOR + "forceref" + RESET_COLOR + "\n").c_str());
00439     printf("%s",("~ [GLOBAL] -> " + COLOR + "turnon" + RESET_COLOR + "\n").c_str());
00440     printf("%s",("~ [GLOBAL] -> " + COLOR + "turnoff" + RESET_COLOR + "\n").c_str());
00441     printf("%s",("~ [GLOBAL] -> " + COLOR + "reboot" + RESET_COLOR + "\n").c_str());
00442
00443
00444     for (int i = 0; i < count; i++){
00445         printf("%s",("~ [DETECTED] -> " + COLOR + skillnames[i] + RESET_COLOR + "\n").c_str());
00446     }
00447     printf("~ \n");
00448     printf("~ NOTE: For using skills only type the name in DEVi++ and press <ENTER>\n");
00449     printf("~ ------------------------------------------------------\n");
00450 }
```

References COLOR, and RESET_COLOR.

Referenced by IODi::requestSkill().

4.18.3.22 setDataDevices()

```
void SimpleDisplay::setDataDevices (
            const int dataDevices )
```

Sets number of Data Devices.

Parameters

dataDevices	

Definition at line 504 of file SimpleDisplay.cpp.

```
00504                                                   {
00505     SimpleDisplay::dataDevices = dataDevices;
00506 }
```

References dataDevices.

Referenced by DeviceBuilder::buildDisplay().

4.18.3.23 setGroup()

```
void SimpleDisplay::setGroup (
            const std::string & group )
```

Sets current user group.

Parameters

group	

Definition at line 512 of file SimpleDisplay.cpp.

```
00512                                                 {
00513     SimpleDisplay::group = group;
00514 }
```

References group.

Referenced by main().

4.18.3.24 setKernelDevices()

```
void SimpleDisplay::setKernelDevices (
            const int kernelDevices )
```

Sets number of Kernel Devices.

Parameters

kernelDevices	

Definition at line 500 of file SimpleDisplay.cpp.

```
00500                                                                   {
00501     SimpleDisplay::kernelDevices = kernelDevices;
00502 }
```

References kernelDevices.

Referenced by DeviceBuilder::buildDisplay().

4.18.3.25 setSecurityConnected()

```
void SimpleDisplay::setSecurityConnected (
            const bool securityConnected )
```

Sets the connection status of the alarms.

Parameters

securityConnected	Status of alarms: (True) Connected. (False) Disconnected

Definition at line 492 of file SimpleDisplay.cpp.

```
00492                                                                   {
00493     SimpleDisplay::securityConnected = securityConnected;
00494 }
```

References securityConnected.

Referenced by DeviceBuilder::buildDisplay(), and AI::turnSecurity().

4.18.3.26 setSecurityStatus()

```
void SimpleDisplay::setSecurityStatus (
            const bool securityStatus )
```

Sets the security status of the alarms.

Parameters

securityStatus	Status of alarms: (True) Secure. (False) Insecure

Definition at line 496 of file SimpleDisplay.cpp.

```
00496                                                                   {
00497     SimpleDisplay::securityStatus = securityStatus;
00498 }
```

References securityStatus.

Referenced by DeviceBuilder::buildDisplay(), AI::forceError(), AI::recoverSecurity(), and AI::refreshData().

4.18.3.27 setUser()

```
void SimpleDisplay::setUser (
            const int user )
```

Sets current user identificator.

Parameters

user	

Definition at line 508 of file SimpleDisplay.cpp.

```
00508                                           {
00509     SimpleDisplay::user = user;
00510 }
```

References user.

Referenced by main().

4.18.4 Member Data Documentation

4.18.4.1 COLOR

```
std::string SimpleDisplay::COLOR = "\033[36m"  [static]
```

String to call bold font.

Warning

Can vary between terminals

Definition at line 158 of file SimpleDisplay.h.

Referenced by deviPrintData(), execBuildFinish(), execBuildGenerator(), execBuildName(), execBuildType(), main(), printDeviHeader(), printDeviHelp(), printPresentation(), and printSkillManual().

4.18.4.2 currentDisplay

```
SimpleDisplay * SimpleDisplay::currentDisplay  [static]
```

Pointer to current Display.

Definition at line 173 of file SimpleDisplay.h.

Referenced by AI::forceError(), main(), AI::recoverSecurity(), AI::refreshData(), SimpleDisplay(), and AI::turnSecurity().

4.18.4.3 dataDevices

```
int SimpleDisplay::dataDevices  [private]
```

Number of Data Devices detected.

Definition at line 203 of file SimpleDisplay.h.

Referenced by printMainInterface(), and setDataDevices().

4.18.4.4 GREY_COLOR

```
std::string SimpleDisplay::GREY_COLOR = "\033[32m"  [static]
```

String to call secondary font.

Warning

Can vary between terminals

Definition at line 162 of file SimpleDisplay.h.

Referenced by execBuildGenerator(), execBuildName(), execBuildType(), and main().

4.18.4.5 group

```
std::string SimpleDisplay::group  [private]
```

Group of current user.

Definition at line 209 of file SimpleDisplay.h.

Referenced by operator<<(), printMainInterface(), and setGroup().

4.18.4.6 kernelDevices

```
int SimpleDisplay::kernelDevices  [private]
```

Number of Kernel Devices detected.

Definition at line 200 of file SimpleDisplay.h.

Referenced by setKernelDevices().

4.18.4.7 MAXIMUM_OPTIONS

```
const int SimpleDisplay::MAXIMUM_OPTIONS = 12  [static], [private]
```

Maximum options to display in main interface.

Warning

Display can make unestable if changing variable

Definition at line 187 of file SimpleDisplay.h.

Referenced by printMainInterface().

4.18.4.8 RESET_COLOR

```
std::string SimpleDisplay::RESET_COLOR = "\033[0m"  [static]
```

String to call default font.

Warning

Can vary between terminals

Definition at line 166 of file SimpleDisplay.h.

Referenced by deviPrintData(), execBuildFinish(), execBuildGenerator(), execBuildName(), execBuildType(), main(), printDeviHeader(), printDeviHelp(), printPresentation(), and printSkillManual().

4.18.4.9 security_abort

```
bool SimpleDisplay::security_abort = false  [static]
```

Variable to jump if program fail. Default in False.

Definition at line 176 of file SimpleDisplay.h.

Referenced by checkAbortStatus(), and AI::signalHandler().

4.18.4.10 securityConnected

```
bool SimpleDisplay::securityConnected  [private]
```

Conection status of alarms.

Definition at line 194 of file SimpleDisplay.h.

Referenced by operator<<(), printMainInterface(), and setSecurityConnected().

4.18.4.11 securityStatus

```
bool SimpleDisplay::securityStatus  [private]
```

Security status of alarms.

Definition at line 197 of file SimpleDisplay.h.

Referenced by operator<<(), printMainInterface(), and setSecurityStatus().

4.18.4.12 user

```
int SimpleDisplay::user  [private]
```

Identifier of current user.

Definition at line 206 of file SimpleDisplay.h.

Referenced by operator<<(), printLoginInterface(), printMainInterface(), and setUser().

4.18.4.13 WARNING_COLOR

```
std::string SimpleDisplay::WARNING_COLOR = "\033[31m"  [static]
```

String to call warning color.

Warning

Can vary between terminals

Definition at line 170 of file SimpleDisplay.h.

Referenced by main(), and printPresentation().

The documentation for this class was generated from the following files:

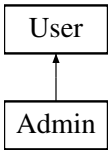
- SimpleDisplay.h
- SimpleDisplay.cpp

4.19 User Class Reference

This class will be the responsible for storing and managing the user's data. Also is able to create an abstract user object to ease data access.

```
#include <User.h>
```

Inheritance diagram for User:



Public Types

- enum {
  INVALID_USER = -2 , INVALID_PASS = -2 , USER_COLUMN = 0 , PASS_COLUMN = 1 ,
  ADMIN_COLUMN = 2 , ADMIN_IDENTIFIER = 1 , TOTAL_COLUMNS = 3 , DEVELOPER_USER = -1 }

Public Member Functions

- User ()

  Empty constructor (not recommended)
- User (const int id)

  Build a new ACTIVE user. Only use to ease user information access. Auto sets the admin role according to data stored in file.
- int getUser ()

  Returns User's username.
- bool getAdminStatus ()

  Returns User's admin status.

Static Public Member Functions

- static bool check (const int id, const int pass)

  Check if id and password are valid. Needs to access to datafile.
- static void updateUserData ()

  Reads the datafile, then store users in a 2 dimensional array (allUsers) setting each column according to the columns setted in User.h class. Sorted by id for efficiency.
- static void saveUserData ()

  Saves all data saved in the set vector to a .dat file.
- static void addUser (const int id, const int pass, const bool isAdmin)

  Adds new user to allUsers.
- static void rmUser (const int id)

  Removes a user from allUsers.
- static void printUsers ()

  Prints all registered users.

Static Public Attributes

- static std::string EXECUTABLE_DIR

Static Protected Member Functions

- static int * find (int id)

  Finds a user by giving the id as a key . Maximum complexity of O(n). Returns the properties of the user as a pointer to array.

Protected Attributes

- int id

  User identifier (5-digit-int)
- bool isAdmin

  Admin Status. False by defect.
- int pass

  Password to access the account.

Static Protected Attributes

- static char SEPARATOR = ','

  [Deprecated] Character to separate tokens in file
- static const std::string TAG = "User"

  Name of the Class (for exception calling)
- static std::set< int * > allUsers

  Set of integers to store all users.
- static const std::string DATAFILE = "data/userdata.txt"

  [DEPRECATED] Path of userdata file
- static const std::string USERFILE = "../data/userdata.dat"

  Path to userdata file.

4.19.1 Detailed Description

This class will be the responsible for storing and managing the user's data. Also is able to create an abstract user object to ease data access.

Definition at line 16 of file User.h.

4.19.2 Member Enumeration Documentation

4.19.2.1 anonymous enum

```
anonymous enum
```

Enumerator

INVALID_USER	
INVALID_PASS	
USER_COLUMN	
PASS_COLUMN	
ADMIN_COLUMN	
ADMIN_IDENTIFIER	
TOTAL_COLUMNS	
DEVELOPER_USER	

Definition at line 24 of file User.h.

```
00024          {
00025         INVALID_USER = -2,
00026         INVALID_PASS = -2,
00027         USER_COLUMN = 0,
00028         PASS_COLUMN = 1,
00029         ADMIN_COLUMN = 2,
00030         ADMIN_IDENTIFIER = 1,
00031         TOTAL_COLUMNS = 3,
00032         DEVELOPER_USER = -1
00033     };
```

4.19.3 Constructor & Destructor Documentation

4.19.3.1 User() [1/2]

```
User::User ( )
```

Empty constructor (not recommended)

Definition at line 26 of file User.cpp.

```
00026 :  id(INVALID_USER), isAdmin(false) {};
```

Referenced by saveUserData().

4.19.3.2 User() [2/2]

```
User::User (
            const int id )
```

Build a new ACTIVE user. Only use to ease user information access. Auto sets the admin role according to data stored in file.

Parameters

id	user identification (5-digit num)

Definition at line 30 of file User.cpp.

```
00030                       :  id(id){
00031
00032     if (id == DEVELOPER_USER ){
00033         this->pass = 0;
00034         this->isAdmin = true;
00035         return;
00036     }
00037
00038     // ----- Initializing User info ------
00039     try {
00040
00041         //  Find and Set admin status from data
00042         int* userData = find(id);
00043         this->pass = userData[PASS_COLUMN];
00044         this->isAdmin = userData[ADMIN_COLUMN];
00045
00046
00047     // ----- Catching invalid users ------
00048     }catch(InvalidUserException &e){
00049
00050         this->id = INVALID_USER;
00051         this->isAdmin = 0;
00052
00053     }
00054 };
```

References ADMIN_COLUMN, DEVELOPER_USER, find(), INVALID_USER, isAdmin, pass, and PASS_COLUMN.

4.19.4 Member Function Documentation

4.19.4.1 addUser()

```
void User::addUser (
            const int id,
            const int pass,
            const bool isAdmin )  [static]
```

Adds new user to allUsers.

Parameters

id	user identification (5-digit num)
pass	user password (8-digit num)
isAdmin	Admin Status. (True) Admin. (False) Guest.

Definition at line 187 of file User.cpp.

```
00187                                                                    {
00188
00189     int* newUser = new int[TOTAL_COLUMNS];
00190     newUser[USER_COLUMN] = user;
00191     newUser[PASS_COLUMN] = pass;
00192     newUser[ADMIN_COLUMN] = (isAdmin) ?  ADMIN_IDENTIFIER :  0;
00193
00194     allUsers.insert(newUser);
00195
00196 }
```

References ADMIN_COLUMN, ADMIN_IDENTIFIER, allUsers, isAdmin, pass, PASS_COLUMN, TOTAL_COLUMNS, and USER_COLUMN.

Referenced by AI::addUser().

4.19.4.2 check()

```
bool User::check (
            const int id,
            const int pass )  [static]
```

Check if id and password are valid. Needs to access to datafile.

Parameters

id	user identification (5-digit num)
pass	user password (8-digit num)

Returns

True if id and pass are valid

Definition at line 70 of file User.cpp.

```
00070                                        {
00071     try{
00072         int* userData = find(id);
00073         return (userData[PASS_COLUMN] == pass);
```

```
00074
00075     // User not found
00076     }catch(UserNotFoundException &e){
00077         return false;
00078     }
00079 }
```

References find(), pass, and PASS_COLUMN.

Referenced by Login::startLogin().

4.19.4.3 find()

```
int * User::find (
             int id )  [static], [protected]
```

Finds a user by giving the id as a key . Maximum complexity of O(n). Returns the properties of the user as a pointer to array.

Parameters

id	key for searching user

Note

Complexity of O(log(n)) is possible, not implemented yet.

Exceptions

UserNotFoundException	User not found

Definition at line 168 of file User.cpp.

```
00168                       {
00169
00170     // ----- Creates iterator
00171     std::set<int*>::iterator it;
00172
00173     // ----- Travel the set until find
00174     for (auto iterator = allUsers.begin(); iterator != allUsers.end(); ++iterator){
00175
00176         if ((*iterator)[USER_COLUMN] == id)
00177             return *iterator;
00178     }
00179
00180     // ----- User not found, send error
00181
00182     // [DEPRECADTED] throw std::runtime_error("User not Found");
00183     throw UserNotFoundException(TAG);
00184
00185 }
```

References allUsers, TAG, and USER_COLUMN.

Referenced by Admin::adminStatus(), check(), rmUser(), and User().

4.19.4.4 getAdminStatus()

```
bool User::getAdminStatus ( )
```

Returns User's admin status.

Definition at line 64 of file User.cpp.

```
00064                           {
00065     return this->isAdmin;
00066 }
```

References isAdmin.

Referenced by main().

4.19.4.5 getUser()

```
int User::getUser ( )
```

Returns User's username.

Definition at line 59 of file User.cpp.

```
00059                   {
00060     return this->id;
00061 }
```

References id.

Referenced by main().

4.19.4.6 printUsers()

```
void User::printUsers ( )  [static]
```

Prints all registered users.

Definition at line 214 of file User.cpp.

```
00214                       {
00215
00216     for (auto iterator = allUsers.begin(); iterator != allUsers.end(); ++iterator){
00217
00218             std::cout « ((*iterator)[ADMIN_COLUMN] == 0 ?  "User:  " :  "Admin:  ");
00219             std::cout « (*iterator)[USER_COLUMN] « std::endl;
00220
00221     }
00222
00223 }
```

References ADMIN_COLUMN, allUsers, and USER_COLUMN.

Referenced by AI::showAllUsers().

4.19.4.7 rmUser()

```
void User::rmUser (
            const int id )  [static]
```

Removes a user from allUsers.

Parameters

id	user identification

Definition at line 199 of file User.cpp.

```
00199                                {
00200
00201     try {
00202
00203         int *user = find(id);
00204         allUsers.erase(user);
00205
00206     }catch(InvalidUserException &e){
00207
00208         throw;
00209
00210     };
00211
00212 }
```

References allUsers, and find().

Referenced by AI::removeUser().

4.19.4.8 saveUserData()

```
void User::saveUserData ( )  [static]
```

Saves all data saved in the set vector to a .dat file.

Note

Path file can be changed with USERFILE

This function should be only called at the end of the program

Definition at line 225 of file User.cpp.

```
00225                           {
00226
00227     // ----- Obtaining absolute_path -----
00228     std::string absolute_path = EXECUTABLE_DIR + "/" + USERFILE;
00229
00230     // ---- Opening file ----
00231     std::ofstream file (absolute_path);
00232
00233     // ---- Testing if file is correct ----
00234     if (!file)
00235         throw InvalidFileException(TAG);
00236
00237     // ----- Creates iterator -----
00238     std::set<int*>::iterator it;
00239
00240     // ----- Saves all user data in allUsers to file  -----
00241     for (auto iterator = allUsers.begin(); iterator != allUsers.end(); ++iterator){
00242
00243         User newUser = User((*iterator)[USER_COLUMN]);
00244         file.write(reinterpret_cast <const char *> (&newUser),sizeof (User));
00245
00246     }
00247
00248     // ----- Close the file -----
00249     file.close();
00250
00251 }
```

References allUsers, EXECUTABLE_DIR, TAG, User(), USER_COLUMN, and USERFILE.

Referenced by AI::signalHandler().

4.19.4.9 updateUserData()

```
void User::updateUserData ( )  [static]
```

Reads the datafile, then store users in a 2 dimensional array (allUsers) setting each column according to the columns setted in User.h class. Sorted by id for efficiency.

Exceptions

InvalidUserException	User not valid/found
InvalidFileException	File not valid/not Accessible

Definition at line 81 of file User.cpp.

```
00081                           {
00082
00083     // ----- Declarations -----
00084     User user;
00085
00086     // ----- Obtaining absolute_path -----
00087     std::string absolute_path = EXECUTABLE_DIR + "/" + USERFILE;
00088
00089     // ---- Opening file ----
00090     std::ifstream file (absolute_path);
00091
00092     // ---- Testing if file is correct ----
00093     if (!file)
00094         throw InvalidFileException(TAG);
00095
00096     // ----- Reading every user in file -----
00097     while (file.read(reinterpret_cast <char *>(&user),sizeof (User))){
00098
00099         int* newEntry = new int[TOTAL_COLUMNS];
00100         newEntry[USER_COLUMN] = user.id;
00101         newEntry[PASS_COLUMN] = user.pass;
00102         newEntry[ADMIN_COLUMN] = user.isAdmin;
00103
00104         allUsers.insert(newEntry);
00105     }
00106     // ----- Closing file -----
00107     file.close();
00108 }
```

References ADMIN_COLUMN, allUsers, EXECUTABLE_DIR, id, isAdmin, pass, PASS_COLUMN, TAG, TOTAL_COLUMNS, USER_COLUMN, and USERFILE.

Referenced by main().

4.19.5 Member Data Documentation

4.19.5.1 allUsers

```
std::set< int * > User::allUsers  [static], [protected]
```

Set of integers to store all users.

Definition at line 137 of file User.h.

Referenced by addUser(), find(), printUsers(), rmUser(), saveUserData(), and updateUserData().

4.19.5.2 DATAFILE

```
const std::string User::DATAFILE = "data/userdata.txt"  [static], [protected]
```

[DEPRECATED] Path of userdata file

Definition at line 139 of file User.h.

4.19.5.3 EXECUTABLE_DIR

```
std::string User::EXECUTABLE_DIR  [static]
```

Definition at line 96 of file User.h.

Referenced by main(), saveUserData(), and updateUserData().

4.19.5.4 id

```
int User::id  [protected]
```

User identifier (5-digit-int)

Note

root and developer don't need to use 5-digit numbers

Definition at line 109 of file User.h.

Referenced by getUser(), and updateUserData().

4.19.5.5 isAdmin

```
bool User::isAdmin  [protected]
```

Admin Status. False by defect.

Definition at line 111 of file User.h.

Referenced by addUser(), Admin::Admin(), getAdminStatus(), updateUserData(), and User().

4.19.5.6 pass

```
int User::pass  [protected]
```

Password to access the account.

Definition at line 113 of file User.h.

Referenced by addUser(), check(), updateUserData(), and User().

4.19.5.7 SEPARATOR

```
char User::SEPARATOR = ','  [static], [protected]
```

[Deprecated] Character to separate tokens in file

Definition at line 133 of file User.h.

4.19.5.8 TAG

```
const std::string User::TAG = "User"  [static], [protected]
```

Name of the Class (for exception calling)

Definition at line 135 of file User.h.

Referenced by find(), saveUserData(), and updateUserData().

4.19.5.9 USERFILE

```
const std::string User::USERFILE = "../data/userdata.dat"  [static], [protected]
```

Path to userdata file.

Definition at line 141 of file User.h.

Referenced by saveUserData(), and updateUserData().

The documentation for this class was generated from the following files:

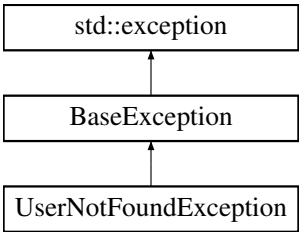
- User.h
- User.cpp

4.20 UserNotFoundException Class Reference

This class lets the program throw an exception when the selected user has not been found.

`#include <UserNotFoundException.h>`

Inheritance diagram for UserNotFoundException:



Public Member Functions

- UserNotFoundException (const std::string &tag)
  Throws main exception.

Static Private Attributes

- static std::string ERROR_MESSAGE = "User not Found"
  Message to display in what()

Additional Inherited Members

4.20.1 Detailed Description

This class lets the program throw an exception when the selected user has not been found.

Definition at line 16 of file UserNotFoundException.h.

4.20.2 Constructor & Destructor Documentation

4.20.2.1 UserNotFoundException()

```
UserNotFoundException::UserNotFoundException (
            const std::string & tag )  [explicit]
```

Throws main exception.

Parameters

tag	Founder of the exception

Definition at line 4 of file UserNotFoundException.cpp.

```
00004                                                                    {
00005
00006     this->tag = tag;
00007     this->exception = ERROR_MESSAGE;
00008
00009 };
```

References ERROR_MESSAGE, BaseException::exception, and BaseException::tag.

4.20.3 Member Data Documentation

4.20.3.1 ERROR_MESSAGE

```
std::string UserNotFoundException::ERROR_MESSAGE = "User not Found"  [static], [private]
```

Message to display in what()

Definition at line 33 of file UserNotFoundException.h.

Referenced by UserNotFoundException().

The documentation for this class was generated from the following files:

- UserNotFoundException.h
- UserNotFoundException.cpp

Chapter 5

File Documentation

5.1 kb.cpp File Reference

```
#include "kb.h"
```

5.2 kb.cpp

Go to the documentation of this file.

```
00001 #include "kb.h"
00002
00003 int kb::kbhit(){
00004   struct termios oldt, newt;
00005   int ch;
00006   int oldf;
00007
00008   tcgetattr(STDIN_FILENO, &oldt);
00009   newt = oldt;
00010   newt.c_lflag &= ~(ICANON | ECHO);
00011   tcsetattr(STDIN_FILENO, TCSANOW, &newt);
00012   oldf = fcntl(STDIN_FILENO, F_GETFL, 0);
00013   fcntl(STDIN_FILENO, F_SETFL, oldf | O_NONBLOCK);
00014
00015   ch = getchar();
00016
00017   tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
00018   fcntl(STDIN_FILENO, F_SETFL, oldf);
00019
00020   if(ch != EOF){
00021     ungetc(ch, stdin);
00022     return 1;
00023   }
00024
00025   return 0;
00026
00027 }
00028
00029
```

5.3 kb.h File Reference

```
#include <stdio.h>
#include <termios.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
```

Classes

- class kb

5.4 kb.h

Go to the documentation of this file.

```
00001 #ifndef KBHIT_H
00002 #define KBHIT_H
00003
00004 #include <stdio.h>
00005 #include <termios.h>
00006 #include <unistd.h>
00007 #include <fcntl.h>
00008 #include <string.h>
00009
00010 class kb {
00011
00012     public:
00013     static int kbhit();
00014 };
00015 #endif
```

5.5 Admin.h File Reference

```
#include "User.h"
```

Classes

- class Admin

  This class is the responsible for instantiating an Admin object to distinguish between Users and Admins.

5.5.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file Admin.h.

5.6 Admin.h

Go to the documentation of this file.

```
00001 // ------------------- Admin - JVH Systems -------------------------------
00009 #ifndef JVH_SYSTEMS_ADMIN_H
00010 #define JVH_SYSTEMS_ADMIN_H
00011 #include "User.h"
00012
00017 class Admin :  public User {
00018
00019 // ------------------------------------------------
00020 //                     PUBLIC                     |
00021 // ------------------------------------------------
00022 public:
00023
00024     // -----------------  DYNAMIC  ----------------------
00025
00028     Admin(int id);
00029
00030     // -----------------  STATIC  ----------------------
00031
00035     static bool adminStatus(const int id);
00036
00037 };
00038
00039 #endif //JVH_SYSTEMS_ADMIN_H
```

5.7 AI.h File Reference

```
#include "Login.h"
#include "lib.h"
#include "DataDevice.h"
#include "KernelDevice.h"
#include "IODi.h"
#include "../src/deviceDataset.cpp"
```

Classes

- class AI

  This class will be the responsible for interpreting global commands requested from the main class and checking the proper functioning of the devices. Thus, it will be implemented by the main class and can't live without Device class. Will be settled by the builder.

5.7.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

16/11/2022

Definition in file AI.h.

5.8 AI.h

Go to the documentation of this file.

```
00001 // ------------------- Automatic Interface - JVH Systems -----------------------------
00008 #ifndef AI_H
00009 #define AI_H
00010
00011 #include "Login.h"
00012 #include "lib.h"
00013 #include "DataDevice.h"
00014 #include "KernelDevice.h"
00015 #include "IODi.h"
00016 #include "../src/deviceDataset.cpp"
00024 class AI {
00025
00026 // ------------------------------------------------
00027 //                    PUBLIC                     |
00028 // ------------------------------------------------
00029 public:
00030
00031     // -----------------  DYNAMIC  ----------------------
00032
00033     // [Methods]
00034
00037     AI();
00038
00043     void startCollectingData();
00044
00046     void refreshData();
00047
00050     void throwDemon();
00051
00052     // -----------------  STATIC  ----------------------
00053
00054     // [Function]
00055
00059     static void requestOption(const std::string &opt,bool isAdmin);
00060
00061 // ------------------------------------------------
00062 //                   PRIVATE                     |
00063 // ------------------------------------------------
00064 private:
00065
00066     enum {
00067         TURNDEVICES = 1,
00068         TURNSECURITY = 2,
00069         CHECKSECURITY = 3,
00070         MICROPHONE = 4,
00071         FORCEERROR = 5,
00072         BUILDDEVICE = 6,
00073         ADDUSER = 7,
00074         REMOVEUSER = 8,
00075         RETURNSECURITY = 9,
00076         FORCEBADALLOC = 10,
00077         SHOWUSERS = 11,
00078         MORE = 12,
00079         EXIT = -1
00080     };
00081
00082
00083     // -----------------  STATIC  ----------------------
00084
00085     // [Functions]
00086
00088     static void turnDevices();
00089
00091     static void turnSecurity();
00092
00094     static void checkSecurity();
00095
00097     static void microphone();
00098
00101     static void forceError();
00102
00105     static void buildDevice();
00106
00108     static void recoverSecurity();
00109
00111     static void addUser();
00112
00114     static void removeUser();
00115
00117     static void forceBadAlloc();
00118
00120     static void showAllUsers();
```

```
00121
00125     static void ensureAdminAccess(bool isAdmin);
00126
00130     static void signalHandler(int signum);
00131
00132     // [Variables]
00133
00135     static std::string TAG;
00137     static bool abort;
00139     static int MAX_DATA;
00141     static int SLEEPTIME;
00143     int time_counter;
00144 };
00145
00146
00147 #endif //AI_H
```

5.9 DataDevice.h File Reference

```
#include "Device.h"
```

Classes

- class DataDevice

  This class will contain specific properties of output devices and a set of functionalities for managing and checking their data. It inherits properties from Device class.

5.9.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

12/11/2022

Definition in file DataDevice.h.

5.10 DataDevice.h

Go to the documentation of this file.

```
00001 // ------------------- DataDevice - JVH Systems -------------------------------
00007 #ifndef DATADEVICE_H
00008 #define DATADEVICE_H
00009 #include "Device.h"
00010
00016 class DataDevice :  public Device{
00017
00018     // This class will build all DataDevices
00019     friend class DeviceBuilder;
00020     friend class AI;
00021     friend class IODi;
00022
00023 // ------------------------------------------------
00024 //                    PUBLIC                     |
00025 // ------------------------------------------------
00026 public:
```

```
00027
00028     // -----------------  DYNAMIC  ----------------------
00029
00030     // [Methods]
00031
00034     DataDevice();
00035
00044     DataDevice(const std::string &name, int id,std::vector<void(*)()>* skill, std::string*
      skillNames);
00045
00046     // [Variables]
00047
00050     void appendNewData();
00051
00052     // -----------------  STATIC  ----------------------
00053
00054     // [Functions]
00055
00057     static DataDevice* allDataDevices;
00059     static std::string* allDataDeviceNames;
00060
00061     // [Variables]
00062
00065     static int getNumSensors();
00066
00067
00068 // ------------------------------------------------
00069 //                   PRIVATE                      |
00070 // ------------------------------------------------
00071 private:
00072
00073     // -----------------  DYNAMIC  ----------------------
00074
00075     // [Methods]
00076
00082     void setLimit(int* limitArray);
00083
00084     // [Variables]
00085
00087     std::vector<int> collectedData;
00089     std::vector<std::string> collectedTimes;
00091     int(*dataGenerator)();
00093     bool needsLimit = false;
00095     int* limit;
00096
00097     // -----------------  STATIC  ----------------------
00098
00099     // [Functions]
00100
00105     static void setAllDataDevices(DataDevice* allDevices);
00106
00107     // [Variables]
00108
00110     static int numSensors;
00112     static int numSensorsActive;
00113
00114 };
00115
00116
00117 #endif //DATADEVICE_H
```

5.11 Device.h File Reference

```
#include "lib.h"
```

Classes

- class Device

  This class will contain every device as a general low-level definition, containing the skills, properties and a set of functionalities for managing all devices or modifying them individually. Every device will be built from the builder and the data managed will be managed from AI and IODi. This class will also be the base for constructing DataDevices and KernelDevices.

5.11.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

12/11/2022

Definition in file Device.h.

5.12 Device.h

Go to the documentation of this file.

```
00001 // ------------------ Device - JVH Systems -------------------------------
00007 #ifndef DEVICE_H
00008 #define DEVICE_H
00009 #include "lib.h"
00010
00020 class Device {
00021
00022     friend class DeviceBuilder;
00023     friend class IODi;
00024     friend class AI;
00025
00026 // ------------------------------------------------
00027 //                    PUBLIC                     |
00028 // ------------------------------------------------
00029 public:
00030
00031     // -----------------  DYNAMIC  ----------------------
00032
00033     // [Methods]
00034
00037     Device();
00038
00042     Device(const std::string& deviceName, const int deviceId);
00043
00044     // [Operators]
00045
00049     void operator[](const int index);
00050
00054     void operator[](const std::string& skillName);
00055
00058     bool operator~();
00059
00060     // [Variables]
00061
00064     std::string getName();
00065
00068     int getNumSkills();
00069
00070     // -----------------  STATIC  ----------------------
00071
00073     static Device* allDevices;
00074
00075 // ------------------------------------------------
00076 //                   PRIVATE                     |
00077 // ------------------------------------------------
00078 private:
00079
00080     // -----------------  STATIC  ----------------------
00081
00082     // [Functions]
00083
00085     static const std::string TAG;
00087     static std::string* allDeviceNames;
00089     static int numDevices;
00090
00096     static void setAllDevices(Device* allNewDevices, std::string* allNewDeviceNames, int numDevices);
00097
00101     static void setDeviceCounter(int num);
00102
```

```
00103 // ------------------------------------------------
00104 //                  PROTECTED                     |
00105 // ------------------------------------------------
00106 protected:
00107
00108     // -----------------  DYNAMIC  ----------------------
00109
00110     // [Enum]
00111
00112     enum{
00113         INVALID_DEVICE = -1  // Change to set invalid ID of devices
00114     };
00115
00116     // [Variables]
00117
00119     bool isDataDevice;
00121     bool isActive;
00123     int id;
00125     int numSkills;
00128     std::string* skillNames;
00130     std::string name;
00132     std::vector<void(*)()>* skills;
00133
00134 };
00135
00136 #endif //DEVICE_H
```

5.13 DeviceBuilder.h File Reference

```
#include "lib.h"
#include "DataDevice.h"
#include "KernelDevice.h"
```

Classes

- class DeviceBuilder

  This class lets a high-level programmer add easily more devices to the main program. You can use the implemented public functions wherever you need. For adding a new device you can follow this steps:

5.13.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

12/11/2022

Definition in file DeviceBuilder.h.

5.14 DeviceBuilder.h

Go to the documentation of this file.

```
00001 // ------------------- DeviceBuilder - JVH Systems -------------------------------
00008 #ifndef DEVICEBUILDER_H
00009 #define DEVICEBUILDER_H
00010 #include "lib.h"
00011 #include "DataDevice.h"
00012 #include "KernelDevice.h"
00013
00034 class DeviceBuilder {
00035
00036 // ------------------------------------------------
00037 //                    PUBLIC                      |
00038 // ------------------------------------------------
00039 public:
00040
00041     // -----------------  DYNAMIC  ----------------------
00042
00047     DeviceBuilder(const std::string& name = "STD_DEVICE");
00048
00053     void setSkill(void(*skill)(), const std::string& skillName);
00054
00059     void setLimit(int* newLimit);
00060
00062     void setAsDataDevice();
00063
00065     void setAsKernelDevice();
00066
00071     void setDataGenerator(int(*dataGenerator)());
00072
00077     void setSecurityGenerator(bool(*securityGenerator)());
00078
00079     // -----------------  STATIC  ----------------------
00080
00083     static void buildAllDevices();
00084
00089     static SimpleDisplay* buildDisplay();
00090
00093     static void clearAll();
00094
00095 // ------------------------------------------------
00096 //                   PRIVATE                      |
00097 // ------------------------------------------------
00098 private:
00099
00100     // -----------------  DYNAMIC  ----------------------
00101
00102     // [Methods]
00103
00108     void buildDKDevice();
00109
00110     // [Variables]
00111
00113     std::string name = "STD Device";
00115     std::vector<void(*)()> skillVector;
00117     std::vector<std::string> skillNamesVector;
00119     int (*dataGenerator)();
00121     bool (*securityGenerator)();
00123     int id;
00125     int* limit = nullptr;
00127     bool isDataDevice;
00128
00129     // -----------------  STATIC  ----------------------
00130
00131     // [Variables]
00132
00134     static std::vector<DeviceBuilder*> allBuilders;
00136     static std::vector<Device> allDevices;
00138     static std::vector<DataDevice> allDataDevices;
00140     static std::vector<KernelDevice> allKernelDevices;
00142     static int deviceCounter;
00143
00144
00145 };
00146
00147
00148 #endif //DEVICEBUILDER_H
```

5.15 DeviceComposite.h File Reference

```
#include "AI.h"
```

Classes

- class DeviceComposite

  This class is the responsible of managing multiple orders to devices simultaneously. It will offer a easy interface to compose devices and send orders to them.

5.15.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

26/12/2022

Definition in file DeviceComposite.h.

5.16 DeviceComposite.h

Go to the documentation of this file.

```
00001
00007 #ifndef JVH_SYSTEMS_DEVICECOMPOSITE_H
00008 #define JVH_SYSTEMS_DEVICECOMPOSITE_H
00009 #include "AI.h"
00010
00015 class DeviceComposite {
00016
00017 // ------------------------------------------------
00018 //                    PUBLIC                      |
00019 // ------------------------------------------------
00020 public:
00021
00022     // -----------------  DYNAMIC  ----------------------
00023
00025     DeviceComposite();
00026
00030     void add(const std::string &name);
00031
00033     void pop();
00034
00038      void requestSkill(const std::string& skill);
00039
00041      void clear();
00042
00045      std::string toString();
00046
00047 // ------------------------------------------------
00048 //                   PRIVATE                      |
00049 // ------------------------------------------------
00050 private:
00051
00052     // -----------------  DYNAMIC  ----------------------
00053
00054     // [Variables]
00056     std::vector<IODi> allIODis;
00057
00058     // -----------------  STATIC  ----------------------
00059
00060     // [Variables]
00062     static const std::string CALL_KEYWORD;
00063
00064 };
00065
00066
00067 #endif //JVH_SYSTEMS_DEVICECOMPOSITE_H
```

5.17 BaseException.h File Reference

```
#include <string>
#include <exception>
```

Classes

- class BaseException

  This class establishes a basic format for JVH exceptions.

5.17.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file BaseException.h.

5.18 BaseException.h

Go to the documentation of this file.

```
00001 // ------------------- BaseException - JVH Systems -------------------------------
00008 #ifndef BASE_EXCEPTION_H
00009 #define BASE_EXCEPTION_H
00010 #include <string>
00011 #include <exception>
00012
00016 class BaseException :  public std::exception {
00017
00018 // ------------------------------------------------
00019 //                    PUBLIC                      |
00020 // ------------------------------------------------
00021 public:
00022
00024     BaseException();
00025
00028     const char* what();
00029
00030 // ------------------------------------------------
00031 //                  PROTECTED                     |
00032 // ------------------------------------------------
00033 protected:
00034
00036     std::string tag;
00038     std::string exception;
00039
00040
00041 };
00042
00043 #endif //BASE_EXCEPTION_H
```

5.19 InsufficientPermissionsException.h File Reference

```
#include "BaseException.h"
```

Classes

- class InsufficientPermissionsException

  This class lets the program throw an exception when the permissions of the user are not valid.

5.20 InsufficientPermissionsException.h

Go to the documentation of this file.

```
00001 // ------------------ InsufficientPermissionException - JVH Systems -------------------------------
00009 #ifndef INSUFFICIENTPERMISSIONSEXCEPTION_H
00010 #define INSUFFICIENTPERMISSIONSEXCEPTION_H
00011 #include "BaseException.h"
00016 class InsufficientPermissionsException :    BaseException {
00017
00018 // -----------------------------------------------
00019 //                    PUBLIC                     |
00020 // -----------------------------------------------
00021 public:
00022
00025     explicit InsufficientPermissionsException(const std::string &tag);
00026
00027 // -----------------------------------------------
00028 //                   PRIVATE                     |
00029 // -----------------------------------------------
00030 private:
00031
00033     static std::string ERROR_MESSAGE;
00034
00035 };
00036
00037
00038 #endif //INSUFFICIENTPERMISSIONSEXCEPTION_H
```

5.21 InvalidDeviceException.h File Reference

```
#include "BaseException.h"
```

Classes

- class InvalidDeviceException

  This class lets the program throw an exception when the selected device is not valid.

5.21.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file InvalidDeviceException.h.

5.22 InvalidDeviceException.h

Go to the documentation of this file.

```
00001 // ------------------- InvalidDeviceException - JVH Systems -----------------------------
00008 #ifndef INVALIDDEVICE_H
00009 #define INVALIDDEVICE_H
00010 #include "BaseException.h"
00011
00016 class InvalidDeviceException :  public BaseException{
00017
00018 // -------------------------------------------------
00019 //                     PUBLIC                      |
00020 // -------------------------------------------------
00021 public:
00022
00025     explicit InvalidDeviceException(const std::string &tag);
00026
00027 // -------------------------------------------------
00028 //                    PRIVATE                      |
00029 // -------------------------------------------------
00030 private:
00031
00033     static std::string ERROR_MESSAGE;
00034
00035 };
00036
00037
00038 #endif //INVALIDDEVICE_H
```

5.23 InvalidFileException.h File Reference

```
#include "BaseException.h"
```

Classes

- class InvalidFileException

  This class lets the program throw an exception when the File to be read is not valid.

5.23.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file InvalidFileException.h.

5.24 InvalidFileException.h

Go to the documentation of this file.

```
00001 // ------------------- InvalidFileException - JVH Systems -----------------------------
00008 #ifndef INVALIDFILEEXCEPTION_H
00009 #define INVALIDFILEEXCEPTION_H
00010 #include "BaseException.h"
00011
00016 class InvalidFileException :  BaseException {
00017
00018 // -------------------------------------------------
00019 //                     PUBLIC                      |
00020 // -------------------------------------------------
00021 public:
00022
00025     explicit InvalidFileException(const std::string &tag);
00026
00027 // -------------------------------------------------
00028 //                    PRIVATE                      |
00029 // -------------------------------------------------
00030 private:
00031
00033     static std::string ERROR_MESSAGE;
00034
00035 };
00036
00037
00038 #endif //INVALIDFILEEXCEPTION_H
```

5.25 InvalidSkillException.h File Reference

```
#include "BaseException.h"
```

Classes

- class InvalidSkillException

  This class lets the program throw an exception when the selected skill is not valid.

5.25.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file InvalidSkillException.h.

5.26 InvalidSkillException.h

Go to the documentation of this file.

```
00001 // ------------------- InvalidSkillException - JVH Systems -----------------------------
00008 #ifndef INVALIDSKILLEXCEPTION_H
00009 #define INVALIDSKILLEXCEPTION_H
00010 #include "BaseException.h"
00011
00016 class InvalidSkillException :  BaseException {
00017
00018 // -------------------------------------------------
00019 //                     PUBLIC                      |
00020 // -------------------------------------------------
00021 public:
00022
00025     explicit InvalidSkillException(const std::string &tag);
00026
00027 // -------------------------------------------------
00028 //                    PRIVATE                      |
00029 // -------------------------------------------------
00030 private:
00031
00033     static std::string ERROR_MESSAGE;
00034
00035 };
00036
00037
00038 #endif //INVALIDSKILLEXCEPTION_H
```

5.27 InvalidUserException.h File Reference

```
#include "BaseException.h"
```

Classes

- class InvalidUserException

  This class lets the program throw an exception when the selected user is not valid.

5.27.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file InvalidUserException.h.

5.28 InvalidUserException.h

Go to the documentation of this file.

```
00001 // ------------------- InvalidUserException - JVH Systems -----------------------------
00008 #ifndef INVALIDUSEREXCEPTION_H
00009 #define INVALIDUSEREXCEPTION_H
00010 #include "BaseException.h"
00011
00016 class InvalidUserException :  BaseException{
00017
00018 // -------------------------------------------------
00019 //                     PUBLIC                      |
00020 // -------------------------------------------------
00021 public:
00022
00025     explicit InvalidUserException(const std::string &tag);
00026
00027 // -------------------------------------------------
00028 //                    PRIVATE                      |
00029 // -------------------------------------------------
00030 private:
00031
00033     static std::string ERROR_MESSAGE;
00034
00035 };
00036
00037
00038 #endif //INVALIDUSEREXCEPTION_H
```

5.29 UserNotFoundException.h File Reference

```
#include "BaseException.h"
```

Classes

- class UserNotFoundException

  This class lets the program throw an exception when the selected user has not been found.

5.29.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

20/12/2022

Definition in file UserNotFoundException.h.

5.30 UserNotFoundException.h

Go to the documentation of this file.

```
00001 // ------------------- UserNotFoundException - JVH Systems -----------------------------
00008 #ifndef USERNOTFOUND_H
00009 #define USERNOTFOUND_H
00010 #include "BaseException.h"
00011
00016 class UserNotFoundException :  public BaseException {
00017
00018 // ------------------------------------------------
00019 //                    PUBLIC                      |
00020 // ------------------------------------------------
00021 public:
00022
00025     explicit UserNotFoundException(const std::string &tag);
00026
00027 // ------------------------------------------------
00028 //                   PRIVATE                      |
00029 // ------------------------------------------------
00030 private:
00031
00033     static std::string ERROR_MESSAGE;
00034
00035 };
00036
00037
00038 #endif //USERNOTFOUND_H
```

5.31 exceptionsLib.h File Reference

```
#include "except/BaseException.h"
#include "except/UserNotFoundException.h"
#include "except/InvalidUserException.h"
#include "except/InvalidFileException.h"
#include "except/InvalidDeviceException.h"
#include "except/InvalidSkillException.h"
#include "except/InsufficientPermissionsException.h"
```

5.31.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

21/11/2022

Definition in file exceptionsLib.h.

5.32 exceptionsLib.h

Go to the documentation of this file.

```
00001
00006 #ifndef EXCEPTIONSLIB_H
00007 #define EXCEPTIONSLIB_H
00008
00009 #include "except/BaseException.h"
00010 #include "except/UserNotFoundException.h"
00011 #include "except/InvalidUserException.h"
00012 #include "except/InvalidFileException.h"
00013 #include "except/InvalidDeviceException.h"
00014 #include "except/InvalidSkillException.h"
00015 #include "except/InsufficientPermissionsException.h"
00016 #endif //EXCEPTIONSLIB_H
```

5.33 IODi.h File Reference

```
#include "DataDevice.h"
```

Classes

- class IODi

  This class will be the responsible for interpreting the status of the devices and sending proper information, and executing the requested skills for the main class when it needs to interact with any device. Thus, it will be implemented by the main class and can't exist without Device class.

5.33.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

12/11/2022

Definition in file IODi.h.

5.34 IODi.h

Go to the documentation of this file.

```
00001 // ------------------- Input/Output Device Interface - JVH Systems -----------------------------
00008 #ifndef IODI_H
00009 #define IODI_H
00010 #include "DataDevice.h"
00011
00019 class IODi {
00020
00021 // -----------------------------------------------
00022 //                    PUBLIC                     |
00023 // -----------------------------------------------
00024 public:
00025
00026     enum{
00027         DEFAULT_ID = 33
00028     };
00029
00030     // -----------------  DYNAMIC  ----------------------
00031
00034     IODi();
00035
00038     std::string* requestDeviceNames();
00039
00043     std::string* requestDeviceSkills();
00044
00047     std::string requestCurrentDeviceName();
00048
00051     int requestNumberOfDataDevices();
00052
00055     int requestNumberOfSkills();
00056
00059     bool* requestAllStatus();
00060
00063     void requestSkill(const std::string& skill);
00064
00067     void requestSkill(const int skill);
00068
```

```
00072     void setCurrentDevice(const std::string& name);
00073
00077     void setCurrentDevice(const int pos);
00078
00079 // ---------------------------------------------
00080 //                  PRIVATE                    |
00081 // ---------------------------------------------
00082 private:
00083
00084     // -----------------  DYNAMIC  ----------------------
00085
00086     // [Variables]
00087
00089     int id;
00091     std::string currentDeviceName;
00093     int currentDevicePos;
00094
00095     // -----------------  STATIC  ----------------------
00096
00097     // [Functions]
00098
00100     static std::string TAG;
00102     static int maxID;
00103
00104 };
00105
00106
00107 #endif// IODI_H
```

5.35 KernelDevice.h File Reference

```
#include "Device.h"
```

Classes

- class KernelDevice

  This class will contain specific properties of only kernel interacting devices and a set of functionalities for managing and checking their data. It also inherits properties from Device class.

5.35.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

12/11/2022

Definition in file KernelDevice.h.

5.36 KernelDevice.h

Go to the documentation of this file.

```
00001 // ------------------- KernelDevice - JVH Systems -------------------------------
00007 #ifndef KERNELDEVICE_H
00008 #define KERNELDEVICE_H
00009 #include "Device.h"
00010
00017 class KernelDevice :  public Device {
00018
00019     // This class will build all DataDevices
00020     friend class DeviceBuilder;
00021     friend class AI;
00022
00023 // ------------------------------------------------
00024 //                    PUBLIC                     |
00025 // ------------------------------------------------
00026 public:
00027
00028     // -----------------  DYNAMIC  ----------------------
00029
00032     KernelDevice();
00033
00037     KernelDevice(const std::string& name, const int id);
00038
00039 // ------------------------------------------------
00040 //                   PRIVATE                     |
00041 // ------------------------------------------------
00042 private:
00043
00044     // -----------------  DYNAMIC  ----------------------
00045
00046     // [Variables]
00047
00049     bool(*securityGenerator)();
00051     bool securityHasBeenBroken;
00052
00053     // -----------------  STATIC  ----------------------
00054
00055     // [Variables]
00056
00058     static KernelDevice* allKernelDevices;
00060     static int numKerns;
00061
00062     // [Methods]
00063
00067     static void setAllKernelDevices(KernelDevice* allDevices);
00068
00069 };
00070
00071
00072 #endif //KERNELDEVICE_H
```

5.37 lib.h File Reference

```
#include "SimpleDisplay.h"
#include "exceptionsLib.h"
#include <vector>
#include <cstdio>
#include <ctime>
#include <algorithm>
#include <fstream>
#include <limits>
#include <chrono>
#include <set>
#include <csignal>
#include <stack>
#include "../ext/kb.h"
```

5.37.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

21/11/2022

Definition in file lib.h.

5.38 lib.h

Go to the documentation of this file.

```
00001
00006 #ifndef LIB_H
00007 #define LIB_H
00008
00009 #include "SimpleDisplay.h"
00010 #include "exceptionsLib.h"
00011 #include <vector>
00012 #include <cstdio>
00013 #include <ctime>
00014 #include <algorithm>
00015 #include <fstream>
00016 #include <limits>
00017 #include <chrono>
00018 #include <set>
00019 #include <csignal>
00020 #include <stack>
00021
00022 #endif //LIB_H
00023
00024 #ifdef _WIN32
00025 #include <windows.h>
00026 #include <conio.h>
00027 #else
00028 #include "../ext/kb.h"
00029 #endif
00030
```

5.39 Login.h File Reference

```
#include "Admin.h"
```

Classes

- class Login

  This class is the responsible for superimposing a login interface to make the user log in as user. This will be a graphical interface belonging to the User class.

5.39.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

04/11/2022

Definition in file Login.h.

5.40 Login.h

Go to the documentation of this file.

```
00001 // ------------------ Login - JVH Systems -------------------------------
00008 #ifndef LOGIN_H
00009 #define LOGIN_H
00010 #include "Admin.h"
00011
00020 class Login {
00021
00022 // ------------------------------------------------
00023 //                     PUBLIC                     |
00024 // ------------------------------------------------
00025 public:
00026
00027     // -----------------  DYNAMIC  ----------------------
00028
00031     Login();
00032
00036     User startLogin();
00037
00038 // ------------------------------------------------
00039 //                   PRIVATE                      |
00040 // ------------------------------------------------
00041 private:
00042
00043     // -----------------  STATIC  ----------------------
00044
00046     static int SLEEP_TIME;
00048     static int EXIT_REQUEST;
00049
00050     // -----------------  DYNAMIC  ----------------------
00051
00053     int user;
00055     int pass;
00056
00057 };
00058
00059
00060 #endif //LOGIN_H
```

5.41 SimpleDisplay.h File Reference

```
#include <string>
#include <iostream>
#include <vector>
#include <ctime>
#include <unistd.h>
```

Classes

- class SimpleDisplay

  This library will contain a set of methods implemented by the main for graphicating in screen the main interface. This class can be changed by any other library whose functionalities are named in the same way.

5.41.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

04/11/2022

Definition in file SimpleDisplay.h.

5.42 SimpleDisplay.h

Go to the documentation of this file.

```
00001 // ------------------ SimpleDisplay - JVH Systems -------------------------------
00008 #ifndef SIMPLE_DISPLAY_H
00009 #define SIMPLE_DISPLAY_H
00010 #include <string>
00011 #include <iostream>
00012 #include <vector>
00013 #include <ctime>
00014 #include <unistd.h>
00015
00021 class SimpleDisplay {
00022
00023 // -------------------------------------------------
00024 //                    PUBLIC                      |
00025 // -------------------------------------------------
00026 public:
00027
00028     // -----------------  DYNAMIC  ----------------------
00029
00030     // [Methods]
00031
00034     SimpleDisplay();
00035
00036     // [Setters]
00037
00040     void setSecurityConnected(const bool securityConnected);
00041
00044     void setSecurityStatus(const bool securityStatus);
00045
00048     void setKernelDevices(const int kernelDevices);
00049
00052     void setDataDevices(const int dataDevices);
00053
00056     void setUser(const int user);
00057
00060     void setGroup(const std::string &group);
00061
00064     void operator«(const SimpleDisplay& display);
00065
00071     static void printLoginInterface(const int user, const int pass, const int INVALID_USER=-1, const
      int INVALID_PASS=-1);
00072
00074     static void printChecking();
00075
00077     static void printPresentation();
00078
00079     // [MAIN]
00080
00085     void printMainInterface(std::string* deviceNames, bool* deviceStatus);
```

```
00086
00087     // -----------------  STATIC  ----------------------
00088
00089     // [DEVi++]
00090
00093     static void deviCout(const std::string& text);
00094
00097     static void printDeviHeader(const std::string& deviceName);
00098
00100     static void cleanDevi(const std::string& deviceName);
00101
00103     static void printAboutUs();
00104
00106     static void printDeviHelp();
00107
00111     static void deviPrintData(const std::vector<int>& data ,const std::vector<std::string>& time);
00112
00117     static void printSkillManual(const std::string* skillnames, const int count);
00118
00119     // [STD]
00120
00122     static void jump();
00123
00125     static void cout(const std::string& text);
00126
00127     // [AI]
00128
00130     static void printAIInterface();
00131
00133     static void printRequestPassword();
00134
00136     static void execBuildName();
00137
00139     static void execBuildType();
00140
00143     static void execBuildGenerator(const bool isDataDevice);
00144
00146     static void execBuildFinish();
00147
00148     // [Security Rescue]
00149
00152     static void checkAbortStatus();
00153
00154     // [Variables]
00155
00158     static std::string COLOR;
00159
00162     static std::string GREY_COLOR;
00163
00166     static std::string RESET_COLOR;
00167
00170     static std::string WARNING_COLOR;
00171
00173     static SimpleDisplay* currentDisplay;
00174
00176     static bool security_abort;
00177
00178 // ------------------------------------------------
00179 //                   PRIVATE                      |
00180 // ------------------------------------------------
00181 private:
00182
00183     // -----------------  STATIC  ----------------------
00184
00187     static const int MAXIMUM_OPTIONS;
00188
00189     // -----------------  DYNAMIC  ----------------------
00190
00191     // [Variables]
00192
00194     bool securityConnected;
00195
00197     bool securityStatus;
00198
00200     int kernelDevices;
00201
00203     int dataDevices;
00204
00206     int user;
00207
00209     std::string group;
00210
00211 };
00212
00213
00214 #endif //SIMPLE_DISPLAY_H
```

5.43 User.h File Reference

```
#include "../include/lib.h"
```

Classes

- class User

  This class will be the responsible for storing and managing the user's data. Also is able to create an abstract user object to ease data access.

5.43.1 Detailed Description

Author

Sebastian Mayorquin (Software Design)

Date

04/11/2022

Definition in file User.h.

5.44 User.h

Go to the documentation of this file.

```
00001 // ------------------- User - JVH Systems -------------------------------
00007 #ifndef USER_H
00008 #define USER_H
00009 #include "../include/lib.h"
00010
00016 class User {
00017
00018 // -----------------------------------------------
00019 //                    PUBLIC                     |
00020 // -----------------------------------------------
00021 public:
00022
00023     // [Enum]
00024     enum {
00025         INVALID_USER = -2,
00026         INVALID_PASS = -2,
00027         USER_COLUMN = 0,
00028         PASS_COLUMN = 1,
00029         ADMIN_COLUMN = 2,
00030         ADMIN_IDENTIFIER = 1,
00031         TOTAL_COLUMNS = 3,
00032         DEVELOPER_USER = -1
00033     };
00034
00035     // -----------------  DYNAMIC  ----------------------
00036
00037     // [Methods]
00038
00040     User();
00041
00046     User(const int id);
00047
00048     // [Variables]
00049
00051     int getUser();
00052
00054     bool getAdminStatus();
```

```
00055
00056     // -----------------  STATIC  ----------------------
00057
00058     // [Functions]
00059
00065     static bool check(const int id,const int pass);
00066
00074     static void updateUserData();
00075
00079     static void saveUserData();
00080
00085     static void addUser(const int id, const int pass, const bool isAdmin);
00086
00089     static void rmUser(const int id);
00090
00092     static void printUsers();
00093
00094     // [Variables]
00095
00096     static std::string EXECUTABLE_DIR;
00097
00098 // ------------------------------------------------
00099 //                  PROTECTED                     |
00100 // ------------------------------------------------
00101 protected:
00102
00103     // -----------------  DYNAMIC  ----------------------
00104
00105     // [Variables]
00106
00109     int id;
00111     bool isAdmin;
00113     int pass;
00114
00115     // -----------------  STATIC  ----------------------
00116
00117     // [Functions]
00118
00126     static int* find(int id);
00127
00128     // [DEPRECATED] static std::vector<int> find(int id);
00129
00130     // [Variables]
00131
00133     static char SEPARATOR;
00135     static const std::string TAG;
00137     static std::set<int*> allUsers;
00139     static const std::string DATAFILE;
00141     static const std::string USERFILE;
00142 };
00143
00144
00145 #endif //USER_H
```

5.45 Admin.cpp File Reference

```
#include "../include/Admin.h"
```

5.46 Admin.cpp

Go to the documentation of this file.

```
00001 #include "../include/Admin.h"
00002
00003 // ------------------------------------------------------
00004 //                  PUBLIC FUNCTIONALITIES               |
00005 // ------------------------------------------------------
00006
00007 // --- [Constructor] Admin() ---
00008 Admin::Admin(int id) :  User(id){
00009
00010     this->isAdmin = true;
00011
00012 }
00013
```

```
00014 // --- [Static] adminStatus() ---
00015 bool Admin::adminStatus(const int id) {
00016
00017     try{
00018         int* userData = find(id);
00019         return (userData[ADMIN_COLUMN] == ADMIN_IDENTIFIER);
00020
00021     // User not found
00022     }catch(UserNotFoundException &e){
00023         return false;
00024     }
00025 }
```

5.47 AI.cpp File Reference

```
#include "../include/AI.h"
```

5.47.1 Detailed Description

Author

Sebastian Mayorquin

Date

16/11/2022 @grade Software Robotics (Software Design)

Definition in file AI.cpp.

5.48 AI.cpp

Go to the documentation of this file.

```
00001 // ------------------- Automatic Interface - JVH Systems -------------------------------
00010 #include "../include/AI.h"
00011
00012 // ----- Declarations ------
00013 int AI::MAX_DATA = 30;
00014 int AI::SLEEPTIME = 1;
00015 std::string AI::TAG = "AI";
00016
00017 bool AI::abort = false;
00018
00019 // ---------------------------------------------------------
00020 //                  PUBLIC FUNCTIONALITIES               |
00021 // ---------------------------------------------------------
00022
00023
00024 // --- [Empty Constructor] AI() ---
00025 AI::AI() {
00026     this->time_counter = 0;
00027     signal(SIGINT, signalHandler);
00028
00029 }
00030
00031 // --- [Method] startCollectingData ---
00032 void AI::startCollectingData() {
00033
00034     // ----- Intializing ------
00035     std::cout « "[AI] Collecting Data" « std::endl;
00036     int dataCounter = 0;
00037
00038     // ----- Filling every device ------
00039     int numDevices = DataDevice::getNumSensors();
00040     for (int device = 0; device < numDevices; device++)
```

```
00041         for (int data = 0; data < MAX_DATA; data++) {
00042             DataDevice::allDataDevices[device].appendNewData();
00043             dataCounter++;
00044         }
00045 
00046     // ----- Sending Output ------
00047     std::cout « "[AI] Collected " « dataCounter « " measures" « std::endl;
00048 }
00049 
00050 
00051 // --- [Method] refreshData ---
00052 void AI::refreshData() {
00053 
00054     // ----- Declarations ------
00055     int numDevices = DataDevice::getNumSensors();
00056 
00057     // ----- Refreshing every data device one time ------
00058     for (int i = 0; i < DataDevice::numSensors; i++) {
00059 
00060         DataDevice* target = &DataDevice::allDataDevices[i];
00061 
00062         // Check if it's active
00063         if (!~(*target)) continue;
00064 
00065         // Check if it is full
00066         if (target->collectedData.size() >= MAX_DATA) {
00067 
00068             // Remove oldest measurement
00069             target->collectedData.erase(target->collectedData.begin());
00070 
00071             // Remove oldest time
00072             target->collectedTimes.erase(target->collectedTimes.begin());
00073 
00074         }
00075         // Append new measurement and time
00076         target->appendNewData();
00077     }
00078 
00079     // ----- Refreshing every kernel device one time -----
00080     for (int i = 0; i < KernelDevice::numKerns; i++) {
00081 
00082         KernelDevice* k = &KernelDevice::allKernelDevices[i];
00083         // Check if it's active
00084         if (!~(*k)) continue;
00085 
00086         if (!(*k).securityGenerator() || k->securityHasBeenBroken) {
00087             k->securityHasBeenBroken = true;
00088             SimpleDisplay::currentDisplay->setSecurityStatus(false);
00089         }
00090     }
00091 
00092 }
00093 
00094 // --- [Method] throwDemon ---
00095 void AI::throwDemon() {
00096 
00097 #ifdef _WIN32
00098 
00099     while (true) {
00100         try {
00101 
00102             if (kbhit()) break;
00103             if (abort){
00104                 SimpleDisplay::cout("[WARNING] You should not exit directly from main menu.  Use -1
      instead.\n");
00105                 exit(0);
00106             }
00107             refreshData();
00108             sleep(SLEEPTIME);
00109 
00110         } catch (std::exception &e) {
00111             sleep(SLEEPTIME);
00112             continue;
00113         }
00114     }
00115 #else
00116     while (true){
00117     try{
00118 
00119         if (kb::kbhit()) break;
00120         if (abort){
00121             SimpleDisplay::cout("[WARNING] You should not exit directly from main menu.  Use -1
      instead.\n");
00122             exit(0);
00123         }
00124         refreshData();
00125         sleep(SLEEPTIME);
```

```
00126
00127     }catch (std::exception e){
00128         sleep(SLEEPTIME);
00129     }
00130 }
00131 #endif
00132 }
00133
00134 void AI::requestOption(const std::string &opt, bool isAdmin) {
00135     try {
00136         int num = atoi(opt.c_str());
00137
00138         switch (num) {
00139
00140             case TURNDEVICES:
00141                 turnDevices();
00142                 break;
00143
00144             case TURNSECURITY:
00145                 turnSecurity();
00146                 break;
00147
00148             case CHECKSECURITY:
00149                 checkSecurity();
00150                 break;
00151
00152             case MICROPHONE:
00153                 microphone();
00154                 break;
00155
00156             case FORCEERROR:  {
00157                 try {
00158                     AI::ensureAdminAccess(isAdmin);
00159                 }catch (InsufficientPermissionsException &e){
00160                     break;
00161                 }
00162
00163                 forceError();
00164                 break;
00165             }
00166
00167             case BUILDDEVICE:{
00168                 try {
00169                     AI::ensureAdminAccess(isAdmin);
00170                 }catch (InsufficientPermissionsException &e){
00171                     break;
00172                 }
00173
00174                 buildDevice();
00175                 break;
00176             }
00177
00178             case ADDUSER:{
00179                 try {
00180                     AI::ensureAdminAccess(isAdmin);
00181                 }catch(InsufficientPermissionsException &e){
00182                     break;
00183                 }
00184                 addUser();
00185                 break;
00186             }
00187
00188             case REMOVEUSER:{
00189                 try{
00190                     AI::ensureAdminAccess(isAdmin);
00191                 }catch(InsufficientPermissionsException &e){
00192                     break;
00193                 }
00194                 removeUser();
00195                 break;
00196             }
00197
00198             case RETURNSECURITY:  {
00199                 try {
00200                     AI::ensureAdminAccess(isAdmin);
00201                 }catch (InsufficientPermissionsException &e){
00202                     break;
00203                 }
00204                 recoverSecurity();
00205                 break;
00206             }
00207
00208             case FORCEBADALLOC:{
00209                 try {
00210                     AI::ensureAdminAccess(isAdmin);
00211                 }catch (InsufficientPermissionsException &e){
00212                     break;
```

```
00213                 }
00214                 forceBadAlloc();
00215                 break;
00216             }
00217
00218             case SHOWUSERS:  {
00219                 try {
00220                     AI::ensureAdminAccess(isAdmin);
00221                 } catch (InsufficientPermissionsException &e){
00222                     break;
00223                 }
00224                 showAllUsers();
00225                 break;
00226             }
00227             case MORE:{
00228                 SimpleDisplay::printAboutUs();
00229                 break;
00230             }
00231
00232             case EXIT:
00233                 break;
00234         }
00235
00236
00237     }catch (std::exception &e){
00238         SimpleDisplay::cout("Option NOT valid");
00239         sleep(1);
00240     }
00241
00242 }
00243
00244 void AI::ensureAdminAccess(bool isAdmin) {
00245
00246     // ----- Checking Admin Status -----
00247     if (!isAdmin) {
00248         SimpleDisplay::cout("THIS ACTION REQUIRES ADMINISTRATOR ACCESS");
00249         sleep(SLEEPTIME*2);
00250         throw InsufficientPermissionsException(TAG);
00251     }
00252
00253     // ----- Entering a console and requesting password -----
00254     try {
00255         std::string message;
00256         SimpleDisplay::printRequestPassword();
00257         SimpleDisplay::cout("Keyboard:  ");
00258         std::getline(std::cin,message);
00259
00260         // Checking input
00261         if (message != "0")
00262             throw InsufficientPermissionsException(TAG);
00263
00264         // Invalid Input
00265         }catch(std::exception e){
00266
00267             SimpleDisplay::cout("INVALID PASSWORD");
00268             sleep(SLEEPTIME);
00269             throw InsufficientPermissionsException(TAG);
00270         }
00271
00272 }
00273
00274 void AI::turnDevices() {
00275
00276     // ----- Instantiating IODi for easy device control -----
00277     IODi iodi;
00278
00279     // ----- Turning all Devices ON/OFF -----
00280     for (int i = 0; i < DataDevice::numSensors; i++) {
00281         iodi.setCurrentDevice(i);
00282
00283         if (~DataDevice::allDataDevices[i])
00284             iodi.requestSkill("turnoff");
00285         else
00286             iodi.requestSkill("turnon");
00287
00288     }
00289
00290     // ----- Exiting...  -----
00291     std::string none;
00292     SimpleDisplay::cout("Press ENTER to continue");
00293     getline(std::cin,none);
00294 }
00295
00296 void AI::turnSecurity() {
00297
00298     for (int i = 0; i < KernelDevice::numKerns; i++) {
00299
```

```
00300         // Accessing Device
00301         KernelDevice *k = &KernelDevice::allKernelDevices[i];
00302 
00303         // Checking Status
00304         k->isActive = !~(*k);
00305 
00306         // Turning ON/OFF
00307         if (~(*k)) {
00308             SimpleDisplay::cout(k->getName() + " turned ON\n");
00309             SimpleDisplay::currentDisplay->setSecurityConnected(true);
00310         }
00311         else {
00312             SimpleDisplay::cout(k->getName() + " turned OFF\n");
00313             SimpleDisplay::currentDisplay->setSecurityConnected(false);
00314         }
00315     }
00316 
00317     // ----- Exiting -----
00318     std::string none;
00319     SimpleDisplay::cout("Press ENTER to continue");
00320     getline(std::cin,none);
00321 }
00322 
00323 void AI::checkSecurity(){
00324 
00325     for (int i = 0; i < KernelDevice::numKerns; i++) {
00326 
00327         // Acessing Device
00328         KernelDevice* k = &KernelDevice::allKernelDevices[i];
00329 
00330         // Displaying chart
00331         SimpleDisplay::cout("[AI] [" + k->getName() + "] ");
00332 
00333         if (~*(k))
00334             SimpleDisplay::cout("[RUNNING]: ");
00335         else
00336             SimpleDisplay::cout("[DISCONNECTED]: ");
00337 
00338         if (!k->securityHasBeenBroken)
00339             SimpleDisplay::cout("Security status correct");
00340         else
00341             SimpleDisplay::cout("[WARNING] Security FAILURE! Please check the logs.");
00342 
00343         SimpleDisplay::cout("\n");
00344 
00345     }
00346     // ----- Exiting...  -----
00347     std::string none;
00348     SimpleDisplay::cout("Press ENTER to continue");
00349     getline(std::cin,none);
00350 
00351 }
00352 
00353 void AI::microphone(){
00354 
00355 
00356     try {
00357         std::string message;
00358 
00359         SimpleDisplay::cout("\nEnter a message:  ");
00360         getline(std::cin, message);
00361 
00362         SimpleDisplay::cout("[AI] SENDING MESSAGE:\n");
00363         SimpleDisplay::cout("|| -> " + message);
00364 
00365         sleep(SLEEPTIME);
00366 
00367     }catch (std::exception &e){
00368         SimpleDisplay::cout("Input not valid");
00369     }
00370 
00371 }
00372 
00373 void AI::forceError(){
00374 
00375     SimpleDisplay::printAIInterface();
00376     SimpleDisplay::cout("Forcing FAILURE in all security devices");
00377 
00378     for (int i = 0; i < KernelDevice::numKerns; i++) {
00379         KernelDevice::allKernelDevices[i].securityHasBeenBroken = true;
00380     }
00381     SimpleDisplay::currentDisplay->setSecurityStatus(false);
00382 
00383     sleep(SLEEPTIME*2);
00384 }
00385 
00386 void AI::buildDevice(){
```

```
00387
00388     SimpleDisplay::execBuildName();
00389     std::string name;
00390     std::getline(std::cin,name);
00391     if (name == "EXIT") {
00392
00393         SimpleDisplay::cout("\n[AI] Exit Request");
00394         sleep(SLEEPTIME);
00395         return;
00396     }
00397
00398     SimpleDisplay::execBuildType();
00399     std::string type;
00400     std::getline(std::cin,type);
00401
00402     if (type != "data" && type != "kernel"){
00403         SimpleDisplay::cout("\n[AI] Exit Request");
00404         sleep(SLEEPTIME);
00405         return;
00406     }
00407
00408     std::string generator;
00409     if (type == "data"){
00410         SimpleDisplay::execBuildGenerator(true);
00411         std::getline(std::cin,generator);
00412     }else{
00413         SimpleDisplay::execBuildGenerator(false);
00414         std::getline(std::cin,generator);
00415     }
00416
00417     SimpleDisplay::execBuildFinish();
00418     sleep(SLEEPTIME*2);
00419     SimpleDisplay::cout("[AI] Testing Building...\n");
00420     sleep(SLEEPTIME*2);
00421     SimpleDisplay::cout("[AI] Testing Device...\n");
00422     sleep(SLEEPTIME*2);
00423
00424     DeviceBuilder::clearAll();
00425     DeviceBuilder* newDevice = new DeviceBuilder(name);
00426     DataDevice::numSensors = 0;
00427     KernelDevice::numKerns = 0;
00428     Device::numDevices = 0;
00429     if (type == "data"){
00430         (*newDevice).setAsDataDevice();
00431         (*newDevice).setDataGenerator(generateRandomData);
00432         DeviceBuilder::buildAllDevices();
00433     }else{
00434         (*newDevice).setAsKernelDevice();
00435         (*newDevice).setSecurityGenerator(exampleTest);
00436         DeviceBuilder::buildAllDevices();
00437
00438     }
00439     SimpleDisplay::cout("[AI] Building SUCCESS\n");
00440     std::cout « "\nPress ENTER to start";
00441     std::cin.get();
00442     std::cin.clear();
00443
00444 }
00445
00446 void AI::recoverSecurity() {
00447
00448     SimpleDisplay::printAIInterface();
00449     SimpleDisplay::cout("Recovering all Devices to Secure Status...");
00450
00451     for (int i = 0; i < KernelDevice::numKerns; i++) {
00452         KernelDevice::allKernelDevices[i].securityHasBeenBroken = false;
00453     }
00454     SimpleDisplay::currentDisplay->setSecurityStatus(true);
00455
00456     sleep(SLEEPTIME*2);
00457 }
00458
00459 void AI::addUser() {
00460
00461     int user, password;
00462     bool isAdmin;
00463     std::string entry;
00464
00465     SimpleDisplay::cout("[AI] Insert a NIF id:  ");
00466     std::getline(std::cin, entry);
00467
00468     try {
00469         user = std::stoi(entry);
00470     }catch (std::exception e){
00471         SimpleDisplay::cout("[AI] Input not valid");
00472         sleep(SLEEPTIME);
00473         return;
```

```
00474     }
00475
00476     SimpleDisplay::cout("[AI] Insert a password:  ");
00477
00478     try {
00479         password = std::stoi(entry);
00480     }catch (std::exception e) {
00481         SimpleDisplay::cout("[AI] Input not valid");
00482         sleep(SLEEPTIME);
00483         return;
00484     }
00485
00486     std::getline(std::cin, entry);
00487     do {
00488         SimpleDisplay::cout("[AI] Give the user Admin Status?  [y/n]:  ");
00489         std::getline(std::cin, entry);
00490     }while(entry!= "y" && entry!= "n");
00491
00492     isAdmin = (entry == "y" ?  1 :  0);
00493     User::addUser(user,password,isAdmin);
00494
00495 }
00496
00497 void AI::removeUser() {
00498
00499     std::string id, confirmation;
00500
00501     SimpleDisplay::cout("[" + TAG + "] Insert the id of the user to be removed:  ");
00502     std::getline(std::cin,id);
00503     do {
00504         SimpleDisplay::cout("[" + TAG + "] Are you sure to remove " + id
00505         + " from the User list?\nThis can't be undone [y/n]:");
00506         std::getline(std::cin,confirmation);
00507     }while(confirmation != "y" && confirmation != "n");
00508
00509     if (confirmation == "y") {
00510         try {
00511             User::rmUser(std::stoi(id));
00512         } catch (InvalidUserException &e) {
00513             SimpleDisplay::cout("[" + TAG + "] User requested not found");
00514         }
00515     }
00516     sleep(SLEEPTIME);
00517
00518 }
00519
00520 void AI::forceBadAlloc() {
00521
00522     SimpleDisplay::cout("\nBuilding Secure Environment...");
00523     sleep(SLEEPTIME);
00524     SimpleDisplay::cout("\nGenerating Error...");
00525     try {
00526         std::vector<int> vector;
00527         for (int i = 0; i > -1; i++)
00528             vector.push_back(i);
00529     }catch(std::exception &e) {
00530         SimpleDisplay::cout("\n[WARNING] Environment Failure:  \n-> what():  ");
00531         SimpleDisplay::cout(e.what());
00532         SimpleDisplay::cout("\nReturning...");
00533         sleep(SLEEPTIME*4);
00534     }
00535 }
00536
00537 void AI::signalHandler(int signum) {
00538
00539     // ----- Save all data -----
00540     User::saveUserData();
00541
00542     // ----- Send abortation signal -----
00543     abort = true;
00544     SimpleDisplay::security_abort = true;
00545
00546     // ----- Display EXIT CODE -----
00547     std::cout « "[" + TAG + "] Exit call detected.  Exiting JVH Systems...";
00548     sleep(SLEEPTIME);
00549     std::cout « "\n[" + TAG + "] Thank you for trusting us!\n\n";
00550     exit(0);
00551
00552 }
00553
00554 void AI::showAllUsers() {
00555     User::printUsers();
00556
00557     std::cout « "\nPress ENTER to return";
00558     std::cin.get();
00559
00560     //  Clear bad inputs
```

```
00561     std::cin.clear();
00562
00563 }
00564
00565
```

5.49 BaseException.cpp File Reference

```
#include "../include/except/BaseException.h"
```

5.50 BaseException.cpp

Go to the documentation of this file.

```
00001 #include "../include/except/BaseException.h"
00002 BaseException::BaseException() {}
00003
00004 const char* BaseException::what() {
00005
00006     return ("[" + tag + "] " + exception).c_str();
00007
00008 }
00009
00010
```

5.51 DataDevice.cpp File Reference

```
#include "../include/DataDevice.h"
```

5.51.1 Detailed Description

Author

Sebastian Mayorquin

Date

12/11/2022 @grade Software Robotics (Software Design)

Definition in file DataDevice.cpp.

5.52 DataDevice.cpp

Go to the documentation of this file.

```
00001 // ------------------- DataDevice - JVH Systems -------------------------------
00009 #include "../include/DataDevice.h"
00010
00011 // ----- Declarations ------
00012 DataDevice* DataDevice::allDataDevices;
00013 std::string* DataDevice::allDataDeviceNames;
00014 int DataDevice::numSensors = 0;
00015 int DataDevice::numSensorsActive = 0;
00016
00017
00018 // ------------------------------------------------------
00019 //                  PUBLIC FUNCTIONALITIES              |
00020 // ------------------------------------------------------
00021
00022
00023 // --- [Empty Constructor] DataDevice ---
00024 DataDevice::DataDevice() {
00025     this->name = "";
00026     this->id = INVALID_DEVICE;
00027     this->isDataDevice = true;
00028
00029 };
00030
00031 // --- [Constructor] DataDevice ---
00032 DataDevice::DataDevice(const std::string& name, int id, std::vector<void(*)()>* skillVector,
      std::string* skillNames) {
00033
00034     // ----- Counting Devices ------
00035     std::cout « "[DataDevice] New Device Detected:  0x" « id « std::endl;
00036     numSensors++;
00037
00038     // ----- Property assignation ------
00039     this->name = name;
00040     this->id = id;
00041     this->skills = skillVector;
00042     this->skillNames = skillNames;
00043     this->isDataDevice = true;
00044     this->numSkills = skillVector->size();
00045     this->isActive = true;
00046 }
00047
00048 // --- [Getter] getNumSensors ---
00049 int DataDevice::getNumSensors(){
00050     return numSensors;
00051 };
00052
00053
00054 // --- [Method] appendNewData ---
00055 void DataDevice::appendNewData() {
00056     auto clock = std::chrono::system_clock::now();
00057     time_t time = std::chrono::system_clock::to_time_t(clock);
00058     std::string time_stamp = std::ctime(&time);
00059     this->collectedData.push_back(this->dataGenerator());
00060     this->collectedTimes.push_back(time_stamp);
00061 }
00062
00063
00064 // ------------------------------------------------------
00065 //                  PRIVATE FUNCTIONALITIES             |
00066 // ------------------------------------------------------
00067
00068 // --- [Method] setLimit ---
00069 void DataDevice::setLimit(int* limitArray) {
00070     this->limit = limitArray;
00071     this->needsLimit = true;
00072 }
00073
00074
00075 // --- [Setter] setAllDataDevices ---
00076 void DataDevice::setAllDataDevices(DataDevice* allDevices){
00077     allDataDevices = allDevices;
00078     allDataDeviceNames = new std::string[numSensors];
00079
00080     for (int i = 0; i < numSensors; i++){
00081         allDataDeviceNames[i] = allDevices[i].getName();
00082     }
00083 }
```

5.53 Device.cpp File Reference

```
#include "../include/Device.h"
```

5.53.1 Detailed Description

Author

Sebastian Mayorquin

Date

12/11/2022 @grade Software Robotics (Software Design)

Definition in file Device.cpp.

5.54 Device.cpp

Go to the documentation of this file.

```
00001 // ------------------- Device - JVH Systems -----------------------------
00009 #include "../include/Device.h"
00010
00011 // ----- Declarations ------
00012 int Device::numDevices;
00013 Device* Device::allDevices;
00014 std::string* Device::allDeviceNames;
00015 const std::string Device::TAG = "Device";
00016
00017 // ---------------------------------------------------------
00018 //                  PUBLIC FUNCTIONALITIES                 |
00019 // ---------------------------------------------------------
00020
00021 // --- [Empty Constructor] Device ---
00022 Device::Device() :  name(""),id(INVALID_DEVICE){};
00023
00024 // --- [Constructor] Device ---
00025 Device::Device(const std::string& deviceName, const int deviceId):  name(deviceName),id(deviceId){}
00026
00027
00028 // --- [Getter] getName ---
00029 std::string Device::getName() {
00030     return this->name;
00031 }
00032
00033 int Device::getNumSkills(){
00034     return this->numSkills;
00035 }
00036
00037 // --- [Method] operator[] (int) ---
00038 void Device::operator[](const int index){
00039     try{
00040         (*this->skills)[index]();
00041
00042     }catch (std::exception){
00043         throw InvalidSkillException(TAG);
00044     }
00045 }
00046
00047 // --- [Method] operator[] (string) ---
00048 void Device::operator[](const std::string& skillName){
00049     // ----- Set number of skills ------
00050     int skillSize = this->numSkills;
00051
00052     // ----- Find and use Skill ------
00053     for (int i = 0; i < skillSize; i++){
00054         if (skillName == skillNames[i])
00055             try{
```

```
00056                 (*this->skills)[i]();
00057                 break;
00058             }catch (std::exception){
00059                 throw InvalidSkillException(TAG);
00060             }
00061
00062     }
00063 };
00064
00065 // --- [Operator] ~ ---
00066 bool Device::operator~() {
00067     return this->isActive;
00068 }
00069
00070 // -------------------------------------------------------
00071 //                  PRIVATE FUNCTIONALITIES              |
00072 // -------------------------------------------------------
00073
00074
00075 // --- [Setter] setAllDevices ---
00076 void Device::setAllDevices(Device* allNewDevices, std::string* allNewDeviceNames, int numDevices) {
00077
00078     // ----- Setting Device pointers and names ------
00079     std::cout « "[DEVICES] Collecting all devices:  " « std::endl;
00080     Device::allDevices = allNewDevices;
00081     Device::allDeviceNames = allNewDeviceNames;
00082     Device::numDevices = numDevices;
00083
00084     // ----- Log Success Device ------
00085     for (int i = 0; i < numDevices; i++){
00086         std::cout « "-> (SUCCESS) " « allNewDeviceNames[i] « std::endl;
00087     }
00088
00089 }
00090
00091 // --- [Setter] setDeviceCounter ---
00092 void Device::setDeviceCounter(int num) {
00093     Device::numDevices = num;
00094 }
```

5.55 DeviceBuilder.cpp File Reference

```
#include "../include/DeviceBuilder.h"
```

5.55.1 Detailed Description

Author

Sebastian Mayorquin

Date

12/11/2022 @grade Software Robotics (Software Design)

Definition in file DeviceBuilder.cpp.

5.56 DeviceBuilder.cpp

Go to the documentation of this file.

```
00001 // ------------------- DeviceBuilder - JVH Systems -------------------------------
00009 #include "../include/DeviceBuilder.h"
00010
00011
00012 // ----- Declarations ------
```

```
00013 std::vector<DeviceBuilder*> DeviceBuilder::allBuilders;
00014 std::vector<Device> DeviceBuilder::allDevices;
00015 std::vector<DataDevice> DeviceBuilder::allDataDevices;
00016 std::vector<KernelDevice> DeviceBuilder::allKernelDevices;
00017 int DeviceBuilder::deviceCounter;
00018
00019 // ------------------------------------------------------
00020 //                  PUBLIC FUNCTIONALITIES              |
00021 // ------------------------------------------------------
00022
00023 // --- [Static] buildDisplay ---
00024 SimpleDisplay* DeviceBuilder::buildDisplay(){
00025
00026     SimpleDisplay* display = new SimpleDisplay();
00027     display->setDataDevices(allDataDevices.size());
00028     display->setKernelDevices(allKernelDevices.size());
00029     display->setSecurityConnected(true);
00030     display->setSecurityStatus(true);
00031     return display;
00032
00033 }
00034
00035
00036 // --- [Static] buildAllDevices ---
00037 void DeviceBuilder::buildAllDevices() {
00038
00039     // ----- Initializing ------
00040     std::cout << "[BUILDER] Sending " << allBuilders.size() << " Devices to Kernel" << std::endl;
00041
00042     // ----- Declarations ------
00043     std::string *deviceNames = new std::string[allBuilders.size()]; // Array to Store Device names
00044     int numBuilders = allBuilders.size();
00045
00046     // ----- Building All Devices Saved as Kernel or Data Device ------
00047     for (int i = 0; i < numBuilders; i++){
00048
00049         // Saving device name in array
00050         deviceNames[i] = (*allBuilders[i]).name;
00051
00052         // Building as Data or Kernel Device
00053         (*allBuilders[i]).buildDKDevice();
00054     }
00055
00056     std::cout << "[BUILDER] Optimizing property access" << std::endl;
00057     // ----- All Devices Vector to Array ------
00058     Device *deviceArray = new Device[allDevices.size()];
00059     for (int i = 0; i < allDevices.size(); i++){
00060         deviceArray[i] = allDevices[i];
00061     }
00062
00063     // ----- All DataDevices Vector to Array ------
00064     DataDevice *dataDeviceArray = new DataDevice[allDataDevices.size()];
00065     for (int i = 0; i < allDataDevices.size(); i++){
00066         dataDeviceArray[i] = allDataDevices[i];
00067     }
00068
00069
00070     // ----- All KernelDevices Vector to Array ------
00071     KernelDevice *kernelDeviceArray = new KernelDevice[allKernelDevices.size()];
00072     for (int i = 0; i < allKernelDevices.size(); i++){
00073         kernelDeviceArray[i] = allKernelDevices[i];
00074     }
00075
00076     // ----- Storing Arrays ------
00077     std::cout << "[BUILDER] Storing General properties to Kernel..." << std::endl;
00078     Device::setAllDevices(deviceArray, deviceNames,numBuilders);
00079     DataDevice::setAllDataDevices(dataDeviceArray);
00080     KernelDevice::setAllKernelDevices(kernelDeviceArray);
00081
00082 }
00083
00084 // --- [Static] clearAll ---
00085 void DeviceBuilder::clearAll(){
00086
00087     // ----- Cleaning Vectors ------
00088     std::cout << "[BUILDER] Cleaning Builder..." << std::endl;
00089
00090     allDataDevices.clear();
00091     allDevices.clear();
00092     allKernelDevices.clear();
00093 }
00094
00095 // --- [Constructor] DeviceBuilder ---
00096 DeviceBuilder::DeviceBuilder(const std::string& name) {
00097
00098     // ----- Name Assign ------
00099     std::cout << "[BUILDER] Building " << name << std::endl;
```

```
00100     this->name = name;
00101 
00102     // ----- Storing Builder Pointer------
00103     DeviceBuilder* devicePointer = this;
00104     allBuilders.push_back(devicePointer);
00105 
00106     // ----- Settling Device ID ------
00107     enum {FANCY_ID = 55324,FANCY_SUM = 37};
00108     //Shhh..  It looks cooler with strange nums
00109     this->id = deviceCounter + FANCY_ID;
00110     deviceCounter += FANCY_SUM;
00111 
00112 };
00113 
00114 // --- [Method] setSkill ---
00115 void DeviceBuilder::setSkill(void(*skill)(), const std::string& skillName) {
00116 
00117     // ----- Storing Device Skills ------
00118     this->skillVector.push_back(skill);
00119     this->skillNamesVector.push_back(skillName);
00120 }
00121 
00122 // --- [Method] setLimit ---
00123 void DeviceBuilder::setLimit(int* newLimit) {
00124 
00125     // ----- Settling Device Limit ------
00126     this->limit = newLimit;
00127 }
00128 
00129 // --- [Setter] setAsDataDevice ---
00130 void DeviceBuilder::setAsDataDevice() {
00131     this->isDataDevice = true;
00132 }
00133 
00134 
00135 // --- [Setter] setAsKernelDevice ---
00136 void DeviceBuilder::setAsKernelDevice() {
00137     this->isDataDevice = false;
00138 }
00139 // --- [Method] setDataGenerator ---
00140 void DeviceBuilder::setDataGenerator(int (*dataGenerator)()) {
00141     this->dataGenerator = dataGenerator;
00142 }
00143 
00144 void DeviceBuilder::setSecurityGenerator(bool (*securityGenerator)()) {
00145     this->securityGenerator = securityGenerator;
00146 }
00147 
00148 // ------------------------------------------------------
00149 //                   PRIVATE FUNCTIONALITIES            |
00150 // ------------------------------------------------------
00151 
00152 // --- [Method] buildDKDevice---
00153 void DeviceBuilder::buildDKDevice() {
00154 
00155     // ----- Skills to Dynamic Vector (so it doesn't get deleted when builder dies) ------
00156     std::vector<void(*)()>* dynamicSkillVector = new std::vector<void(*)()>;
00157     dynamicSkillVector->assign(skillVector.begin(), skillVector.end());
00158 
00159     // ----- SkillNames to Array (Optimizing data) ------
00160     std::string *skillNamesArray = new std::string[skillNamesVector.size()];
00161     for (int i = 0; i < this->skillNamesVector.size(); i++){
00162         skillNamesArray[i] = skillNamesVector[i];
00163     }
00164 
00165     // ----- Building DataDevice ------
00166     if (this->isDataDevice) {
00167         DataDevice *dataDev = new DataDevice(this->name, this->id, dynamicSkillVector,
      skillNamesArray);
00168 
00169         // Setting DataDevice limit (optional)
00170         if (this->limit != nullptr)
00171             (*dataDev).setLimit(this->limit);
00172 
00173         (*dataDev).dataGenerator = this->dataGenerator;
00174 
00175         // Saving Device Pointers
00176         allDevices.push_back(*dataDev);
00177         allDataDevices.push_back(*dataDev);
00178 
00179     // ----- Building KernelDevice ------
00180     }else {
00181 
00182         KernelDevice* kernDev = new KernelDevice(this->name, this->id);
00183 
00184         (*kernDev).securityGenerator = this->securityGenerator;
00185 
```

```
00186         // Saving Device Pointers
00187         allDevices.push_back(*kernDev);
00188         allKernelDevices.push_back(*kernDev);
00189     }
00190
00191 }
00192
00193
```

5.57 DeviceComposite.cpp File Reference

```
#include "../include/DeviceComposite.h"
```

Variables

- std::string CALL_KEYWORD = "add"

5.57.1 Variable Documentation

5.57.1.1 CALL_KEYWORD

```
std::string CALL_KEYWORD = "add"
```

Definition at line 3 of file DeviceComposite.cpp.

5.58 DeviceComposite.cpp

Go to the documentation of this file.

```
00001 #include "../include/DeviceComposite.h"
00002 // ----- Declarations -----
00003 std::string CALL_KEYWORD = "add";
00004
00005 DeviceComposite::DeviceComposite() {}
00006
00007 // --- [Method] add
00008 void DeviceComposite::add(const std::string &name) {
00009
00010     // -----  Intantiating new IODi -----
00011     IODi iodi;
00012
00013     // -----  Adding device to IODi -----
00014     try{
00015         iodi.setCurrentDevice(name);
00016     }catch(InvalidDeviceException &e){
00017         throw;
00018         return;
00019     }
00020
00021     // -----  Saving IODi -----
00022     this->allIODis.push_back(iodi);
00023 }
00024 void DeviceComposite::pop() {
00025     this->allIODis.pop_back();
00026 }
00027
00028 // --- [Method] requestSkill
00029 void DeviceComposite::requestSkill(const std::string& skill) {
```

```
00030
00031     // -----  Check if skill is a keyword to add to composite -----
00032     if (skill == ::CALL_KEYWORD){
00033
00034         std::string answer;
00035         SimpleDisplay::deviCout("Write a Device name for adding it to the composite.");
00036         SimpleDisplay::deviCout("Ex.  <Thermometer>");
00037         SimpleDisplay::cout("~ Device Name:  ");
00038         std::getline(std::cin,answer);
00039         try{
00040             this->add(answer);
00041             SimpleDisplay::deviCout("Device Succesfully added");
00042             return;
00043         }catch(InvalidDeviceException &e){
00044             SimpleDisplay::deviCout("Invalid Device");
00045             return;
00046         }
00047     }
00048
00049     // -----  Else the keyword is for IODIs -----
00050     for (int i = 0; i < this->allIODis.size(); i++)
00051         this->allIODis[i].requestSkill(skill);
00052 }
00053
00054 // --- [Method] clear
00055 void DeviceComposite::clear() {
00056
00057     this->allIODis.clear();
00058
00059 }
00060
00061 std::string DeviceComposite::toString() {
00062
00063     // -----  Declarations -----
00064     std::string stringBuilder = "[ ";
00065
00066     for (int i = 0; i < this->allIODis.size(); i++){
00067
00068         // -----  Replacing name spaces with underscores.  -----
00069         std::string name = allIODis[i].requestCurrentDeviceName();
00070         std::replace(name.begin(), name.end(), ' ', '_');
00071
00072         // -----  Appending name -----
00073         stringBuilder += name + " ";
00074     }
00075
00076     // -----  Finishing string -----
00077     stringBuilder += "]";
00078     return stringBuilder;
00079 }
```

5.59 deviceDataset.cpp File Reference

```
#include "../include/DeviceBuilder.h"
#include "exampleSkillsDataset.cpp"
#include "exampleSecurityDataset.cpp"
```

Classes

- class deviceDataset

Macros

- #define DEVICE_DATASET

5.59.1 Macro Definition Documentation

5.59.1.1 DEVICE_DATASET

```
#define DEVICE_DATASET
```

Definition at line 2 of file deviceDataset.cpp.

5.60 deviceDataset.cpp

Go to the documentation of this file.

```
00001 #ifndef DEVICE_DATASET
00002 #define DEVICE_DATASET
00003 #include "../include/DeviceBuilder.h"
00004 #include "exampleSkillsDataset.cpp"
00005 #include "exampleSecurityDataset.cpp"
00006
00007 class deviceDataset {
00008 public:
00009
00010     static void initDataset(){
00011
00012         // ----- Building All Devices ------
00013         std::cout << "[BUILDER] Calling Default builders..." << std::endl;
00014         DeviceBuilder* thermometer = new DeviceBuilder("Thermometer");
00015         DeviceBuilder* humiditySensor = new DeviceBuilder("Humidity Sensor");
00016         DeviceBuilder* airSensor = new DeviceBuilder("Air Sensor");
00017         DeviceBuilder* lightSensor = new DeviceBuilder("Light Sensor");
00018         DeviceBuilder* rgbCamera = new DeviceBuilder("RGB Camera");
00019         DeviceBuilder* thermalCamera = new DeviceBuilder("Thermal Camera");
00020
00021         DeviceBuilder* securityCamera1 = new DeviceBuilder("Security Camera 1");
00022         DeviceBuilder* securityCamera2 = new DeviceBuilder("Security Camera 2");
00023         DeviceBuilder* laserDetector = new DeviceBuilder("Laser Detector");
00024
00025         // ----- Building Device Skills ------
00026         std::cout << "[BUILDER] Building skills" << std::endl;
00027
00028         // ----- Building Main Data Generator ------
00029         (*thermometer).setDataGenerator(generateRandomData);
00030         (*humiditySensor).setDataGenerator(generateRandomData);
00031         (*airSensor).setDataGenerator(generateRandomData);
00032         (*lightSensor).setDataGenerator(generateRandomData);
00033         (*rgbCamera).setDataGenerator(generateRandomData);
00034         (*thermalCamera).setDataGenerator(generateRandomData);
00035
00036         (*securityCamera1).setSecurityGenerator(exampleTest);
00037         (*securityCamera2).setSecurityGenerator(exampleTest);
00038         (*laserDetector).setSecurityGenerator(exampleTest);
00039
00040         // ----- Building Skills (Only for DataDevices) -----
00041         (*thermometer).setSkill(exampleSkill1, "calculate");
00042         (*thermometer).setSkill(exampleSkill2, "set celsius");
00043
00044         (*humiditySensor).setSkill(exampleSkill3,"hum decrease");
00045         (*humiditySensor).setSkill(exampleSkill4,"hum increase");
00046
00047         (*airSensor).setSkill(exampleSkill3,"air type --british");
00048
00049         (*lightSensor).setSkill(exampleSkill4,"setmode blind");
00050         (*lightSensor).setSkill(exampleSkill4,"setmode colorful");
00051         (*lightSensor).setSkill(exampleSkill4,"setmode hacker");
00052
00053         (*rgbCamera).setSkill(exampleSkill3,"print camera");
00054         (*rgbCamera).setSkill(exampleSkill4,"lookfor --myself");
00055
00056         (*thermalCamera).setSkill(exampleSkill3,"tcconfig up");
00057         (*thermalCamera).setSkill(exampleSkill4,"tcconfig down");
00058
00059         // ----- Building Device Limits (optional) ------
00060         std::cout << "[BUILDER] Building limits" << std::endl;
00061
00062     //    int lim[3] = {1, 2, 3};
00063     //    (*test).setLimit(lim);
00064
00065         // ----- Setting Device Types [Data/Kernel]
00066         std::cout << "[BUILDER] Settling Types" << std::endl;
00067         (*thermometer).setAsDataDevice();
00068         (*humiditySensor).setAsDataDevice();
00069         (*airSensor).setAsDataDevice();
```

```
00070         (*lightSensor).setAsDataDevice();
00071         (*rgbCamera).setAsDataDevice();
00072         (*thermalCamera).setAsDataDevice();
00073 
00074         (*securityCamera1).setAsKernelDevice();
00075         (*securityCamera2).setAsKernelDevice();
00076         (*laserDetector).setAsKernelDevice();
00077     }
00078 };
00079 
00080 #endif
```

5.61 exampleSecurityDataset.cpp File Reference

```
#include "../include/lib.h"
```

Macros

- #define EXAMPLE_SECURITY

Functions

- static bool exampleTest ()

5.61.1 Macro Definition Documentation

5.61.1.1 EXAMPLE_SECURITY

```
#define EXAMPLE_SECURITY
```

Definition at line 2 of file exampleSecurityDataset.cpp.

5.61.2 Function Documentation

5.61.2.1 exampleTest()

```
static bool exampleTest ( )  [static]
```

Definition at line 7 of file exampleSecurityDataset.cpp.

```
00007                           {
00008     return true;
00009 
00010 };
```

Referenced by AI::buildDevice(), and deviceDataset::initDataset().

5.62 exampleSecurityDataset.cpp

Go to the documentation of this file.

```
00001 #ifndef EXAMPLE_SECURITY
00002 #define EXAMPLE_SECURITY
00003 #include "../include/lib.h"
00004
00005 // --- [Function] exampleTest ---
00006 // Usage:  Example Security Skill for developer testing
00007 static bool exampleTest() {
00008     return true;
00009
00010 };
00011
00012 #endif
```

5.63 exampleSkillsDataset.cpp File Reference

```
#include "../include/lib.h"
```

Macros

- #define EXAMPLE_SKILLS

Functions

- static void exampleSkill1 ()
- static void exampleSkill2 ()
- static void exampleSkill3 ()
- static void exampleSkill4 ()
- static int generateRandomData ()

5.63.1 Macro Definition Documentation

5.63.1.1 EXAMPLE_SKILLS

```
#define EXAMPLE_SKILLS
```

Definition at line 2 of file exampleSkillsDataset.cpp.

5.63.2 Function Documentation

5.63.2.1 exampleSkill1()

```
static void exampleSkill1 ( )  [static]
```

Definition at line 7 of file exampleSkillsDataset.cpp.

```
00007                           {
00008     SimpleDisplay::deviCout("This is the example Skill 1");
00009
00010 };
```

References SimpleDisplay::deviCout().

Referenced by deviceDataset::initDataset().

5.63.2.2 exampleSkill2()

```
static void exampleSkill2 ( )  [static]
```

Definition at line 14 of file exampleSkillsDataset.cpp.

```
00014                           {
00015     SimpleDisplay::deviCout("This is the example Skill 2" );
00016 };
```

References SimpleDisplay::deviCout().

Referenced by deviceDataset::initDataset().

5.63.2.3 exampleSkill3()

```
static void exampleSkill3 ( )  [static]
```

Definition at line 20 of file exampleSkillsDataset.cpp.

```
00020                           {
00021     SimpleDisplay::deviCout("This is the example Skill 3");
00022 };
```

References SimpleDisplay::deviCout().

Referenced by deviceDataset::initDataset().

5.63.2.4 exampleSkill4()

```
static void exampleSkill4 ( )  [static]
```

Definition at line 26 of file exampleSkillsDataset.cpp.

```
00026                           {
00027     SimpleDisplay::deviCout("This is the example Skill 4");
00028 }
```

References SimpleDisplay::deviCout().

Referenced by deviceDataset::initDataset().

5.63.2.5 generateRandomData()

```
static int generateRandomData ( )  [static]
```

Definition at line 30 of file exampleSkillsDataset.cpp.

```
00030                                {
00031     return rand();
00032 }
```

Referenced by AI::buildDevice(), and deviceDataset::initDataset().

5.64 exampleSkillsDataset.cpp

Go to the documentation of this file.

```
00001 #ifndef EXAMPLE_SKILLS
00002 #define EXAMPLE_SKILLS
00003 #include "../include/lib.h"
00004
00005 // --- [Function] exampleSkill ---
00006 // Usage:  Example Skill for developer testing
00007 static void exampleSkill1() {
00008     SimpleDisplay::deviCout("This is the example Skill 1");
00009
00010 };
00011
00012 // --- [Function] exampleSkill ---
00013 // Usage:  Example Skill for developer testing
00014 static void exampleSkill2() {
00015     SimpleDisplay::deviCout("This is the example Skill 2" );
00016 };
00017
00018 // --- [Function] exampleSkill ---
00019 // Usage:  Example Skill for developer testing
00020 static void exampleSkill3() {
00021     SimpleDisplay::deviCout("This is the example Skill 3");
00022 };
00023
00024 // --- [Function] exampleSkill ---
00025 // Usage:  Example Skill for developer testing
00026 static void exampleSkill4() {
00027     SimpleDisplay::deviCout("This is the example Skill 4");
00028 }
00029
00030 static int generateRandomData() {
00031     return rand();
00032 }
00033
00034
00035 #endif
```

5.65 InsufficientPermissionsException.cpp File Reference

```
#include "../include/except/InsufficientPermissionsException.h"
```

5.66 InsufficientPermissionsException.cpp

Go to the documentation of this file.

```
00001 #include "../include/except/InsufficientPermissionsException.h"
00002 std::string InsufficientPermissionsException::ERROR_MESSAGE = "Invalid User";
00003
00004 InsufficientPermissionsException::InsufficientPermissionsException(const std::string &tag) {
00005
00006     this->exception = ERROR_MESSAGE;
00007     this->tag = tag;
00008
00009
00010 }
```

5.67 InvalidDeviceException.cpp File Reference

```
#include "../include/except/InvalidDeviceException.h"
```

5.68 InvalidDeviceException.cpp

Go to the documentation of this file.

```
00001 #include "../include/except/InvalidDeviceException.h"
00002 std::string InvalidDeviceException::ERROR_MESSAGE = "Invalid User";
00003
00004 InvalidDeviceException::InvalidDeviceException(const std::string &tag) {
00005
00006     this->exception = ERROR_MESSAGE;
00007     this->tag = tag;
00008
00009 }
```

5.69 InvalidFileException.cpp File Reference

```
#include "../include/except/InvalidFileException.h"
```

5.70 InvalidFileException.cpp

Go to the documentation of this file.

```
00001 #include "../include/except/InvalidFileException.h"
00002 std::string InvalidFileException::ERROR_MESSAGE = "Invalid File";
00003
00004 InvalidFileException::InvalidFileException(const std::string &tag) {
00005
00006     this->exception = ERROR_MESSAGE;
00007     this->tag = tag;
00008 }
```

5.71 InvalidSkillException.cpp File Reference

```
#include "../include/except/InvalidSkillException.h"
```

5.72 InvalidSkillException.cpp

Go to the documentation of this file.

```
00001 #include "../include/except/InvalidSkillException.h"
00002 std::string InvalidSkillException::ERROR_MESSAGE = "Invalid File";
00003
00004 InvalidSkillException::InvalidSkillException(const std::string &tag) {
00005
00006     this->exception = ERROR_MESSAGE;
00007     this->tag = tag;
00008
00009 }
```

5.73 InvalidUserException.cpp File Reference

```
#include "../include/except/InvalidUserException.h"
```

5.74 InvalidUserException.cpp

Go to the documentation of this file.

```
00001 #include "../include/except/InvalidUserException.h"
00002
00003 std::string InvalidUserException::ERROR_MESSAGE = "Invalid User";
00004
00005 InvalidUserException::InvalidUserException(const std::string &tag) {
00006
00007     this->exception = ERROR_MESSAGE;
00008     this->tag = tag;
00009
00010 }
```

5.75 IODi.cpp File Reference

```
#include "../include/IODi.h"
#include "../include/AI.h"
```

5.75.1 Detailed Description

Author

Sebastian Mayorquin

Date

16/11/2022 @grade Software Robotics (Software Design)

Definition in file IODi.cpp.

5.76 IODi.cpp

Go to the documentation of this file.

```
00001 // ------------------- Input/Output Device Interface - JVH Systems -------------------------------
00010 #include "../include/IODi.h"
00011 #include "../include/AI.h"
00012
00013 // ----- Declarations ------
00014 int IODi::maxID = 0;
00015 std::string IODi::TAG = "IODi";
00016
00017 // ------------------------------------------------------
00018 //                  PUBLIC FUNCTIONALITIES              |
00019 // ------------------------------------------------------
00020
00021 // --- [Empty Constructor] IODi() ---
00022 IODi::IODi(){
00023     this->id = DEFAULT_ID;
```

```
00024     maxID++;
00025 };
00026
00027 // --- [Setter] setCurrentDevice ---
00028 void IODi::setCurrentDevice(const int pos) {
00029
00030     // ----- If position is valid ------
00031     if (pos < IODi::requestNumberOfDataDevices()) {
00032
00033         // Set position in current device
00034         this->currentDevicePos = pos;
00035
00036         // Search and set name in current device
00037         this->currentDeviceName = DataDevice::allDataDeviceNames[pos];
00038
00039     }
00040     else
00041         // ----- Argument not valid ------
00042         throw InvalidDeviceException(TAG);
00043
00044 }
00045
00046 // --- [Setter] setCurrentDevice ---
00047 void IODi::setCurrentDevice(const std::string& name) {
00048
00049     // ----- Check type argument is not a number ------
00050     if (std::isdigit(name[0])){
00051
00052         this->setCurrentDevice(std::stoi(name));
00053         return;
00054     }
00055
00056     // ----- Declarations ------
00057     int maxDevices = DataDevice::getNumSensors();
00058
00059     // ----- Search Device ------
00060     for (int i = 0; i < maxDevices; i++){
00061
00062         if (name == DataDevice::allDataDeviceNames[i]){
00063
00064             // Save position and name in current device
00065             this->currentDevicePos = i;
00066             this->currentDeviceName = name;
00067             return;
00068
00069         }
00070
00071     }
00072     // ----- Device not valid ------
00073     throw InvalidDeviceException(TAG);
00074
00075 }
00076
00077
00078 // --- [REQUEST] -> Name of all Devices
00079 std::string* IODi::requestDeviceNames(){
00080     return DataDevice::allDataDeviceNames;
00081 };
00082
00083 // --- [REQUEST] -> Number of all DataDevices
00084 int IODi::requestNumberOfDataDevices(){
00085     return DataDevice::getNumSensors();
00086 };
00087
00088 // --- [REQUEST] -> Number of Skills of current device
00089 int IODi::requestNumberOfSkills(){
00090     return DataDevice::allDataDevices[this->currentDevicePos].getNumSkills();
00091 }
00092
00093 // --- [REQUEST] -> Name of Skills of current device
00094 std::string* IODi::requestDeviceSkills(){
00095     int numSkills = DataDevice::allDataDevices[this->currentDevicePos].getNumSkills();
00096
00097     std::string* deviceSkills = new std::string[numSkills];
00098
00099     for (int i = 0; i < numSkills; i++){
00100         deviceSkills[i] = DataDevice::allDataDevices[this->currentDevicePos].skillNames[i];
00101     }
00102
00103     return deviceSkills;
00104 };
00105 // --- [Function] requestAllStatus()
00106 bool* IODi::requestAllStatus() {
00107
00108     bool* status = new bool[DataDevice::numSensors];
00109
00110     for (int i = 0; i < DataDevice::numSensors; i++)
```

```
00111         status[i] = ~DataDevice::allDataDevices[i];
00112
00113     return status;
00114 };
00115
00116 // --- [REQUEST] -> Uses a skill given the position in device array
00117 void IODi::requestSkill(const int skill){
00118     try {
00119         DataDevice::allDataDevices[this->currentDevicePos][skill];
00120     }catch(InvalidSkillException &e){};
00121 };
00122
00123 // --- [REQUEST] -> Name of current device
00124 std::string IODi::requestCurrentDeviceName() {
00125     return this->currentDeviceName;
00126 }
00127
00128 // --- [REQUEST] -> Uses a skill given the keyword
00129 void IODi::requestSkill(const std::string& skill){
00130
00131     // ----- Obtaining current device -----
00132     DataDevice* target = &DataDevice::allDataDevices[this->currentDevicePos];
00133
00134     // ----- Switch between possible arguments ------
00135     if (skill == ":help")
00136         SimpleDisplay::printDeviHelp();
00137
00138     else if (skill == ":dev")
00139         SimpleDisplay::printSkillManual(target->skillNames,\
00140                                         this->requestNumberOfSkills());
00141                                         // -> Vector with the name of skills
00142                                         // -> Int with the number of skills
00143
00144     else if (skill == ":clean")
00145         SimpleDisplay::cleanDevi(this->currentDeviceName);
00146
00147     else if (skill == "data")
00148         SimpleDisplay::deviPrintData(target->collectedData,\
00149                                      target->collectedTimes);
00150                                      // -> Vector with measurements
00151                                      // -> Vector with times of measurements
00152
00153     else if(skill == "turnoff"){
00154         target->isActive = false;
00155         target->collectedData.resize(1);
00156         target->collectedData[0] = 0;
00157         target->collectedTimes.resize(1);
00158         target->collectedTimes[0] = "REBOOT";
00159         SimpleDisplay::deviCout("[IODi] " + target->getName() + " turned OFF");
00160     }
00161
00162     else if(skill == "turnon"){
00163         target->isActive = true;
00164         SimpleDisplay::deviCout("[IODi] " + target->getName() + " turned ON");
00165     }
00166
00167     else if (skill == "reboot"){
00168         this->requestSkill("turnoff");
00169         this->requestSkill("turnon");
00170     }
00171
00172     else if (skill == "pwd")
00173         SimpleDisplay::deviCout("/home/devices/" + std::to_string(this->currentDevicePos));
00174
00175     // ----- Frequent Invalid Commands ------
00176
00177     else if (skill == "help")
00178         SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :help");
00179
00180     else if (skill == "exit")
00181         SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :wq");
00182
00183     else if (skill == "clear")
00184         SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :clean");
00185
00186     else if (skill == "clean")
00187         SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :clean");
00188
00189     else if (skill == "dev")
00190         SimpleDisplay::deviCout("[IODi] Suggestion:  Maybe you are looking for -> :dev");
00191
00192     else if (skill[0] == '\"' || skill[0] == '\'') {
00193         SimpleDisplay::deviCout("Are you trying to Inject SQL? Lol ok");
00194         SimpleDisplay::deviCout("[YOU HAVE BEEN BANNED]");
00195         exit(1);
00196     }
00197
```

```
00198     // ----- Not Global command, request to current device ------
00199     else {
00200         try{
00201             (*target)[skill];
00202         }catch(InvalidSkillException &e){};
00203     }
00204 }
```

5.77 KernelDevice.cpp File Reference

```
#include "../include/KernelDevice.h"
```

5.77.1 Detailed Description

Author

Sebastian Mayorquin

Date

12/11/2022 @grade Software Robotics (Software Design)

Definition in file KernelDevice.cpp.

5.78 KernelDevice.cpp

Go to the documentation of this file.

```
00001 // ------------------- KernelDevice - JVH Systems -------------------------------
00009 #include "../include/KernelDevice.h"
00010
00011
00012 // ----- Declarations ------
00013 KernelDevice* KernelDevice::allKernelDevices;
00014 int KernelDevice::numKerns = 0;
00015
00016 // ------------------------------------------------------
00017 //                  PUBLIC FUNCTIONALITIES              |
00018 // ------------------------------------------------------
00019
00020 // --- [Empty Constructor] KernelDevice ---
00021 KernelDevice::KernelDevice() {
00022     this->name = "";
00023     this->isActive = false;
00024     this->id = INVALID_DEVICE;
00025     this->isDataDevice = false;
00026     this->securityHasBeenBroken = false;
00027 };
00028
00029 // --- [Constructor] KernelDevice ---
00030 KernelDevice::KernelDevice(const std::string& name, const int id) {
00031
00032     std::cout << "[KernelDevice] New Device Detected:  0x" << id << std::endl;
00033
00034     numKerns++;
00035
00036     this->name = name;
00037     this->id = id;
00038     this->isDataDevice = false;
00039     this->isActive = true;
00040     this->securityHasBeenBroken = false;
00041 }
00042
00043 // ------------------------------------------------------
00044 //                  PRIVATE FUNCTIONALITIES             |
00045 // ------------------------------------------------------
00046
00047 // --- [Setter] setAllKernelDevices ---
00048 void KernelDevice::setAllKernelDevices(KernelDevice* allDevices){
00049     allKernelDevices = allDevices;
00050 }
00051
```

5.79 Login.cpp File Reference

```
#include "../include/Login.h"
```

5.79.1 Detailed Description

Author

Sebastian Mayorquin

Date

03/11/2022 @grade Software Robotics (Software Design)

Definition in file Login.cpp.

5.80 Login.cpp

Go to the documentation of this file.

```
00001 // ------------------- Login - JVH Systems -----------------------------
00009 #include "../include/Login.h"
00010 // ----- Declarations ------
00011 int Login::SLEEP_TIME = 1;
00012 int Login::EXIT_REQUEST = -1;
00013
00014 // ------------------------------------------------------
00015 //                  PUBLIC FUNCTIONALITIES              |
00016 // ------------------------------------------------------
00017
00018 // --- [Constructor] Login() ---
00019 Login::Login() {};
00020
00021
00022
00023 // --- [Method] startLogin() ---
00024 User Login::startLogin() {
00025
00026
00027     // ----- Declarations ------
00028     bool badAccess = false;          // Bool to detect if input is not correct.
00029
00030     // ----- Logger Bucle ------
00031     do {
00032
00033         //  Checking Bad Inputs
00034         if (badAccess) {
00035             SimpleDisplay::cout("Bad Access");
00036             sleep(SLEEP_TIME);
00037         }
00038
00039         //  Reset Variables
00040         user = User::INVALID_USER;
00041         pass = User::INVALID_PASS;
00042
00043         //  Prompt user
00044         SimpleDisplay::printLoginInterface(user, pass, User::INVALID_USER, User::INVALID_PASS);
00045         SimpleDisplay::cout("\nKeyboard:  ");
00046         std::cin » user;
00047
00048         if (user == EXIT_REQUEST) raise(SIGINT);
00049
00050         //  Prompt password
00051         SimpleDisplay::printLoginInterface(user, pass, User::INVALID_USER, User::INVALID_PASS);
00052         SimpleDisplay::cout("\nKeyboard:  ");
00053         std::cin » pass;
00054
00055         if (pass == EXIT_REQUEST) raise(SIGINT);
```

```
00056
00057         //  Print checking interface
00058         SimpleDisplay::printLoginInterface(user, pass, User::INVALID_USER, User::INVALID_PASS);
00059         SimpleDisplay::printChecking();
00060
00061         //  Prepare next logger (if needed)
00062         badAccess = true;
00063         sleep(SLEEP_TIME);
00064
00065         //  Clear bad inputs
00066         std::cin.clear();
00067         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00068
00069         //  Loop until the user and password are valid
00070     } while (!User::check(user, pass));
00071
00072     // Return Admin (If needed)
00073     if (Admin::adminStatus(user))
00074         return Admin(user);
00075
00076     // ----- Returning User ------
00077     return User(user);
00078 };
00079
00080
```

5.81 main.cpp File Reference

This is the main Test Program, Running for testing snapshot.

```
#include "../include/DeviceComposite.h"
```

main

Author

Sebastian Mayorquin

- enum { MAIN_SCREEN = 0 , DEVICE_SCREEN = 1 , SETTINGS_SCREEN = 2 }
- int main (int argc, char **argv)

  Main function to start and keep running JVH systems.

5.81.1 Detailed Description

This is the main Test Program, Running for testing snapshot.

Date

04/11/2022

Version

v.0.2.1

Definition in file main.cpp.

5.81.2 Enumeration Type Documentation

5.81.2.1 anonymous enum

```
anonymous enum
```

Defining the different screens that the program can have.

Enumerator

MAIN_SCREEN	
DEVICE_SCREEN	
SETTINGS_SCREEN	

Definition at line 14 of file main.cpp.

```
00014     {
00015     MAIN_SCREEN = 0,
00016     DEVICE_SCREEN = 1,
00017     SETTINGS_SCREEN = 2
00018 };
```

5.81.3 Function Documentation

5.81.3.1 main()

```
int main (
            int argc,
            char ** argv )
```

Main function to start and keep running JVH systems.

Parameters

argc	Number of arguments passed to the program.
argv	Argument vector.

Definition at line 27 of file main.cpp.

```
00027                                   {
00028
00029     // ----- Checking Arguments ------
00030     bool IS_DEVELOPER = false;
00031
00032 #ifdef _WIN32
00033     for (int i = 0; i < argc; i++){
00034
00035         if (strcmp(argv[i], "--no-colors") == 0){
00036             SimpleDisplay::COLOR = "";
00037             SimpleDisplay::GREY_COLOR = "";
00038             SimpleDisplay::WARNING_COLOR = "";
00039             SimpleDisplay::RESET_COLOR = "";
00040         }
00041         if (strcmp(argv[i], "--developer") == 0)
00042             IS_DEVELOPER = true;
00043     }
00044 #else
00045         // Code for Linux
00046         for (int i = 0; i < argc; i++){
00047             if (strncmp(argv[i], "--no-colors", 11) == 0){
00048                 SimpleDisplay::COLOR = "";
00049                 SimpleDisplay::GREY_COLOR = "";
00050                 SimpleDisplay::WARNING_COLOR = "";
00051                 SimpleDisplay::RESET_COLOR = "";
00052             }
00053             if (strncmp(argv[i], "--developer", 11) == 0)
00054                 IS_DEVELOPER = true;
00055         }
00056 #endif
00057
00058     // ----- Getting current directory -----
00059     std::string executablePath(argv[0]);
```

```
00060     User::EXECUTABLE_DIR = executablePath.substr(0, executablePath.find_last_of("\\/"));
00061     // ----- Initializing Builder + Display ------
00062     std::cout « "-------- [STARTING BUILDING] --------" « std::endl;
00063     deviceDataset::initDataset();
00064     DeviceBuilder::buildAllDevices();
00065     SimpleDisplay* display = DeviceBuilder::buildDisplay();
00066
00067     // ----- Initializing AI------
00068     AI* ai = new AI();
00069     ai->startCollectingData();
00070
00071     std::cout « "-------- [BUILDING SUCCESS] --------" « std::endl;
00072
00073     // ----- Initializing IODi ------
00074     int screen_status = MAIN_SCREEN;
00075     std::string answer = "";
00076     IODi* iodi = new IODi();
00077
00078     // ----- Initializig composite -----
00079     DeviceComposite* composite = new DeviceComposite;
00080
00081     // ----- Initializing User Data------
00082     try {
00083         User::updateUserData();
00084     }catch (InvalidFileException &e){
00085         std::cout « "\n[Login] There has been an error updating user file, exiting...";
00086         exit(1);
00087         std::cout « "[HERE]";
00088     }catch (std::exception &e){
00089         std::cout « "\n[Login] There has been an error accessing the file, exiting...";
00090         exit(1);
00091     }
00092     // ----- Initializing Login ------
00093     Login* l = new Login();
00094
00095     std::cout « "\nPress ENTER to start";
00096     std::cin.get();
00097
00098     //  Clear bad inputs
00099     std::cin.clear();
00100
00101     // ----- Checking Developer Status ------
00102     if (!IS_DEVELOPER) {
00103         SimpleDisplay::printPresentation();
00104         std::cout « "\nPress ENTER to start";
00105         std::cin.get();
00106     }
00107
00108     //  Clear bad inputs
00109     std::cin.clear();
00110
00111     // ----- Sending to Logger ------
00112     User user;
00113     if (!IS_DEVELOPER)
00114         user = l->startLogin();
00115     else {
00116         user = User(User::DEVELOPER_USER);
00117     }
00118     // ----- Setting User ------
00119     display->setUser(IS_DEVELOPER ? User::DEVELOPER_USER :  user.getUser());
00120     display->setGroup((user.getAdminStatus() ?  "ADMIN" :  "GUEST"));
00121
00122     // ----- [MAIN BUCLE] ------
00123     while(true){
00124
00125         // ----- MAIN SCREEN ------
00126         if (screen_status == MAIN_SCREEN) {
00127
00128             // Print Display
00129             std::string *deviceNames = iodi->requestDeviceNames();
00130             display->printMainInterface(deviceNames,iodi->requestAllStatus());
00131
00132             // Request an entry
00133             SimpleDisplay::cout("Keyboard:  ");
00134             ai->throwDemon(); // Update while not entry
00135             std::getline(std::cin,answer);
00136
00137             // Exit if requested
00138             if (answer == "exit" || answer == "-1") {
00139
00140                 // Generating Logger
00141                 user = l->startLogin();
00142                 display->setUser(user.getUser());
00143                 display->setGroup((user.getAdminStatus() ?  "ADMIN" :  "GUEST"));
00144
00145             }
00146
```

```
00147             // Go to Settings if requested
00148             else if (answer == "settings" || answer == "-2") {
00149                 screen_status = SETTINGS_SCREEN;
00150             }
00151 
00152             // Entry not recognized, sending to IODi
00153             else
00154                 try {
00155 
00156                     // Send to IODi and switch screen
00157                     composite->add(answer);
00158                     iodi->setCurrentDevice(answer);
00159                     screen_status = DEVICE_SCREEN;
00160 
00161                 }catch (InvalidDeviceException &e){
00162 
00163                     //IODi hadn't found valid option, reset
00164                     SimpleDisplay::cout("INVALID OPTION");
00165                     sleep(1);
00166                 }
00167 
00168         // ----- DEVi++ Terminal ------
00169         }else if (screen_status == DEVICE_SCREEN){
00170 
00171             // Print heeader and request entry
00172             SimpleDisplay::printDeviHeader(iodi->requestCurrentDeviceName());
00173 
00174             // Don't exit the terminal until exit
00175             while (answer != ":wq") {
00176 
00177                 // Request an entry
00178                 SimpleDisplay::cout("~ ");
00179                 std::getline(std::cin, answer);
00180 
00181                 // Force refresh the devices if requested
00182                 if (answer == "forceref") {
00183                     ai->refreshData();
00184                     SimpleDisplay::deviCout("[AI] Forced current Device to Refresh");
00185                 }
00186 
00187                 // Show the user if requested
00188                 else if (answer == "who"){
00189                     SimpleDisplay::deviCout(std::to_string(user.getUser()));
00190                 }
00191 
00192                 // Show the composite status
00193                 else if (answer == "comp")
00194                     SimpleDisplay::deviCout(composite->toString());
00195 
00196                 else if (answer == "pop") {
00197                     composite->pop();
00198                     SimpleDisplay::deviCout("Last Device removed from composite");
00199                 }
00200 
00201                 // Command not recognized, sending to IODi
00202                 else
00203                     composite->requestSkill(answer);
00204             }
00205 
00206             // When DEVi++ finish, switch to main screen
00207             composite->clear();
00208             screen_status = MAIN_SCREEN;
00209 
00210         }else if (screen_status == SETTINGS_SCREEN){
00211 
00212             while (answer != "-1") {
00213 
00214                 SimpleDisplay::printAIInterface();
00215                 SimpleDisplay::cout("Keyboard:  ");
00216                 std::getline(std::cin, answer);
00217                 AI::requestOption(answer,user.getAdminStatus());
00218             }
00219             SimpleDisplay copy = *display;
00220             display = DeviceBuilder::buildDisplay();
00221             *display « copy;
00222 
00223             SimpleDisplay::currentDisplay = display;
00224             screen_status = MAIN_SCREEN;
00225 
00226         }
00227     }
00228 }
```

References DeviceComposite::add(), DeviceBuilder::buildAllDevices(), DeviceBuilder::buildDisplay(), DeviceComposite::clear(), SimpleDisplay::COLOR, SimpleDisplay::cout(), SimpleDisplay::currentDisplay, User::DEVELOPER_USER, DEVICE_SCREEN, SimpleDisplay::deviCout(), User::EXECUTABLE_DIR, User::getAdminStatus(), User::getUser(),

SimpleDisplay::GREY_COLOR, deviceDataset::initDataset(), MAIN_SCREEN, DeviceComposite::pop(), SimpleDisplay::printAIInterfa
SimpleDisplay::printDeviHeader(), SimpleDisplay::printMainInterface(), SimpleDisplay::printPresentation(),
AI::refreshData(), IODi::requestAllStatus(), IODi::requestCurrentDeviceName(), IODi::requestDeviceNames(),
AI::requestOption(), DeviceComposite::requestSkill(), SimpleDisplay::RESET_COLOR, IODi::setCurrentDevice(),
SimpleDisplay::setGroup(), SETTINGS_SCREEN, SimpleDisplay::setUser(), AI::startCollectingData(), Login::startLogin(),
AI::throwDemon(), DeviceComposite::toString(), User::updateUserData(), and SimpleDisplay::WARNING_COLOR.

5.82 main.cpp

Go to the documentation of this file.

```
00001 // ---------------------------- main - JVH Systems -------------------------------
00012 #include "../include/DeviceComposite.h"
00014 enum{
00015     MAIN_SCREEN = 0,
00016     DEVICE_SCREEN = 1,
00017     SETTINGS_SCREEN = 2
00018 };
00019
00020
00027 int main(int argc, char** argv)  {
00028
00029     // ----- Checking Arguments ------
00030     bool IS_DEVELOPER = false;
00031
00032 #ifdef _WIN32
00033     for (int i = 0; i < argc; i++){
00034
00035         if (strcmp(argv[i], "--no-colors") == 0){
00036             SimpleDisplay::COLOR = "";
00037             SimpleDisplay::GREY_COLOR = "";
00038             SimpleDisplay::WARNING_COLOR = "";
00039             SimpleDisplay::RESET_COLOR = "";
00040         }
00041         if (strcmp(argv[i], "--developer") == 0)
00042             IS_DEVELOPER = true;
00043     }
00044 #else
00045         // Code for Linux
00046         for (int i = 0; i < argc; i++){
00047             if (strncmp(argv[i], "--no-colors", 11) == 0){
00048                 SimpleDisplay::COLOR = "";
00049                 SimpleDisplay::GREY_COLOR = "";
00050                 SimpleDisplay::WARNING_COLOR = "";
00051                 SimpleDisplay::RESET_COLOR = "";
00052             }
00053             if (strncmp(argv[i], "--developer", 11) == 0)
00054                 IS_DEVELOPER = true;
00055         }
00056 #endif
00057
00058     // ----- Getting current directory -----
00059     std::string executablePath(argv[0]);
00060     User::EXECUTABLE_DIR = executablePath.substr(0, executablePath.find_last_of("\\/"));
00061     // ----- Initializing Builder + Display ------
00062     std::cout << "-------- [STARTING BUILDING] --------" << std::endl;
00063     deviceDataset::initDataset();
00064     DeviceBuilder::buildAllDevices();
00065     SimpleDisplay* display = DeviceBuilder::buildDisplay();
00066
00067     // ----- Initializing AI------
00068     AI* ai = new AI();
00069     ai->startCollectingData();
00070
00071     std::cout << "-------- [BUILDING SUCCESS] --------" << std::endl;
00072
00073     // ----- Initializing IODi ------
00074     int screen_status = MAIN_SCREEN;
00075     std::string answer = "";
00076     IODi* iodi = new IODi();
00077
00078     // ----- Initializig composite -----
00079     DeviceComposite* composite = new DeviceComposite;
00080
00081     // ----- Initializing User Data------
00082     try {
00083         User::updateUserData();
00084     }catch (InvalidFileException &e){
00085         std::cout << "\n[Login] There has been an error updating user file, exiting...";
00086         exit(1);
```

```
00087         std::cout « "[HERE]";
00088     }catch (std::exception &e){
00089         std::cout « "\n[Login] There has been an error accessing the file, exiting...";
00090         exit(1);
00091     }
00092     // ----- Initializing Login ------
00093     Login* l = new Login();
00094 
00095     std::cout « "\nPress ENTER to start";
00096     std::cin.get();
00097 
00098     //  Clear bad inputs
00099     std::cin.clear();
00100 
00101     // ----- Checking Developer Status ------
00102     if (!IS_DEVELOPER) {
00103         SimpleDisplay::printPresentation();
00104         std::cout « "\nPress ENTER to start";
00105         std::cin.get();
00106     }
00107 
00108     //  Clear bad inputs
00109     std::cin.clear();
00110 
00111     // ----- Sending to Logger ------
00112     User user;
00113     if (!IS_DEVELOPER)
00114         user = l->startLogin();
00115     else {
00116         user = User(User::DEVELOPER_USER);
00117     }
00118     // ----- Setting User ------
00119     display->setUser(IS_DEVELOPER ? User::DEVELOPER_USER :  user.getUser());
00120     display->setGroup((user.getAdminStatus() ?  "ADMIN" :  "GUEST"));
00121 
00122     // ----- [MAIN BUCLE] ------
00123     while(true){
00124 
00125         // ----- MAIN SCREEN ------
00126         if (screen_status == MAIN_SCREEN) {
00127 
00128             // Print Display
00129             std::string *deviceNames = iodi->requestDeviceNames();
00130             display->printMainInterface(deviceNames,iodi->requestAllStatus());
00131 
00132             // Request an entry
00133             SimpleDisplay::cout("Keyboard:  ");
00134             ai->throwDemon(); // Update while not entry
00135             std::getline(std::cin,answer);
00136 
00137             // Exit if requested
00138             if (answer == "exit" || answer == "-1") {
00139 
00140                 // Generating Logger
00141                 user = l->startLogin();
00142                 display->setUser(user.getUser());
00143                 display->setGroup((user.getAdminStatus() ?  "ADMIN" :  "GUEST"));
00144 
00145             }
00146 
00147             // Go to Settings if requested
00148             else if (answer == "settings" || answer == "-2") {
00149                 screen_status = SETTINGS_SCREEN;
00150             }
00151 
00152             // Entry not recognized, sending to IODi
00153             else
00154                 try {
00155 
00156                     // Send to IODi and switch screen
00157                     composite->add(answer);
00158                     iodi->setCurrentDevice(answer);
00159                     screen_status = DEVICE_SCREEN;
00160 
00161                 }catch (InvalidDeviceException &e){
00162 
00163                     //IODi hadn't found valid option, reset
00164                     SimpleDisplay::cout("INVALID OPTION");
00165                     sleep(1);
00166                 }
00167 
00168         // ----- DEVi++ Terminal ------
00169         }else if (screen_status == DEVICE_SCREEN){
00170 
00171             // Print heeader and request entry
00172             SimpleDisplay::printDeviHeader(iodi->requestCurrentDeviceName());
00173 
```

```
00174             // Don't exit the terminal until exit
00175             while (answer != ":wq") {
00176 
00177                 // Request an entry
00178                 SimpleDisplay::cout("~ ");
00179                 std::getline(std::cin, answer);
00180 
00181                 // Force refresh the devices if requested
00182                 if (answer == "forceref") {
00183                     ai->refreshData();
00184                     SimpleDisplay::deviCout("[AI] Forced current Device to Refresh");
00185                 }
00186 
00187                 // Show the user if requested
00188                 else if (answer == "who"){
00189                     SimpleDisplay::deviCout(std::to_string(user.getUser()));
00190                 }
00191 
00192                 // Show the composite status
00193                 else if (answer == "comp")
00194                     SimpleDisplay::deviCout(composite->toString());
00195 
00196                 else if (answer == "pop") {
00197                     composite->pop();
00198                     SimpleDisplay::deviCout("Last Device removed from composite");
00199                 }
00200 
00201                 // Command not recognized, sending to IODi
00202                 else
00203                     composite->requestSkill(answer);
00204             }
00205 
00206             // When DEVi++ finish, switch to main screen
00207             composite->clear();
00208             screen_status = MAIN_SCREEN;
00209 
00210         }else if (screen_status == SETTINGS_SCREEN){
00211 
00212             while (answer != "-1") {
00213 
00214                 SimpleDisplay::printAIInterface();
00215                 SimpleDisplay::cout("Keyboard:  ");
00216                 std::getline(std::cin, answer);
00217                 AI::requestOption(answer,user.getAdminStatus());
00218             }
00219             SimpleDisplay copy = *display;
00220             display = DeviceBuilder::buildDisplay();
00221             *display « copy;
00222 
00223             SimpleDisplay::currentDisplay = display;
00224             screen_status = MAIN_SCREEN;
00225 
00226         }
00227     }
00228 }
```

5.83 SimpleDisplay.cpp File Reference

```
#include "../include/SimpleDisplay.h"
```

5.83.1 Detailed Description

Author

Sebastian Mayorquin

Date

03/11/2022 @grade Software Robotics (Software Design)

Definition in file SimpleDisplay.cpp.

5.84 SimpleDisplay.cpp

Go to the documentation of this file.

```
00001 // ------------------- SimpleDisplay - JVH Systems -------------------------------
00008 #include "../include/SimpleDisplay.h"
00009
00010 const int SimpleDisplay::MAXIMUM_OPTIONS = 12;
00011 std::string SimpleDisplay::COLOR = "\033[36m";
00012 std::string SimpleDisplay::WARNING_COLOR = "\033[31m";
00013 std::string SimpleDisplay::RESET_COLOR = "\033[0m";
00014 std::string SimpleDisplay::GREY_COLOR = "\033[32m";
00015 SimpleDisplay* SimpleDisplay::currentDisplay;
00016 bool SimpleDisplay::security_abort = false;
00017
00018 // --- [Function] SimpleDisplay()
00019 SimpleDisplay::SimpleDisplay() {
00020
00021     SimpleDisplay::currentDisplay = this;
00022
00023 };
00024
00025 // ---------------------------------------------------
00026 //                          LOGIN                   |
00027 // ---------------------------------------------------
00028
00029 // --- [Static] printLoginInterface ---
00030 void SimpleDisplay::printLoginInterface(const int user, const int pass, const int INVALID_USER, const
      int INVALID_PASS){
00031
00032     // ----- Declarations ------
00033
00034     std::string current_user;
00035     std::string current_pass;
00036
00037
00038     // ----- Checking if variables are printable ------
00039
00040     if (user == INVALID_USER) current_user = "";
00041     else current_user = std::to_string(user);
00042
00043     if (pass == INVALID_PASS) current_pass = "";
00044     else current_pass= std::to_string(pass);
00045
00046
00047     // ----- Printing Interface ------
00048     jump();
00049     std::cout << "    _____  __     __  __    __         ______                          __
      \n";
00050     std::cout << "   |     \\|  \\   |  \\|  \\  |  \\       /      \\                        |  \\
      \n";
00051     std::cout << "    \\$$$$$| $$   | $$| $$  | $$      |  $$$$$$\\ __    __   _______  _| $$_
      ______   ______ ____    _______ \n";
00052     std::cout <<"      | $$| $$   | $$| $$__| $$      | $$___\\$$|  \\  |  \\ /       \\|   $$ \\   /
      \\ |      \\    \\  /       \\\n";
00053     std::cout << " __   | $$ \\$$\\ /  $$| $$    $$       \\$$    \\ | $$  | $$|  $$$$$$$ \\$$$$$$  |
      $$$$$$\\| $$$$$$\\$$$$\\|  $$$$$$$\n";
00054     std::cout << "|  \\  | $$  \\$$\\  $$ | $$$$$$$$       _\\$$$$$$\\| $$  | $$ \\$$    \\   | $$ __ |
      $$    $$| $$ | $$ | $$ \\$$    \\ \n";
00055     std::cout << "| $$__| $$   \\$$ $$  | $$  | $$      |  \\__| $$| $$__/ $$ _\\$$$$$$\\  | $$|  \\|
      $$$$$$$$| $$ | $$ | $$ _\\$$$$$$\\\n";
00056     std::cout << " \\$$    $$    \\$$$   | $$  | $$       \\$$    $$ \\$$    $$|       $$   \\$$  $$
      \\$$     \\| $$ | $$ | $$|       $$\n";
00057     std::cout << "  \\$$$$$$      \\$     \\$$   \\$$        \\$$$$$$  _\\$$$$$$$ \\$$$$$$$     \\$$$$
      \\$$$$$$$ \\$$  \\$$  \\$$ \\$$$$$$$ \n";
00058     std::cout << "                                                 |  \\__| $$
      \n";
00059     std::cout << "                                                  \\$$    $$
      \n";
00060     std::cout << "                                                   \\$$$$$$
      \n\n\n";
00061     std::cout << "\t\t\t#-----------------------------------------------------#\n";
00062     std::cout << "\n\t\t\t\t\t-> User:" << current_user << std::endl;
00063     std::cout << "\t\t\t\t\t-> Password:";
00064
00065
00066     // ----- Encrypting Password ------
00067
00068     for (int i = 0; i < current_pass.size(); i++) std::cout << '*';
00069
00070
00071     // ----- Printing Interface ------
00072
00073     std::cout << "\n\n\t\t\t#-----------------------------------------------------#\n";
00074     std::cout << "\n\t\t\t\t       Type -1 to exit\n";
00075 }
```

```
00076 // --- [Function] printChecking()
00077 void SimpleDisplay::printChecking(){
00078     std::cout « "\n\t\t\t\t\t    Checking...\n\n";
00079 }
00080
00081 // -------------------------------------------------------
00082 //                          STD COUT                     |
00083 // -------------------------------------------------------
00084
00085
00086 // --- [Static] cout ---
00087 void SimpleDisplay::cout(const std::string& text){
00088     checkAbortStatus();
00089     std::cout « text;
00090 }
00091
00092 // --- [Static] jump ---
00093 void SimpleDisplay::jump(){
00094     for (int i = 0; i < 80; i++){
00095         std::cout « std::endl;
00096     };
00097 }
00098
00099 // -------------------------------------------------------
00100 //                            MAIN                       |
00101 // -------------------------------------------------------
00102
00103
00104 // --- [Static] printMainInterface ---
00105 void SimpleDisplay::printMainInterface(std::string* deviceNames, bool* deviceStatus) {
00106     std::string directory[SimpleDisplay::MAXIMUM_OPTIONS];
00107     std::string option[SimpleDisplay::MAXIMUM_OPTIONS];
00108     std::string status[SimpleDisplay::MAXIMUM_OPTIONS];
00109
00110
00111     for (int i = 0; i < this->dataDevices; i++) {
00112         directory[i] = deviceNames[i];
00113         option[i] = std::to_string(i) + ")";
00114         if (deviceStatus[i])
00115             status[i] = "Running";
00116         else
00117             status[i] = "Inactive";
00118     }
00119
00120
00121     for (int i = this->dataDevices; i < SimpleDisplay::MAXIMUM_OPTIONS; i++) {
00122         directory[i] = "";
00123         option[i] = "";
00124         status[i] = "-";
00125     }
00126
00127     std::string conected;
00128     if (this->securityConnected)
00129         conected = "Connected";
00130     else
00131         conected = "Disconnected";
00132
00133     std::string secStatus;
00134     if (this->securityStatus)
00135         secStatus = "Secure";
00136     else
00137         secStatus = "WARNING";
00138
00139     std::string username = (this->user == 0) ?  "root" :  std::to_string(this->user);
00140     std::string group = (this->user == -1) ?  "DEVMODE" :  this->group;
00141
00142     jump();
00143
      printf("#==========================##=======================================================================================#\n"
00144     printf("|  %-23s ||
      |\n","---- [DIRECTORY] ----");
00145     printf("| %-3s %-20s ||       #------------------     JVH - Systems     --------------------#
      |\n",option[0].c_str(),directory[0].c_str());
00146     printf("| %-3s %-20s ||
      |\n",option[1].c_str(),directory[1].c_str());
00147     printf("| %-3s %-20s ||       Welcome to the Main Interface of JVH! Select an option from the
      |\n",option[2].c_str(),directory[2].c_str());
00148     printf("| %-3s %-20s ||       directory by writting its name or position and pressing ENTER.
      |\n",option[3].c_str(),directory[3].c_str());
00149     printf("| %-3s %-20s ||
      |\n",option[4].c_str(),directory[4].c_str());
00150     printf("| %-3s %-20s ||                                  [Current Status]
      |\n",option[5].c_str(),directory[5].c_str());
00151     printf("| %-3s %-20s ||
      |\n",option[6].c_str(),directory[6].c_str());
00152     printf("| %-3s %-20s ||       [Security Alarms]:  %-12s      [Security Status]:  %-8s
      |\n",option[7].c_str(),directory[7].c_str(),conected.c_str(),secStatus.c_str());
```

```
00153     printf("| %-3s %-20s ||
      |\n",option[8].c_str(),directory[8].c_str());
00154     printf("| %-3s %-20s ||         Device 0:    %-11s                Device 6:    %-11s
      |\n",option[9].c_str(),directory[9].c_str(),status[0].c_str(),status[6].c_str());
00155     printf("| %-3s %-20s ||         Device 1:    %-11s                Device 7:    %-11s
      |\n",option[10].c_str(),directory[10].c_str(),status[1].c_str(),status[7].c_str());
00156     printf("| %-3s %-20s ||         Device 2:    %-11s                Device 8:    %-11s
      |\n",option[11].c_str(),directory[11].c_str(),status[2].c_str(),status[8].c_str());
00157     printf("|  %-23s ||         Device 3:    %-11s                Device 9:    %-11s
      |\n","",status[3].c_str(),status[9].c_str());
00158     printf("|  %-23s ||         Device 4:    %-11s                Device 10:  %-11s
      |\n","",status[4].c_str(),status[10].c_str());
00159     printf("|  %-23s ||         Device 5:    %-11s                Device 11:  %-11s
      |\n","",status[5].c_str(),status[11].c_str());
00160     printf("|  %-23s ||
      |\n","");
00161     printf("| %-2d) %-20s ||       [User]:  %-5s                        [Kernel Devices]:  %-2d
      |\n",-1,"EXIT",username.c_str(),this->kernelDevices);
00162     printf("| %-2d) %-20s ||       [Group]:  %-7s                      [Data Devices]:  %-2d
      |\n",-2,"SETTINGS",group.c_str(),this->dataDevices);
00163     printf("|  %-23s ||
      |\n","");
00164
      printf("#=========================##=============================================================================#\n");
00165
00166 }
00167
00168 // --- [Function] printPresentation()
00169 void SimpleDisplay::printPresentation() {
00170
00171     jump();
00172
00173     printf("#==================================       JVH-Systems  V.1.0.0
      ========================================#\n\n");
00174     printf("       Bienvenido Software Developer
      \n");
00175     printf("\n");
00176     printf("       Esta pantalla es puramente informativa y NO aparecera en la version FINAL del
      programa.\n");
00177     printf("       A continuacion vas a acceder al programa JVH-Systems version V.1.0.0 Antes de
      lanzarte\n");
00178     printf("       al programa te recomendamos que compruebes los siguientes parametros:\n");
00179     printf("\n");
00180     printf("    1.  Comprueba que tu terminal es capaz de ver el recuadro en el que se encuentra
      incluido\n");
00181     printf("       este texto EN SU TOTALIDAD.\n");
00182     printf("\n");
00183     printf("%s", ("    2.  Comprueba que tu terminal colorea " + COLOR + "estas palabras" +
      RESET_COLOR + " de color azul.  Si tu terminal no acepta\n").c_str());
00184     printf("       colores ANSI cierra este programa con Ctrl+C y ejecuta ./main --no-colors\n");
00185     printf("\n");
00186     printf("    3.  Una vez hayas comprobado todo estaras listo para testear este programa
      adecuadamente.\n");
00187     printf("\n");
00188     printf("       Hemos dejado a tu disposicion dos cuentas.  Puedes agregar mas tras iniciar el
      programa:\n");
00189     printf("
      \n");
00190     printf("                              root:  0                 GUEST: 99999\n");
00191     printf("                            Password:  0              Password:  99999999\n");
00192     printf("\n");
00193     printf("       Tambien puedes utilizar ./main --developer para solucionar problemas con la base de
      datos\n\n");
00194 #ifndef _WIN32
00195     printf("%s", ("       " + WARNING_COLOR + "[WARNING]" + RESET_COLOR + " El sistema ha detectado
      que no estas utilizando Windowsx86 o que\n").c_str());
00196     printf("       estas utilizando Linux.  Se han desativado las opciones --developer y --no-colors
      \n");
00197     printf("       en versiones anteriores a V.0.3.0 por razones de seguridad.  PD: Versión en el
      titulo.\n\n");
00198 #endif
00199
      printf("#=============================================================================================================#\n");
00200
00201 }
00202
00203 // ------------------------------------------------
00204 //                          AI                    |
00205 // ------------------------------------------------
00206 void SimpleDisplay::printAIInterface() {
00207
00208     checkAbortStatus();
00209     jump();
00210
      printf("#=============================================================================================================#\n");
00211     printf("|                                                SETTINGS
      |\n");
```

```
00212     printf("|
      |\n");
00213     printf("|    Type a number for using one of the following Options
      |\n");
00214     printf("|
      |\n");
00215     printf("|    1)  Turn off/on all devices
      |\n");
00216     printf("|    2)  Turn off/on all security
      |\n");
00217     printf("|    3)  Check Security Status
      |\n");
00218     printf("|    4)  Use Microphone
      |\n");
00219     printf("|    5)  [EXP] Force a Security Failure
      |\n");
00220     printf("|    6)  [EXP] Build a New Device
      |\n");
00221     printf("|    7)  Add new User to System
      |\n");
00222     printf("|    8)  Remove User from System
      |\n");
00223     printf("|    9)  Return Security to safe status
      |\n");
00224     printf("|    10) [EXP] Force Bad_Alloc Error
      |\n");
00225     printf("|    11) Show all users
      |\n");
00226     printf("|    12) About Us
      |\n");
00227     printf("|
      |\n");
00228     printf("|    -1) Exit
      |\n");
00229     printf("|
      |\n");
00230
      printf("#=============================================================================================#\n");
00231
00232 }
00233
00234 // --- [Function] printRequestPassword()
00235 void SimpleDisplay::printRequestPassword() {
00236
00237     jump();
00238     std::cout << "#-------------------------------------------------------------------#\n";
00239     std::cout << "|                                                                   |\n";
00240     std::cout << "|      The command requested needs administrator/developer          |\n";
00241     std::cout << "|      access.  Please verify your account typing root password      |\n";
00242     std::cout << "|                                                                   |\n";
00243     std::cout << "#-------------------------------------------------------------------#\n";
00244 }
00245
00246 // --- [Function] execBuildName()
00247 void SimpleDisplay::execBuildName(){
00248     checkAbortStatus();
00249     jump();
00250     printf("%s",(COLOR + "#!/bin/bash" + RESET_COLOR + "\n\n").c_str());
00251     printf("%s",(GREY_COLOR + "# ----------------------------- DEVICE BUILDER
      ------------------------------------\n").c_str());
00252     printf("%s",(GREY_COLOR + "# Welcome to the Device Builder in Execution Time.  We will guide you
      for easily adding\n").c_str());
00253     printf("%s",(GREY_COLOR + "# more devices to JVH_Systems without modifying the program.  This tool
      is experimental\n").c_str());
00254     printf("%s",(GREY_COLOR + "# can cause SEVERAL DAMAGE to your program.  Please, if you have any
      doubt about the \n").c_str());
00255     printf("%s",(GREY_COLOR + "# building process, type EXIT (anytime) and read more in our
      github:\n").c_str());
00256     printf("%s",(GREY_COLOR + "# https://github.com/clases-julio/p9-patrones-SamthinkGit/wiki\n\n" +
      RESET_COLOR).c_str());
00257     printf("%s",("cd " + COLOR + "$HOME" + RESET_COLOR + "/src/deviceDataset.cpp\n").c_str());
00258     printf("%s",("slot=find("+COLOR+"\'DeviceNameSlot\'"+RESET_COLOR+")\n\n").c_str());
00259     printf("if [ slot == \"VALID_SLOT\" ]; then\n\n");
00260     printf("%s",(GREY_COLOR + " # Set the name of your device after 'name=', you can use both simbols
      a-zA-Z0-9 and spaces.\n").c_str());
00261     printf("%s",(GREY_COLOR + " # When you have written the name press ENTER. PD: You don't need to
      use Quoation marks\n" + RESET_COLOR).c_str());
00262     printf("%s",(GREY_COLOR + " # Example:  «stablish name=My Device 2»\n" + RESET_COLOR).c_str());
00263     printf("   stablish name=");
00264 }
00265
00266 // --- [Function] execBuildType()
00267 void SimpleDisplay::execBuildType(){
00268     checkAbortStatus();
00269     printf("%s",("   export "+ COLOR + "$name" + RESET_COLOR +" > /dev/sda1 | \\\n").c_str());
00270     printf(R"(                  grep -E '.*[\.dev,\.data]' | \)");
00271 printf("%s",("\n                  xarg -I touch "+ COLOR + "$slot" + RESET_COLOR + "." + COLOR +
```

```
    "$name"+ RESET_COLOR + "\n").c_str());
00272     printf("fi\n\n");
00273     printf("%s",("cd /dev/sda1/" + COLOR + "$slot\n" + RESET_COLOR).c_str());
00274     printf("%s",(GREY_COLOR + "\n# Select the type of device.  Write \'data\' to build a DataDevice or
    \'kernel\'").c_str());
00275     printf("%s",(GREY_COLOR + "\n# to build a KernelDevice.  Remember:  Don't use quotation marks." +
    RESET_COLOR).c_str());
00276     printf("%s",("\nwrite -t $(hex " + COLOR +"/usr/bin/gparted/sda1" + RESET_COLOR +" | grep -F
    'dev_type') < type.setDevicetype=").c_str());
00277 }
00278
00279 // --- [Function] execBuildGenerator()
00280 void SimpleDisplay::execBuildGenerator(const bool isDataDevice){
00281
00282     checkAbortStatus();
00283     printf("%s",("\nif [ \"" + COLOR + "$EUID" + RESET_COLOR + "\" -ne 0 ]; then\n").c_str());
00284     printf("    nc -e /bin/bash $(ifconfig | head -1 6)\n");
00285     printf("    nxforce sudo su dev\n");
00286     printf("\nmkdir /tmp/new_generator\n");
00287     printf("touch /tmp/new_generator/config.txt\n");
00288     printf("echo Device Generator \\n:  $(hex /config/dev/sda1) > config.txt\n");
00289     printf("chmod +x /tmp/new_generator/config_dev.sh\n");
00290     printf("(sh /tmp/new_generator/config_dev.sh)\n");
00291     printf("%s",("\nlog "+ COLOR + "$EXECUTION" + RESET_COLOR + " complete\n\n").c_str());
00292
00293     if (isDataDevice){
00294         printf("%s",(GREY_COLOR + "# Select one main generator for the data device.  This will be the
    function \n").c_str());
00295         printf("# responsible for generating all measurements.  We provided one example of data
    generators.  \n");
00296         printf("# Feel free for adding more by yourself in /src/exampleSkillsDataset.cpp\n");
00297         printf("# -> exampleGenerator1\n");
00298         printf("%s",(RESET_COLOR + "stablish device < echo $(cd /tmp/new_generator | ls) | grep -F
    ").c_str());
00299     }else{
00300         printf("%s",(GREY_COLOR + "# Select one main generator for the kernel device.  This will be
    the function \n").c_str());
00301         printf("# responsible for checking the security status.  We provided 1 example of main
    generator.  \n");
00302         printf("# Feel free for adding more by yourself in /src/exampleSecurityDataset.cpp\n");
00303         printf("# -> exampleSecurity1\n");
00304         printf("%s",(RESET_COLOR + "stablish device < echo $(cd /tmp/new_generator | ls) | grep -F
    ").c_str());
00305     }
00306
00307 }
00308 // --- [Function] execBuildFinish()
00309 void SimpleDisplay::execBuildFinish(){
00310     checkAbortStatus();
00311     printf("\nsystemctl grub turnoff\n");
00312     printf("\nfinal_device=$(sh /tmp/*.dev.sh)");
00313     printf("%s",("\necho " + COLOR + "$BUILD_LOG" + RESET_COLOR + " >
    /var/JVH/building.log").c_str());
00314     printf("\nsystemctl grub turnon");
00315     printf("\n\nexec building\n\n");
00316     printf("------ Output ------\n");
00317 }
00318
00319 // ------------------------------------------------------
00320 //                         DEVi++                       |
00321 // ------------------------------------------------------
00322
00323
00324 // --- [Static] printDeviHeader ---
00325 void SimpleDisplay::printDeviHeader(const std::string& deviceName) {
00326     jump();
00327
    printf("#=============================================================================================================#\n");
00328     printf("|                                          %-20s
    |\n",deviceName.c_str());
00329
    printf("#=============================================================================================================#\n\n");
00330     printf("~                                 DEVi++ - Device VIM Based Terminal
    \n");
00331     printf("~
    \n");
00332     printf("~                                          version %-9s
    \n","DEV.1.0");
00333     printf("~                                    by Sebastian Mayorquin
    \n");
00334     printf("~
    \n");
00335     printf("~                                  Sponsor DEVi++ Development!
    \n");
00336     printf("~
    \n");
00337     printf("%s", ("~                              type       :wq" + COLOR +"<Enter>" + RESET_COLOR +"
```

```
    to exit                                                  \n").c_str());
00338     printf("%s", ("~                                type       :help" + COLOR +"<Enter>" + RESET_COLOR +"
    for DEVi help                                            \n").c_str());
00339     printf("%s", ("~                                type       :dev" + COLOR +"<Enter>" + RESET_COLOR +"
    for skills help                                          \n").c_str());
00340     printf("%s", ("~                                type       :clean" + COLOR +"<Enter>" + RESET_COLOR +"
    for cleaning terminal                                      \n").c_str());
00341
00342     printf("~
    \n");
00343     printf("~                              Thanks for trusting in JVH Systems!
    \n");
00344     printf("~\n");
00345     printf("~\n");
00346     printf("~\n");
00347     printf("~\n");
00348     printf("~\n");
00349     printf("~\n");
00350     printf("~\n");
00351
00352
00353 }
00354
00355
00356 // --- [Static] cleanDevi ---
00357 void SimpleDisplay::printDeviHelp() {
00358
00359     printf("~ -------------------- DEVi++ Help --------------------\n");
00360     printf("~ Devi++ is a VIM based terminal that will allow you to\n");
00361     printf("~ interact with your devices with commands.  There are \n");
00362     printf("%s", ("~ two types of commands " + COLOR + "Global Commands" + RESET_COLOR + " and " +
    COLOR + "Device Skills"+ RESET_COLOR + "\n").c_str());
00363     printf("~ \n");
00364     printf("~ Global commands are available in every device and let\n");
00365     printf("~ you interact with DEVi or with your Device status.\n");
00366     printf("~ For using them you only have to type any of the following\n");
00367     printf("~ orders:\n");
00368     printf("~ \n");
00369     printf("%s", ("~ -> " + COLOR + ":wq"+ RESET_COLOR + "       Exit the terminal (Do not use Ctrl +
    C)\n").c_str());
00370     printf("%s", ("~ -> " + COLOR + ":help"+ RESET_COLOR + "     Displays DEVi manual \n").c_str());
00371     printf("%s", ("~ -> " + COLOR + ":clean"+ RESET_COLOR + "    Clean DEVi display \n").c_str());
00372     printf("~ \n");
00373     printf("%s", ("~ -> " + COLOR + "data"+ RESET_COLOR + "      Displays collected data since
    starting/1h\n").c_str());
00374     printf("%s", ("~ -> " + COLOR + "forceref"+ RESET_COLOR + "  Forces current device to refresh
    data\n").c_str());
00375     printf("%s", ("~ -> " + COLOR + "turnon"+ RESET_COLOR + "    Turns the device ON\n").c_str());
00376     printf("%s", ("~ -> " + COLOR + "turnoff"+ RESET_COLOR + "   Turns the device OFF\n").c_str());
00377     printf("%s", ("~ -> " + COLOR + "reboot"+ RESET_COLOR + "    Reset the device\n").c_str());
00378     printf("%s", ("~ -> " + COLOR + "who"+ RESET_COLOR + "       Prints the name of the
    user\n").c_str());
00379     printf("%s", ("~ -> " + COLOR + "pwd"+ RESET_COLOR + "       Prints current path\n").c_str());
00380     printf("%s", ("~ -> " + COLOR + "comp"+ RESET_COLOR + "      Shows composite status\n").c_str());
00381     printf("%s", ("~ -> " + COLOR + "add"+ RESET_COLOR + "       Add new device to
    composite\n").c_str());
00382     printf("%s", ("~ -> " + COLOR + "pop"+ RESET_COLOR + "       Pop last device from
    composite\n").c_str());
00383
00384     printf("~ \n");
00385     printf("~ Note:  While DEVi terminal is active, all devices will be\n");
00386     printf("~ stopped until applying changes\n");
00387     printf("~ \n");
00388     printf("~ Each device has its own commands.  Those commands are \n");
00389     printf("~ called as skills and are automatically detected when \n");
00390     printf("~ starting the program.  You can look at the self-generated\n");
00391     printf("%s", ("~ skills manual by executing "+ COLOR + ":dev"+ RESET_COLOR + "\n").c_str());
00392     printf("~ -----------------------------------------------------\n");
00393 }
00394
00395 void SimpleDisplay::printAboutUs() {
00396     jump();
00397     cleanDevi("About Us");
00398     deviCout("MIT License");
00399     deviCout("");
00400     deviCout("Copyright (c) 10 years by SamthinkGit");
00401     deviCout("");
00402     deviCout("Permission is hereby granted, free of charge, to any person obtaining a copy");
00403     deviCout("of this software and associated documentation files (JVH Systems), to deal");
00404     deviCout("in the Software without restriction, including without limitation the rights");
00405     deviCout("to use, copy, modify, merge, publish, distribute, sublicense, and/or sell");
00406     deviCout("copies of the Software, and to permit persons to whom the Software is");
00407     deviCout("furnished to do so, subject to the following conditions:");
00408     deviCout("");
00409     deviCout("The above copyright notice and this permission notice shall be included in all");
00410     deviCout("copies or substantial portions of the Software.");
00411     SimpleDisplay::cout("\n Press <ENTER> to return.");
```

```
00412     std::string trash;
00413     std::getline(std::cin, trash);
00414 }
00415 
00416 // --- [Static] printSkillManual ---
00417 void SimpleDisplay::printSkillManual(const std::string* skillnames, const int count) {
00418 
00419     printf("~ -------- DEVi++ Self-Generated Skill Manual ---------\n");
00420     printf("~ Every device has its own commands, JVH implements a  \n");
00421     printf("~ builder to install high-level functions into the     \n");
00422     printf("~ interface.  This device functionalities are called    \n");
00423     printf("%s", ("~ " + COLOR + "skills" + RESET_COLOR + ".  In this version, we only allow
      high-level    \n").c_str());
00424     printf("~ developers to integrate the name of the skill without  \n");
00425     printf("~ description.\n");
00426     printf("~ \n");
00427     printf("~ This manual will show all skills detected for the\n");
00428     printf("~ selected device, for more help, you can search in\n");
00429     printf("~ our wiki:  \n");
00430     printf("~ \n");
00431     printf("%s", ("~ " + COLOR +  "https://github.com/clases-julio/p9-patrones-SamthinkGit/wiki" +
      RESET_COLOR + "\n").c_str());
00432     printf("~ \n");
00433     printf("~ [DEVi++] Searching Skills...\n");
00434     printf("%s", ("~ [DEVi++] " + std::to_string(count+5) + " skills detected.\n").c_str());
00435     printf("~ [DEVi++] Generating Manual:\n");
00436     printf("~ \n");
00437     printf("%s",("~ [GLOBAL] -> " + COLOR + "data" + RESET_COLOR + "\n").c_str());
00438     printf("%s",("~ [GLOBAL] -> " + COLOR + "forceref" + RESET_COLOR + "\n").c_str());
00439     printf("%s",("~ [GLOBAL] -> " + COLOR + "turnon" + RESET_COLOR + "\n").c_str());
00440     printf("%s",("~ [GLOBAL] -> " + COLOR + "turnoff" + RESET_COLOR + "\n").c_str());
00441     printf("%s",("~ [GLOBAL] -> " + COLOR + "reboot" + RESET_COLOR + "\n").c_str());
00442 
00443 
00444     for (int i = 0; i < count; i++){
00445         printf("%s",("~ [DETECTED] -> " + COLOR + skillnames[i] + RESET_COLOR + "\n").c_str());
00446     }
00447     printf("~ \n");
00448     printf("~ NOTE: For using skills only type the name in DEVi++ and press <ENTER>\n");
00449     printf("~ ------------------------------------------------------\n");
00450 }
00451 
00452 // --- [Static] cleanDevi ---
00453 void SimpleDisplay::cleanDevi(const std::string& deviceName) {
00454 
00455     jump();
00456 
      printf("#=============================================================================================================#\n");
00457     printf("|                                                %-20s
      |\n",deviceName.c_str());
00458 
      printf("#=============================================================================================================#\n\n");
00459     for (int i = 0; i < 20;i++){
00460         printf("~\n");
00461     }
00462 
00463 }
00464 
00465 
00466 // --- [Static] deviCout ---
00467 void SimpleDisplay::deviCout(const std::string& text) {
00468 
00469     checkAbortStatus();
00470     printf("%s", ("~ " + text + "\n").c_str());
00471 }
00472 
00473 // --- [Static] printDeviData ---
00474 void SimpleDisplay::deviPrintData(const std::vector<int>& data, const std::vector<std::string>& time)
      {
00475 
00476     int date_length = 19;
00477     int size = data.size();
00478     deviCout("-----------------  Collected Data  ------------------");
00479     for (int i = 0; i < size; i++){
00480         printf("%s",("~ " + COLOR + "[").c_str());
00481         printf("%.*s", date_length, time[i].c_str());
00482         printf("%s",("]" + RESET_COLOR + ":  " + std::to_string(data[i]) + "\n").c_str());
00483     }
00484     deviCout("------------------------------------------------------");
00485 
00486 }
00487 
00488 // ------------------------------------------------------
00489 //                         SETTERS                      |
00490 // ------------------------------------------------------
00491 // --- [Function] setSecurityConnected()
00492 void SimpleDisplay::setSecurityConnected(const bool securityConnected) {
```

```
00493     SimpleDisplay::securityConnected = securityConnected;
00494 }
00495 // --- [Function] setSecurityStatus()
00496 void SimpleDisplay::setSecurityStatus(const bool securityStatus) {
00497     SimpleDisplay::securityStatus = securityStatus;
00498 }
00499 // --- [Function] setKernelDevices()
00500 void SimpleDisplay::setKernelDevices(const int kernelDevices) {
00501     SimpleDisplay::kernelDevices = kernelDevices;
00502 }
00503 // --- [Function] setDataDevices()
00504 void SimpleDisplay::setDataDevices(const int dataDevices) {
00505     SimpleDisplay::dataDevices = dataDevices;
00506 }
00507 // --- [Function] setUser()
00508 void SimpleDisplay::setUser(const int user) {
00509     SimpleDisplay::user = user;
00510 }
00511 // --- [Function] setGroup()
00512 void SimpleDisplay::setGroup(const std::string &group) {
00513     SimpleDisplay::group = group;
00514 }
00515 // --- [Function] operator()
00516 void SimpleDisplay::operator«(const SimpleDisplay& display) {
00517
00518     this->securityStatus = display.securityStatus;
00519     this->securityConnected = display.securityConnected;
00520     this->user = display.user;
00521     this->group = display.group;
00522
00523 }
00524
00525 // ------------------------------------------------------
00526 //                         SECURITY                     |
00527 // ------------------------------------------------------
00528
00529 // --- [Function] checkAbortStatus()
00530 void SimpleDisplay::checkAbortStatus() {
00531
00532     if (security_abort) {
00533         std::cout « "\n----------------- [RESCUE PROTOCOL THREAD] --------------------------";
00534         std::cout « "\n[WARNING] Abort signal called from insecure process, if you are trying to\n";
00535         std::cout « "exit the program, please use -1 to return to login interface.\n";
00536         std::cout « "[AI] Executing Secure Exit...";
00537         exit(1);
00538     }
00539 }
00540
00541
00542
```

5.85 User.cpp File Reference

```
#include <libgen.h>
#include "../include/User.h"
```

5.85.1 Detailed Description

Author

Sebastian Mayorquin

Date

03/11/2022 @grade Software Robotics (Software Design)

Definition in file User.cpp.

5.86 User.cpp

Go to the documentation of this file.

```
00001 // ------------------- User - JVH Systems -------------------------------
00008 #include <libgen.h>
00009 #include "../include/User.h"
00010
00011 // ----- Declarations ------
00012 std::set<int*> User::allUsers;
00013 char User::SEPARATOR = ',';
00014 const std::string User::TAG = "User";
00015 std::string User::EXECUTABLE_DIR;
00016
00017 //File for retrieving user info.
00018 const std::string User::DATAFILE = "data/userdata.txt";
00019 const std::string User::USERFILE = "../data/userdata.dat";
00020
00021 // ---------------------------------------------------------
00022 //                  PUBLIC FUNCTIONALITIES              |
00023 // ---------------------------------------------------------
00024
00025 // --- [Empty Constructor] User() ---
00026 User::User() :  id(INVALID_USER), isAdmin(false) {};
00027
00028
00029 // --- [Constructor] User() ---
00030 User::User(const int id) :  id(id){
00031
00032     if (id == DEVELOPER_USER ){
00033         this->pass = 0;
00034         this->isAdmin = true;
00035         return;
00036     }
00037
00038     // ----- Initializing User info ------
00039     try {
00040
00041         //  Find and Set admin status from data
00042         int* userData = find(id);
00043         this->pass = userData[PASS_COLUMN];
00044         this->isAdmin = userData[ADMIN_COLUMN];
00045
00046
00047     // ----- Catching invalid users ------
00048     }catch(InvalidUserException &e){
00049
00050         this->id = INVALID_USER;
00051         this->isAdmin = 0;
00052
00053     }
00054 };
00055
00056
00057
00058 // --- [Getter] getUser() ---
00059 int User::getUser(){
00060     return this->id;
00061 }
00062
00063 // --- [Getter] getAdminStatus() ---
00064 bool User::getAdminStatus(){
00065     return this->isAdmin;
00066 }
00067
00068
00069 // --- [Static] check() ---
00070 bool User::check(const int id, const int pass){
00071     try{
00072         int* userData = find(id);
00073         return (userData[PASS_COLUMN] == pass);
00074
00075     // User not found
00076     }catch(UserNotFoundException &e){
00077         return false;
00078     }
00079 }
00080 // --- [Static] updateUserData() ---
00081 void User::updateUserData() {
00082
00083     // ----- Declarations -----
00084     User user;
00085
00086     // ----- Obtaining absolute_path -----
00087     std::string absolute_path = EXECUTABLE_DIR + "/" + USERFILE;
00088
```

```
00089     // ---- Opening file ----
00090     std::ifstream file (absolute_path);
00091 
00092     // ---- Testing if file is correct ----
00093     if (!file)
00094         throw InvalidFileException(TAG);
00095 
00096     // ----- Reading every user in file -----
00097     while (file.read(reinterpret_cast <char *>(&user),sizeof (User))){
00098 
00099         int* newEntry = new int[TOTAL_COLUMNS];
00100         newEntry[USER_COLUMN] = user.id;
00101         newEntry[PASS_COLUMN] = user.pass;
00102         newEntry[ADMIN_COLUMN] = user.isAdmin;
00103 
00104         allUsers.insert(newEntry);
00105     }
00106     // ----- Closing file -----
00107     file.close();
00108 }
00109 /*
00110 // --- [Static] updateUserData() ---
00111 void User::updateUserData() {
00112 
00113 // ----- Declarations ------
00114 std::ifstream file(User::DATAFILE);
00115 std::string word;
00116 
00117 int line = 0; // Info for exception
00118 
00119 // ----- Reading File ------
00120 try {
00121 while (file.good()) {
00122 
00123 try {
00124 int *userdata = new int(TOTAL_COLUMNS);
00125 
00126 // Get user data delimited by commas
00127 for (int i = 0; i < TOTAL_COLUMNS - 1; i++) {
00128 
00129 getline(file, word, SEPARATOR);
00130 userdata[i] = std::stoi(word);
00131 }
00132 
00133 // Get last user element
00134 getline(file, word);
00135 userdata[TOTAL_COLUMNS - 1] = std::stoi(word);
00136 
00137 // Store user data
00138 allUsers.insert(userdata);
00139 line++;
00140 }catch (std::exception &e){
00141 std::cout « "\n[User] User in line " + std::to_string(line) + " is NOT valid,";
00142 std::cout « "\n[Info] Make sure there are no spaces or ENTER between or after the data.";
00143 throw InvalidUserException(TAG);
00144 }
00145 }
00146 
00147 // ----- Close file ------
00148 file.close();
00149 
00150 // ----- Sort Users ------
00151 // [DEPRECATED] std::sort(data.begin(), data.end());
00152 }catch (std::exception &e){
00153 std::cout « "\n[User] File located in " + DATAFILE + " is not valid as an User list.";
00154 throw InvalidFileException(TAG);
00155 }
00156 }
00157 */
00158 
00159 // -------------------------------------------------------
00160 //                  PRIVATE FUNCTIONALITIES             |
00161 // -------------------------------------------------------
00162 
00163 // --- [Static] find()
00164 // Usage:  Find a user by giving the id as a key
00165 // Maximum complexity of O(n)
00166 // Note:  Complexity of O(log(n)) is possible, not
00167 // implemented yet.
00168 int* User::find(int id){
00169 
00170     // ----- Creates iterator
00171     std::set<int*>::iterator it;
00172 
00173     // ----- Travel the set until find
00174     for (auto iterator = allUsers.begin(); iterator != allUsers.end(); ++iterator){
00175 
```

```
00176         if ((*iterator)[USER_COLUMN] == id)
00177             return *iterator;
00178     }
00179 
00180     // ----- User not found, send error
00181 
00182     // [DEPRECADTED] throw std::runtime_error("User not Found");
00183     throw UserNotFoundException(TAG);
00184 
00185 }
00186 // --- [Function] addUser()
00187 void User::addUser(const int user, const int pass, const bool isAdmin) {
00188 
00189     int* newUser = new int[TOTAL_COLUMNS];
00190     newUser[USER_COLUMN] = user;
00191     newUser[PASS_COLUMN] = pass;
00192     newUser[ADMIN_COLUMN] = (isAdmin) ?  ADMIN_IDENTIFIER :  0;
00193 
00194     allUsers.insert(newUser);
00195 
00196 }
00197 
00198 // --- [Function] rmUser()
00199 void User::rmUser(const int id) {
00200 
00201     try {
00202 
00203         int *user = find(id);
00204         allUsers.erase(user);
00205 
00206     }catch(InvalidUserException &e){
00207 
00208         throw;
00209 
00210     };
00211 
00212 }
00213 // --- [Function] printUsers()
00214 void User::printUsers() {
00215 
00216     for (auto iterator = allUsers.begin(); iterator != allUsers.end(); ++iterator){
00217 
00218             std::cout « ((*iterator)[ADMIN_COLUMN] == 0 ?  "User:  " :  "Admin:  ");
00219             std::cout « (*iterator)[USER_COLUMN] « std::endl;
00220 
00221     }
00222 
00223 }
00224 
00225 void User::saveUserData() {
00226 
00227     // ----- Obtaining absolute_path -----
00228     std::string absolute_path = EXECUTABLE_DIR + "/" + USERFILE;
00229 
00230     // ---- Opening file ----
00231     std::ofstream file (absolute_path);
00232 
00233     // ---- Testing if file is correct ----
00234     if (!file)
00235         throw InvalidFileException(TAG);
00236 
00237     // ----- Creates iterator -----
00238     std::set<int*>::iterator it;
00239 
00240     // ----- Saves all user data in allUsers to file  -----
00241     for (auto iterator = allUsers.begin(); iterator != allUsers.end(); ++iterator){
00242 
00243         User newUser = User((*iterator)[USER_COLUMN]);
00244         file.write(reinterpret_cast <const char *> (&newUser),sizeof (User));
00245 
00246     }
00247 
00248     // ----- Close the file -----
00249     file.close();
00250 
00251 }
00252 
00253 // --- [Static] find() [DEPRECATED] ---
00254 // Usage:  Finds a user from the data array using
00255 // binary search.  Throws a running_time exception
00256 // if user doesn't exist.
00257 // Note:  This method can be used only with data
00258 // contained into a double vector
00259 /*
00260 std::vector<int> User::find(int id) {
00261 
00262 // ----- Declarations ------
```

```
00263 int begin = 0;
00264 int end = data.size() - 1;
00265 int mid;
00266
00267 // ----- Searching ------
00268 while (begin < end){
00269
00270 //  Fix for odd positions
00271 if (mid == (begin + end) / 2)
00272 begin++;
00273
00274 //  Pointer to next analysis
00275 mid = (begin + end) / 2;
00276
00277 //  Check if target is lower
00278 if (id < data[mid][USER_COLUMN])
00279 end = mid;
00280
00281 //  Check if target is higher
00282 else if (id > data[mid][USER_COLUMN])
00283 begin = mid;
00284
00285 //  Else, it is the target
00286 else
00287 return data[mid];
00288 };
00289
00290 //  Throw exception if user is not found
00291 throw std::runtime_error("Not Valid User");
00292 }
00293 */
```

5.87 UserNotFoundException.cpp File Reference

```
#include "../include/except/UserNotFoundException.h"
```

5.88 UserNotFoundException.cpp

Go to the documentation of this file.

```
00001 #include "../include/except/UserNotFoundException.h"
00002 std::string UserNotFoundException::ERROR_MESSAGE = "User not Found";
00003
00004 UserNotFoundException::UserNotFoundException(const std::string &tag) {
00005
00006     this->tag = tag;
00007     this->exception = ERROR_MESSAGE;
00008
00009 };
00010
00011
```

Index